



“Think like a FAANG interviewer before writing a single line of code.”

1 PLAIN ENGLISH

A Graph is a collection of nodes connected by edges.

Trees are special graphs.

Grids are graphs.

Most interview problems hide graphs.

3 BRAIN SWITCH

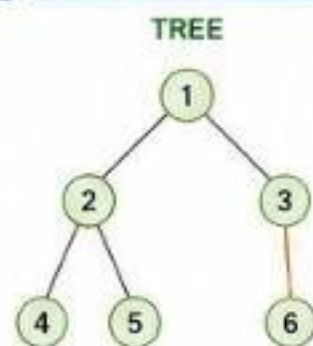
Interview gives...

- Cities & Roads
- Friends
- Flights
- Courses
- Computer Network
- Maze
- Dependencies
- Social Network

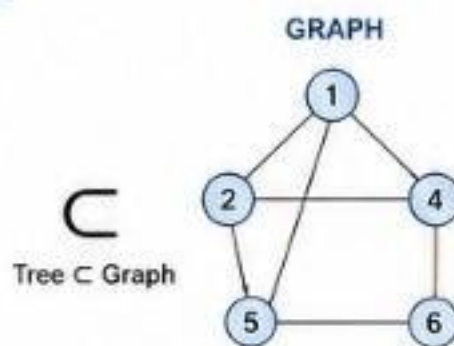


THINK
GRAPH

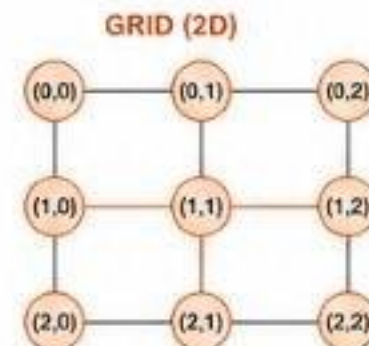
2 VISUAL EXAMPLES



No Cycles, Connected



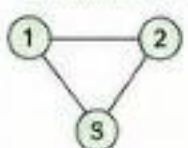
May Have Cycles,
May Be Disconnected



Cells as Nodes,
Neighbors as Edges

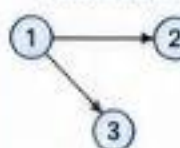
4 GRAPH TYPES

UNDIRECTED



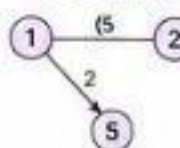
Edges have
no direction.
Ex: Friends

DIRECTED



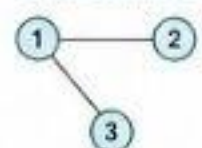
Edges have
direction.
Ex: Flights

WEIGHTED



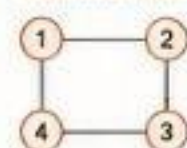
Edges have
weights/costs.
Ex: Roads

UNWEIGHTED



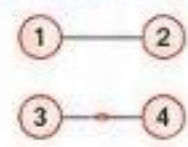
All edges have
equal weight.
Ex: Social Network

CONNECTED



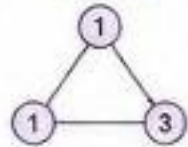
There is a path
between every
pair of nodes.
Ex: Network

DISCONNECTED



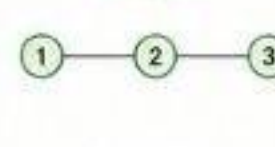
Not all nodes are
reachable.
Ex: Components

CYCLIC



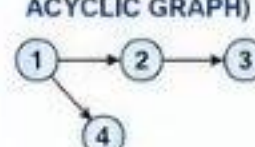
Contains at least
one cycle.
Ex: Dependencies

ACYCLIC



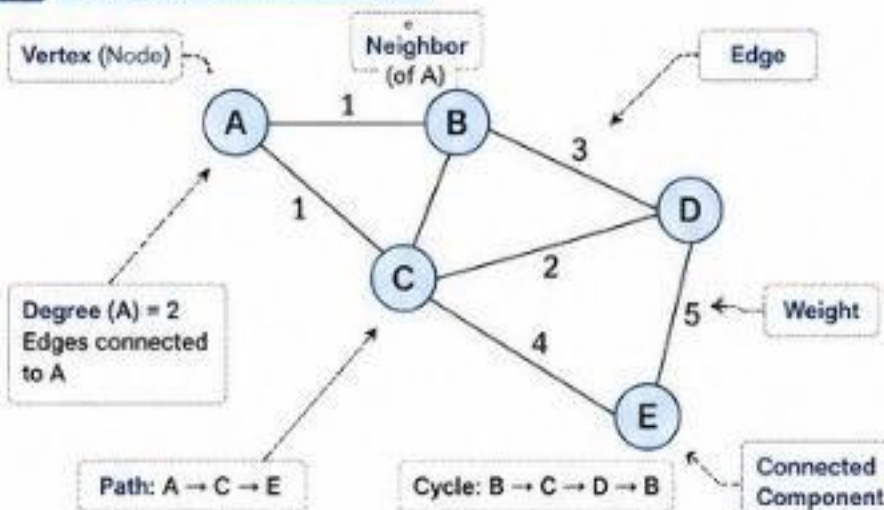
No cycles.
Ex: Tree

DAG (DIRECTED ACYCLIC GRAPH)



Directed and
no cycles.
Ex: Course Schedule

5 GRAPH TERMINOLOGY



6 TREE vs GRAPH

Aspect	TREE	GENERAL GRAPH
Root	Has a root	No root (usually)
Parent	Each node (except root) has exactly one parent	A node can have multiple parents
Neighbor	Limited by parent-child relationship	Any connected node is a neighbor
Cycles	No cycles	May have cycles
Connected	Always connected	May be connected or disconnected
Edges	For n nodes, edges = n - 1	Edges can be any number
Traversal	DFS / BFS works	DFS / BFS works
Need visited[]	Usually Not Needed	Usually Needed
Interview Examples	Binary Tree, N-ary Tree	Social Network, Roads, Flights, Dependencies

A Tree is always a Graph.

A Graph is NOT always a Tree.

8 AHA MOMENTS

- A Tree is a Graph.
- A Grid is a Graph.
- Representation changes. Algorithm usually doesn't.
- Interview problems rarely say "Graph".
- Always identify the hidden graph first.

7 GRAPH REPRESENTATIONS (OVERVIEW)

Representation	Typical Input	Interview Usage	Memory	Highlight
Edge List	List of edges [u, v, w]	Kruskal, Some Graph Problems	$O(E)$	Simple to read
Adjacency List	[[1,2], [2,3], ...]	Most Common in Interviews	$O(V + E)$	★★★★★ Most commonly used
Adjacency Matrix	2D Matrix [n x n]	Dense Graphs, Quick Edge Check	$O(V^2)$	Good for dense graphs
Grid	2D Characters / Integers	Islands, Paths, Flood Fill	$O(m \times n)$	Grid treated as Graph
Node Graph	Nodes with Neighbors	Clone Graph, Complex Structures	$O(V + E)$	Used in advanced problems

★ Adjacency List is the most commonly used representation in interviews.

9 PATTERN RECOGNITION

Interview Says	Think	Expected Graph Type
Cities / Roads	Graph	Undirected, Often Weighted
Flights	Graph	Directed, Often Weighted
Courses / Prerequisites	Graph	Directed, DAG
Grid / Matrix	Graph	Grid Graph (4 / 8 direction)
Network (Servers / Computers)	Graph	Undirected or Directed
Friends / Social Network	Graph	Undirected, Unweighted
Maze / Labyrinth	Graph	Grid Graph
Dependencies / Build Order	Graph	Directed, DAG

10 COMMON INTERVIEW MISTAKES

- Forgetting graphs may be disconnected.
- Confusing Tree with Graph.
- Ignoring direction (Directed vs Undirected).
- Ignoring weights.
- Choosing wrong representation.
- Thinking Grid is different from Graph.
- Not defining neighbors clearly.
- Forgetting to validate with small examples.

11 INTERVIEW THINKING

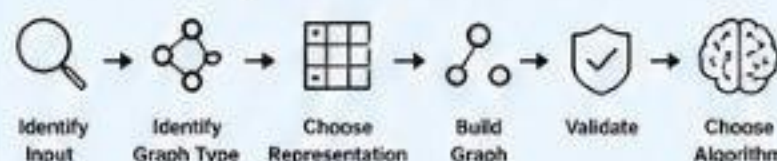
Before writing code ask:

- What is the input?
- Is it Directed?
- Is it Weighted?
- Is it Connected?
- Which Representation should I build?
- Which Algorithm family will probably solve it?

12 SUMMARY

REMEMBER

Graph problems are solved in this order.





— "Before choosing an algorithm, choose the helper structures that power it." —

1 PLAIN ENGLISH



Almost every graph problem uses helper structures.



Choosing the right helper structure is often more important than choosing the algorithm.



Build first.
Choose helper structures.
Then solve.

2 THE GRAPH TOOLBOX (Helper Structures You'll Use in Almost Every Problem)

Structure	Purpose (One-line)	Used In (Typical Problems)	Java Type	Initialization (Example)	Validation / What to Check
★ visited[]	Track visited nodes	DFS, BFS, Cycle Detect, Components	boolean[]	boolean[] visited = new boolean[n];	Print visited nodes (true indices)
★ dist[]	Store shortest distance from source	Shortest Path, Dijkstra, Multi-Source BFS	int[]	int[] dist = new int[n]; Arrays.fill(dist, INF);	Print distances (Arrays.toString)
parent[]	Store parent / predecessor of each node	Path Reconstruction, Union Find, Trees	int[]	int[] parent = new int[n]; for (int i=0; i<n; i++) parent[i] = i;	Print parent array
rank[]	Rank / size for Union Find (Optimized Merge)	Union Find (Disjoint Set)	int[]	int[] rank = new int[n]; (default 0)	Print rank array
indegree[]	Count incoming edges	Topological Sort (Kahn's Algo)	int[]	int[] indegree = new int[n];	Print indegree array
dirs[][]	Directions for grid movement	Grid DFS/BFS, Islands, Flood Fill	int[][]	int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};	Print dirs to verify movement
★ Queue	Process nodes in FIFO order	BFS, Level Order, Kahn's Algo	Queue<Integer>	Queue<Integer> q = new LinkedList<>();	Print queue content
★ PriorityQueue	Process best candidate first	Dijkstra, Prim's, Minimum Effort	PriorityQueue<int>	PriorityQueue<int> pq = new PriorityQueue<>();	Print pq content
HashMap	Mapping / Lookup (Visited, Clone, etc.)	Clone Graph, Store Mappings, Count	HashMap<K, V>	HashMap<K, V> map = new HashMap<>();	Print map entries
★ List<List<Integer>>	Adjacency List for Unweighted Graph	Most Graph Problems (95%)	List<List<Integer>>	List<List<Integer>> g = new ArrayList<>();	Print neighbors for each node
List<List<int>>>	Adjacency List for Weighted Graph	Weighted Graphs, Dijkstra, MST	List<List<int>>>	List<List<int>>> g = new ArrayList<>();	Print (neighbor, weight) for each node

★ Most Frequently Used in Interviews

Adjacency List + visited[] + Queue + PriorityQueue = Your Core Toolkit

3 WHEN DO I NEED WHAT?

👁️ Need to avoid revisiting nodes?	→	visited[]
🏔️ Need shortest distance?	→	dist[]
🏆 Need smallest distance first?	→	PriorityQueue
🔄 Need BFS (level by level)?	→	Queue
👥 Need to merge / union groups?	→	parent[], rank[]
📊 Need dependency count?	→	indegree[]
🗺️ Need grid movement?	→	dirs[][]
🔗 Need clone / mapping / lookup?	→	HashMap

4 INITIALIZATION CHEAT SHEET

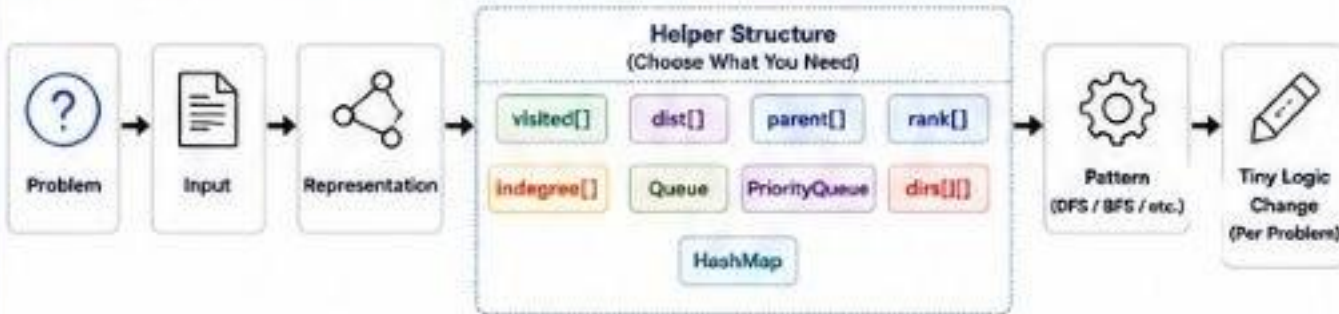
Structure	Java Initialization
visited[]	boolean[] visited = new boolean[n];
dist[]	int[] dist = new int[n]; Arrays.fill(dist, INF);
parent[]	int[] parent = new int[n]; for (int i = 0; i < n; i++) parent[i] = i;
rank[]	int[] rank = new int[n]; // default 0
indegree[]	int[] indegree = new int[n];
Queue	Queue<Integer> q = new LinkedList<>();
PriorityQueue	PriorityQueue<int> pq = new PriorityQueue<>();
HashMap	HashMap<K, V> map = new HashMap<>();
dirs[][]	int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};

5 PRINT & VALIDATE (Always Do This!)

Structure	How to Print	Expected Output
visited[]	Arrays.toString(visited)	[true, false, true, ...]
dist[]	Arrays.toString(dist)	[0, 2, 5, INF, ...]
parent[]	Arrays.toString(parent)	[0, 0, 1, 2, ...]
rank[]	Arrays.toString(rank)	[0, 1, 0, 1, ...]
indegree[]	Arrays.toString(indegree)	[0, 1, 2, 0, ...]
Adjacency List	for (i) print i -> neighbors	0 -> [1,2] 1 -> [0,2,3] ...
Weighted Graph	for (i) print i -> (nbr, w)	0 -> (1,4) (2,7) 1 -> (0,4) ...
PriorityQueue	System.out.println(pq)	[(2,5), (1,8), ...]
Queue	System.out.println(q)	[1, 2, 3, ...]
HashMap	System.out.println(map)	{key1=val1, ...}

⚠️ Never debug DFS/BFS until helper structures look correct.

6 VISUAL MEMORY MAP (How Everything Connects)



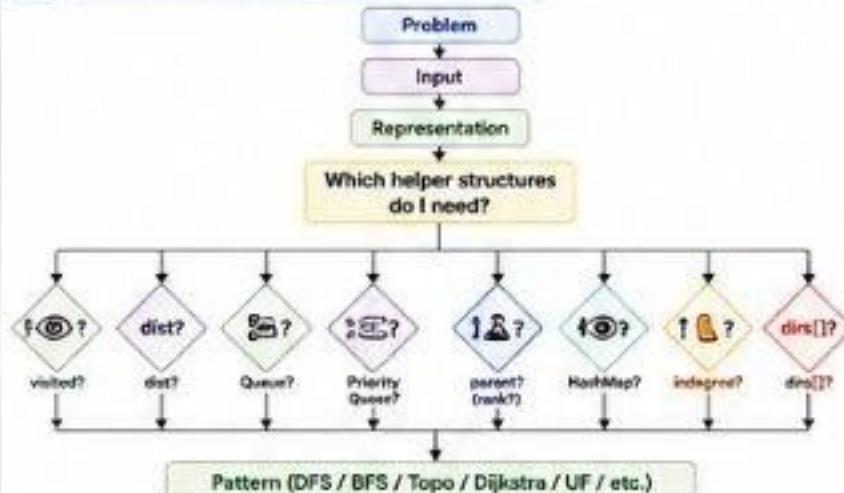
7 AHA MOMENTS

💡 Algorithms usually share helper structures.	💡 Weighted graphs almost always need dist[].
💡 Most DFS problems differ only by return type.	💡 PriorityQueue usually means "best candidate first".
💡 Most BFS problems differ only by queue logic.	💡 Union Find is impossible without parent[].
💡 Grid problems always reuse dirs[].	💡 Topological Sort begins with indegree[].

8 COMMON INTERVIEW MISTAKES

☐ Forgot visited[].	☐ Forgot reverse edge in undirected graph.
☐ Forgot Arrays.fill(dist, INF).	☐ Used Queue instead of PriorityQueue.
☐ Mixed node and distance in structures.	☐ Forgot dirs[].
☐ Forgot parent[i] = i.	☐ Didn't print structures while debugging.

9 INTERVIEW THINKING FLOW (Before Coding)



10 SUMMARY

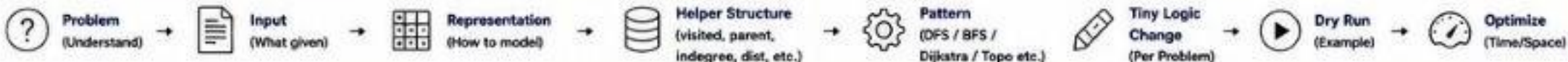
REMEMBER

- ✓ Graph interviews are not won by memorizing algorithms.
- ✓ They are won by choosing the correct helper structures before coding.
- ✓ Build → Helper Structures → Pattern → Tiny Change → Solve.
- ✓ Helper structures are your superpower.



Your Toolkit.
Use it Every Time.

★★★★★





"Every graph interview begins by converting the input into the right representation."

1 PLAIN ENGLISH



Interviewers rarely give you a graph directly.



Instead they give edges, grids, matrices or node objects.



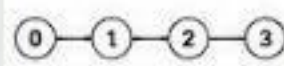
Your first job is to identify the input and convert it into the correct graph representation.

2 INPUT TYPES (What Interviewers Give You)

1 EDGE LIST

Example Input

[[0,1],[1,2],[2,3]]



Typical Problems

- Valid Path
- Connected Components
- Graph Valid Tree

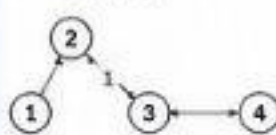
Interview Frequency

★★★★★

2 WEIGHTED EDGE LIST

Example Input

[[2,1,1],
[2,3,1],
[3,4,1]]



Typical Problems

- Network Delay
- Cheapest Flights
- Minimum Effort Path

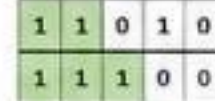
Interview Frequency

★★★★★

3 GRID

Example Input

1 1 0 1 0
1 1 1 0 0



Typical Problems

- Number of Islands
- Flood Fill
- Rotting Oranges
- Pacific Atlantic

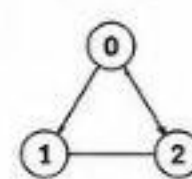
Interview Frequency

★★★★☆

4 ADJACENCY MATRIX

Example Input

0 1 0
1 0 1
0 1 0



Typical Problems

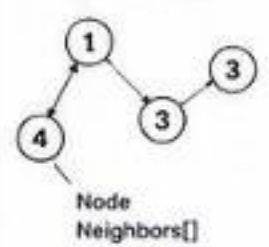
- Number of Provinces
- Dense Graph Problems

Interview Frequency

★★★☆☆

5 NODE GRAPH (OBJECTS)

Example Input



Typical Problems

- Clone Graph

Interview Frequency

★★★☆☆

3 BRAIN SWITCH

INTERVIEW INPUT



Edge List



Build AdjList



Weighted Edge List



Build Weighted AdjList



Grid



Treat Cell as Node



Adjacency Matrix



Convert or Traverse Directly



Node Graph



Use Existing Neighbors



REPRESENTATION FIRST.
ALGORITHM SECOND.

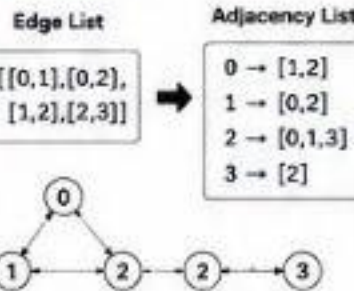
4 GRAPH CONSTRUCTION GUIDE

Input (Interview Gives)	How to Build	Output (Your Representation)	Notes
Edge List [[u,v], ...]	Build Adjacency List (Undirected / Directed)	List<List<Integer>>	Most common representation. Optimal for sparse graphs.
Weighted Edge List [[u,v,w], ...]	Build Weighted Adjacency List	List<List<int[]>> Each int[] = {neighbor, weight}	Use int[] or Pair<node, weight>
Grid matrix[m][n]	Use directions (dirs[]) Treat valid cell as node	Implicit Graph (no explicit structure needed)	Neighbor checks on the fly using dirs[]
Adjacency Matrix matrix[n][n]	Use matrix directly or convert to AdjList	int[][] matrix (or convert to List<List<Integer>>)	Good for dense graphs. O(1) edge check.
Node Graph Node object with neighbors[]	Use existing neighbors directly	Node (given) (no conversion needed)	Already a graph. Just use it.

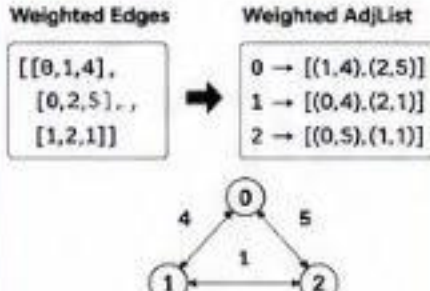
★ Adjacency List is the preferred choice for 95% of interview problems.

5 VISUAL CONVERSION EXAMPLES

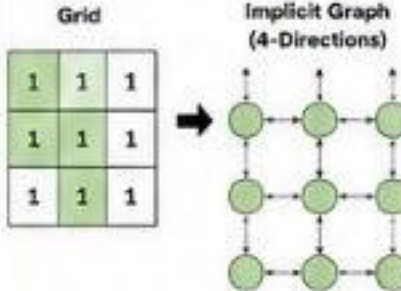
1 Edge List → Adjacency List



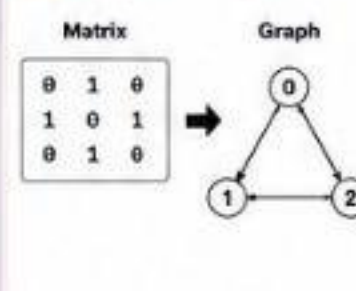
2 Weighted Edge → Weighted AdjList



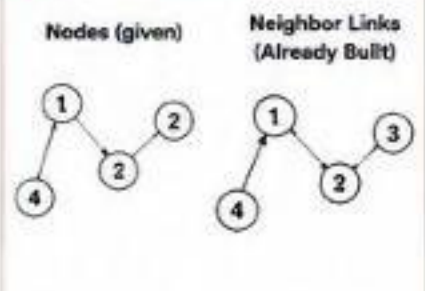
3 Grid → Implicit Graph



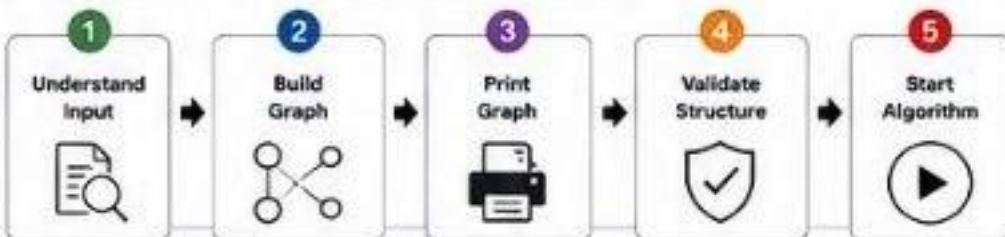
4 Matrix → Graph



5 Node Objects → Graph



6 BUILD → PRINT → VALIDATE (Never Skip This!)



How to Validate Your Graph

Graph Type	How to Validate
Adjacency List	Print each node and its neighbors
Weighted Graph	Print each node with (neighbor, weight)
Grid	Print grid dimensions and values
Matrix	Print the entire matrix
Node Graph	Print each node and neighbor count



IMPORTANT

Never start DFS/BFS
(or any algorithm)
until your graph
structure is verified.

7 COMMON JAVA BUILD TEMPLATES (Construction Only)

1 Build Adjacency List

```
List<List<Integer>>
graph = new ArrayList<>();
for (int i = 0; i < n; i++)
    graph.add(new ArrayList<>());

// For undirected edge (u, v)
graph.get(u).add(v);
graph.get(v).add(u);
```

2 Build Weighted Adjacency List

```
List<List<int[]>>
graph = new ArrayList<>();
for (int i = 0; i < n; i++)
    graph.add(new ArrayList<>());

// Edge (u, v, w)
graph.get(u).add(new int[]{v, w});
graph.get(v).add(new int[]{u, w});
```

3 Directions Array (For Grid)

```
int[][] dirs
int i, j; // down
{-1, 0}, // up
{0, 1}, // right
{0, -1} // left
```

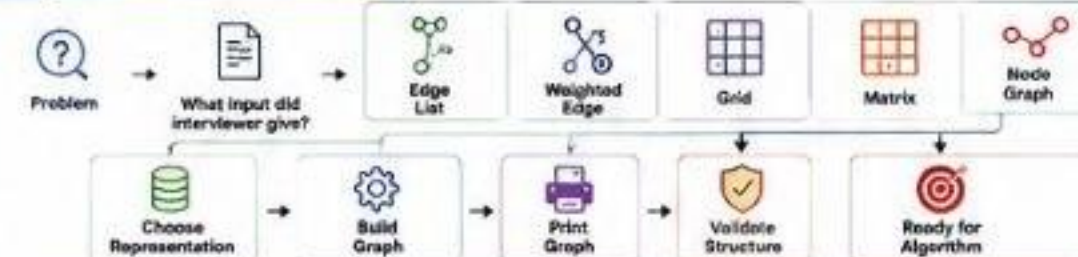
8 AHA MOMENTS

- The algorithm usually doesn't change.
- Only the graph representation changes.
- A Grid is an implicit graph.
- Weighted graphs store (neighbor, weight).
- Node graphs are already built.
- Always build before solving.

9 COMMON INTERVIEW MISTAKES

- Started coding before building the graph.
- Forgot reverse edge in undirected graph.
- Used wrong representation.
- Stored weight incorrectly.
- Forgot 1-index vs 0-index.
- Didn't validate the graph.
- Mixed node and weight positions.

10 INTERVIEW THINKING FLOW



11 SUMMARY

REMEMBER

- Input determines Representation.
- Representation determines Data Structure.
- Data Structure enables Algorithm.
- Never skip graph construction.



Good construction
saves time.



Clear structure
avoids bugs.



Strong foundation
leads to fast solutions.



“Think like a FAANG interviewer before writing a single line of code.”

1 PLAIN ENGLISH

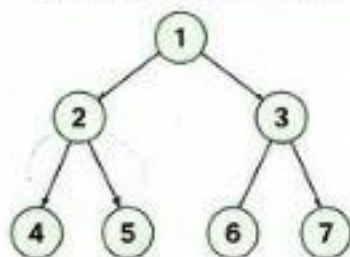
Graphs model relationships.

Most interview problems hide graphs.

Before coding, recognize the hidden graph, understand its type, choose the right representation and the right pattern.

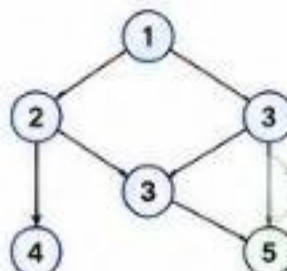
2 VISUAL EXAMPLES

TREE (Special Case of Graph)



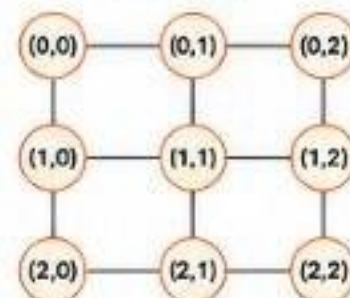
No cycles, Connected,
Has a root.

GENERAL GRAPH



May have cycles,
May be disconnected,
No root

GRID (2D)



Cells are nodes,
Neighbors are edges.

3 BRAIN SWITCH

Interview gives...

- Cities & Roads
- Friends / Social Network
- Flights / Airlines
- Courses / Prerequisites
- Computer Network
- Maze / Game Map
- Dependencies / Tasks
- Files / Folders Structure
- Feeds / Connections

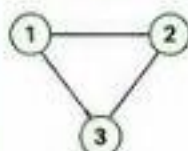


THINK
GRAPH

Whenever you see relationships,
think GRAPH.

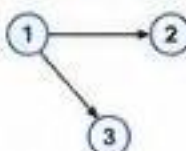
4 GRAPH TYPES

UNDIRECTED



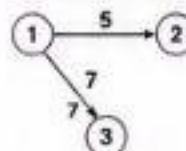
Edges have
no direction.
Ex: Friends

DIRECTED



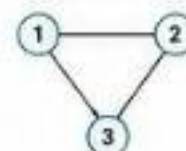
Edges have
direction.
Ex: Flights

WEIGHTED



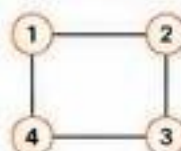
Edges have
weights/costs.
Ex: Roads

UNWEIGHTED



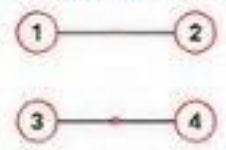
All edges have
equal weight.
Ex: Social Network

CONNECTED



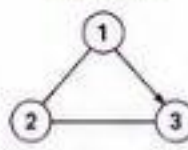
There is a path between
every pair of nodes.
Ex: Network

DISCONNECTED



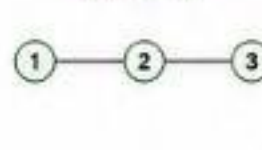
Not all nodes are
reachable.
Ex: Components

CYCLIC



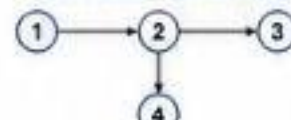
Contains at least
one cycle.
Ex: Dependencies

ACYCLIC



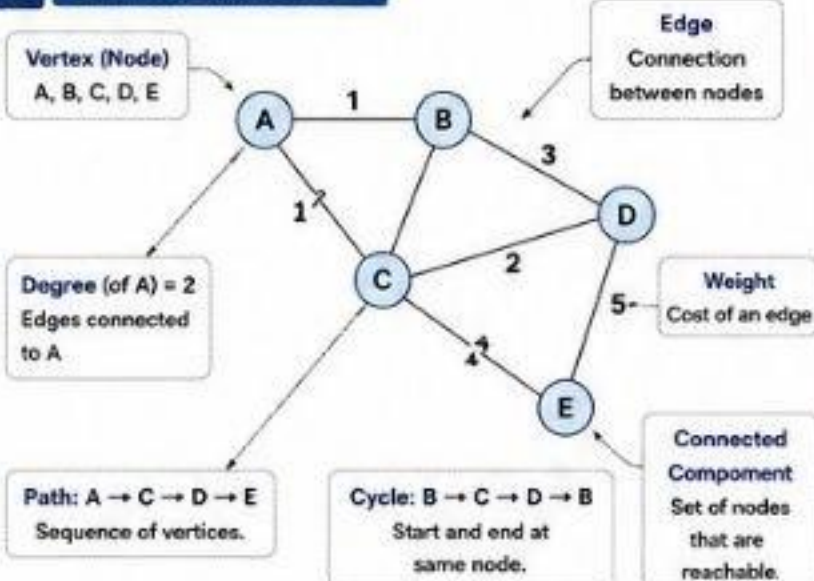
No cycles.
Ex: Tree

DAG (DIRECTED
ACYCLIC GRAPH)



Directed and no cycles.
Ex: Course Schedule

5 GRAPH TERMINOLOGY



6 TREE vs GRAPH

Aspect	TREE	GENERAL GRAPH
Root	Has a root	No root (usually)
Parent	Each node (except root) has exactly one parent	A node can have multiple parents
Neighbor	Limited by parent-child relationship	Any connected node is a neighbor
Cycles	No cycles	May have cycles
Connected	Always connected	May be connected or disconnected
Edges	For n nodes, edges = n - 1	Edges can be any number
Traversal	DFS / BFS works	DFS / BFS works
Need visited[]	Usually Not Needed	Usually Needed
Interview Examples	Binary Tree, N-ary Tree	Social Network, Roads, Flights, Dependencies

A Tree is always a Graph.

A Graph is NOT always a Tree.

8 AHA MOMENTS

- A Tree is a Graph.
- A Grid is a Graph.
- Representation changes. Algorithm usually doesn't.
- Most interview problems hide the graph.
- Outer loop = Multiple Components.

7 GRAPH REPRESENTATIONS (OVERVIEW)

Representation	Visual	Interview Usage	Memory	Notes	
Edge List		List of edges (u, v, w?)	Kruskal, Some Problems	O(E)	Simple to read
Adjacency List		[1: [2, 3], 2: [1, 3], ...]	Most Common in Interviews	O(V + E)	★★★★★ 95% Interview Problems
Adjacency Matrix		2D Matrix n x n	Dense Graphs, Quick Edge Check	O(V ²)	Good for dense graphs
Grid		2D Characters / Integers	Islands, Paths, Flood Fill	O(m x n)	Grid treated as Graph
Node Graph		Nodes with Neighbors	Clone Graph, Complex Structures	O(V + E)	Used in advanced problems

Adjacency List is the most used representation in interviews.

9 PATTERN RECOGNITION

Interview Says...	Think	Expected Graph Type
Cities & Roads	Graph	Undirected, Weighted or Unweighted
Flights / Airlines	Graph	Directed, Weighted
Courses / Prerequisites	Graph	Directed, DAG
Grid / Matrix	Graph	Grid Graph (4 / 8 direction)
Computer Network	Graph	Undirected or Directed
Friends / Social Network	Graph	Undirected, Unweighted
Maze / Game Map	Graph	Grid Graph
Dependencies / Build Order	Graph	Directed, DAG
File / Folder Structure	Graph	Tree (Special Graph)

10 COMMON INTERVIEW MISTAKES

- Forgetting graphs may be disconnected.
- Confusing Tree with Graph.
- Ignoring direction (Directed vs Undirected).
- Ignoring weights.
- Choosing wrong representation.
- Started coding before deciding representation.
- Not defining neighbors clearly.
- Forgetting to validate with small examples.

11 INTERVIEW THINKING (BEFORE CODING)

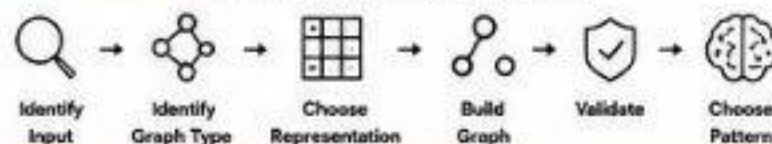
- What is the input? Understand the given data.
- Is it Directed? Do edges have direction?
- Is it Weighted? Do edges have weights/costs?
- Is it Connected? Can all nodes reach other nodes?
- Which Representation should I build? Adj List, Matrix, Grid, etc.
- Which Pattern family will probably solve it? DFS, BFS, Topological, Union Find, Dijkstra, etc.

Common Helper Structures

- visited[]
- dist[]
- parent[]
- indegree[]
- Queue / Deque
- PriorityQueue
- HashMap / Set

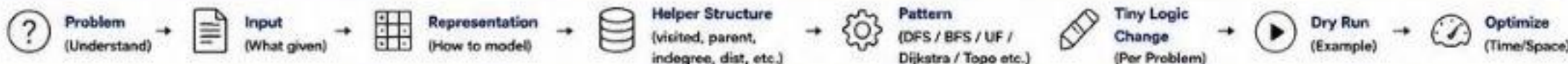
12 SUMMARY

Graph problems are solved in this order.



HIDDEN GRAPHS (Interview disguises)

- Roads
- Flights
- Courses
- People
- Dependencies
- Maze
- Game Map
- Folders
- Networks
- ... Many More



1 MASTER IDEA ★★★★★



Almost every Graph DFS interview problem uses one of five reusable templates.

The DFS traversal almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH DFS TEMPLATE ★★★★★

Input	Adjacency List
Caller	dfs(start)
Visited	boolean[]
Return	void / boolean / int
Used In	- Valid Path - Connected Components - Graph Valid Tree

Java Template

```
private void dfs(int node, List<List<Integer>> graph, boolean[] visited) {  
    visited[node] = true; // mark visited  
    for (int neighbor : graph.get(node)) { // explore neighbors  
        if (!visited[neighbor]) {  
            dfs(neighbor, graph, visited);  
        }  
    }  
}
```

AHA

Graph traversal only changes neighbor source: graph.get(node)

3 GRID DFS TEMPLATE ★★★★★

Java Template

```
private void dfs(int r, int c, char[][] grid, boolean[][] visited) {  
    int n = grid.length;  
    int m = grid[0].length;  
  
    // 1. Boundary check  
    if (r < 0 || r >= n || c < 0 || c >= m) return;  
  
    // 2. Visited or blocked check (assume '0' or '#' is blocked)  
    if (visited[r][c] || grid[r][c] == '0' || grid[r][c] == '#') return;  
  
    // 3. Mark visited  
    visited[r][c] = true;  
  
    // 4. Explore 4 directions  
    int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};  
    for (int[] d : dirs) {  
        int nr = r + d[0];  
        int nc = c + d[1];  
        dfs(nr, nc, grid, visited);  
    }  
}
```

AHA

Grid is an implicit graph.

4 NODE GRAPH TEMPLATE ★★★★★

Java Template

```
private Node dfs(Node node, Map<Node, Node> visited) {  
    if (node == null) return null;  
    if (visited.containsKey(node)) {  
        return visited.get(node); // already cloned  
    }  
  
    // 1. Create clone for current node  
    Node clone = new Node(node.val);  
    visited.put(node, clone);  
  
    // 2. Clone all neighbors  
    for (Node nei : node.neighbors) {  
        clone.neighbors.add(dfs(nei, visited));  
    }  
  
    return clone;  
}
```

AHA

HashMap replaces visited[].

5 MULTI-START DFS ★★★★★

A

OUTER LOOP DFS

Pattern:
for (all nodes)
if (!visited[i])
dfs(i)

Used In
• Connected Components
• Number of Provinces
• Number of Islands

Java Snippet

```
for (int i = 0; i < n; i++) {  
    if (!visited[i]) {  
        dfs(i, graph, visited);  
    }  
}
```

B

BORDER DFS

Pattern:
for (each border cell / node)
if (!visited[b])
dfs(b)

Used In
• Pacific
• Pacific Atlantic
• Surrounded

Java Snippet

```
// Example for grid  
for (int c = 0; c < m; c++) {  
    dfs(0, c, grid, visited); // top row  
    dfs(n - 1, c, grid, visited); // bottom row  
}  
for (int r = 0; r < n; r++) {  
    dfs(r, 0, grid, visited); // left col  
    dfs(r, m - 1, grid, visited); // right col  
}
```



The DFS never changes. Only WHERE it starts changes.

6 WHAT ACTUALLY CHANGES? ★★★★★

Template	Input	Caller	Visited	Return	ONLY THING THAT CHANGES?
Graph DFS (Adjacency List)	Adjacency List	dfs(start)	boolean[]	void / boolean / int	Neighbors come from graph.get(node)
Grid DFS	Grid	dfs(r, c)	boolean[][]	void / int /	Movement using dirs[] + boundary checks
Node DFS	Node Graph	dfs(node)	HashMap<Node, Node>	Node	Visited is HashMap instead of boolean[]
Outer Loop DFS	Adj List / Grid	for all nodes if !visited dfs(i)	boolean[] / boolean[][]	int / List / void	Starts DFS from every unvisited node
Border DFS (Multi-Start)	Grid / Graph	for each border run dfs()	boolean[] / boolean[][]	void	Starts DFS from borders / multiple sources



This table is your 30-second interview revision sheet.

7 RETURN TYPE GUIDE ★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
void	Marking / Traversal (No direct output)	Flood Fill, Surrounded Regions, Mark Components	We only visit nodes / cells. No value returned.
boolean	Searching / Decision	Valid Path, Graph Valid Tree, Cycle Detection	Returns true as soon as condition is met.
int	Counting / Aggregation	Number of Islands, Max Area of Island, Count Components	Accumulate and return numeric result.
Node	Graph Construction	Clone Graph	Returns a new structure (node).
List	Collecting Results	All Paths, Topological Sort, Articulation Points	Collect multiple answers and return.



Changing the return type usually changes the solution.

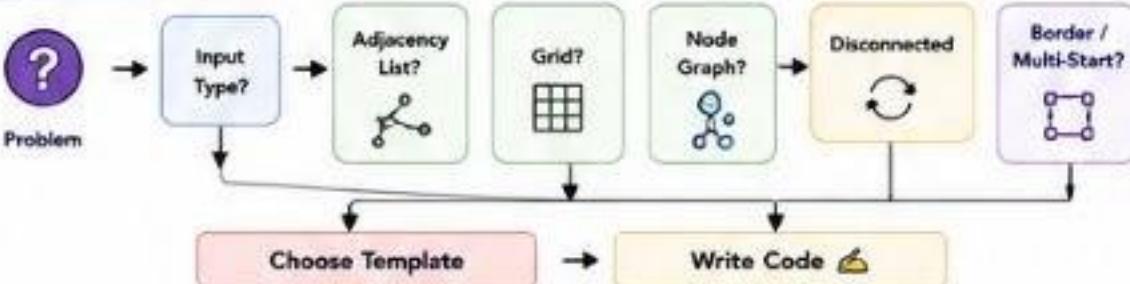
7 AHA MOMENTS

- Adjacency List changes only neighbor traversal.
- Grid DFS changes only movement using dirs[].
- Node DFS changes visited[] into HashMap.
- Outer Loop handles disconnected graphs.
- Border DFS changes only starting nodes.
- 95% of interview problems modify only one or two lines from these templates.

9 COMMON INTERVIEW MISTAKES

- Forgot visited[].
- Forgot boundary check (for grid).
- Forgot dirs[] for grid movement.
- Forgot outer loop for disconnected graphs.
- Forgot HashMap in Clone Graph.
- Infinite recursion (no base case).
- Marked visited after recursion.
- Forgot disconnected graph handling.

10 INTERVIEW THINKING



11 PAGE 5 PREVIEW



Next page contains real interview problems.
For each problem, we show:





1 MASTER IDEA ★★★★★



Almost every Graph DFS problem uses one of a few reusable templates.

The DFS logic stays almost identical.

Only the input, caller, return type and tiny logic change.

2 GRAPH DFS (ADJACENCY LIST) ★★★★★

Input	Adjacency List (List<List<Integer>>)
Used In	Valid Path, Connected Components, Graph Valid Tree
Caller	dfs(startNode)
Visited	boolean[] visited
Return	void / boolean / int / List (as needed)

Java Template

```
private void dfs(int node, List<List<Integer>> graph,
boolean[] visited) {
    visited[node] = true; // mark visited
    for (int nei : graph.get(node)) { // explore neighbors
        if (!visited[nei]) {
            dfs(nei, graph, visited);
        }
    }
}
```

AHA

Graph neighbors come from graph.get(node).

3 GRID DFS ★★★★★

Input	Grid (m x n)
Used In	Number of Islands, Flood Fill, Max Area of Island, Rotting Oranges (DFS version)
Caller	dfs(row, col)
Visited	boolean[][] visited
Helpers	int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}}
Return	void / int (as per problem)

Java Template

```
private void dfs(int r, int c, char[][] grid, boolean[][] visited) {
    int m = grid.length, n = grid[0].length;
    // boundary check
    if (r < 0 || c < 0 || r >= m || c >= n) return;
    // visited or blocked check
    if (visited[r][c] || grid[r][c] == '0') return;
    visited[r][c] = true; // mark visited
    // explore 4 directions
    int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
    for (int[] d : dirs) {
        int nr = r + d[0], nc = c + d[1];
        dfs(nr, nc, grid, visited);
    }
}
```

AHA

Grid is just an implicit graph.

4 NODE GRAPH DFS ★★★★★

Input	Node (graph with neighbors)
Used In	Clone Graph
Caller	dfs(node)
Visited	HashMap<Node, Node> map
Return	Node (cloned node)

Java Template

```
private Node dfs(Node node, HashMap<Node, Node> map) {
    if (map.containsKey(node)) return map.get(node);

    Node clone = new Node(node.val);
    map.put(node, clone); // mark visited & store clone

    for (Node nei : node.neighbors) {
        clone.neighbors.add(dfs(nei, map));
    }

    return clone;
}
```

AHA

HashMap replaces visited[].

5 DISCONNECTED GRAPH DFS (OUTER LOOP) ★★★★★

Input	Graph (Adj List) / Grid
Used In	Connected Components, Number of Provinces, Number of Islands
Caller	for (all nodes/cells) if (!visited) dfs(i) or dfs(r, c)
Visited	boolean[] / boolean[][]
Return	int / void (depends on problem)

Java Template

```
// Graph (Adjacency List) example
int count = 0;
boolean[] visited = new boolean[n];
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i, graph, visited); // use Graph DFS template
        count++;
    }
}
return count; // number of components
```

AHA

Outer loop discovers disconnected graphs.

6 BORDER DFS (MULTI-START POINTS) ★★★★★

Used In	Pacific Atlantic, Surrounded Regions
Caller	Run DFS from border cells / nodes
Visited	boolean[][] / boolean[]
Return	void

Java Template (Grid Example)

```
public void dfsFromBorders(int m, int n, char[][] grid,
boolean[][] visited) {
    // Top & Bottom rows
    for (int c = 0; c < n; c++) {
        if (!visited[0][c] && grid[0][c] != 'X') dfs(0, c, grid, visited);
        if (!visited[m-1][c] && grid[m-1][c] != 'X') dfs(m-1, c, grid, visited);
    }
    // Left & Right columns
    for (int r = 0; r < m; r++) {
        if (!visited[r][0] && grid[r][0] != 'X') dfs(r, 0, grid, visited);
        if (!visited[r][n-1] && grid[r][n-1] != 'X') dfs(r, n-1, grid, visited);
    }
}
```

AHA

Multiple DFS starting points from borders.

7 RETURN TYPE GUIDE ★★★★★

Return Type	Used For	Example Problems	What It Means / When to Use
void	Marking / Visiting	Flood Fill, Surrounded Regions, Pacific Atlantic	You only need to visit and mark nodes/cells.
boolean	Searching / Decision	Valid Path, Detect Cycle, Bipartite	Need to know if something exists / is reachable.
int	Counting / Aggregation	Number of Islands, Max Area of Island, Good Nodes	Need a number (count, area, size, etc.).
Node	Graph Construction	Clone Graph	Need to build and return nodes / structure.
List / Set	Collecting Answers	All Paths, Components, Topological Sort (DFS order), Eventual Safe States	Need to collect multiple results.

★ Changing the return type usually changes the solution.

8 TEMPLATE COMPARISON (30-SECOND RECAP) ★★★★★

Template	Input	Caller	Visited	Return	Special Change
Graph DFS (Adj List)	Adjacency List (List<List<Integer>>)	dfs(startNode)	boolean[]	void / boolean / int / List	Neighbors from graph.get(node)
Grid DFS	Grid (char[][] / int[][])	dfs(r, c)	boolean[][]	void / int / boolean	Boundary check + dirs[] movement
Node DFS	Node (with neighbors)	dfs(node)	HashMap<Node, Node>	Node	Use HashMap to map original → clone
Outer Loop DFS	Graph / Grid	for all nodes/cells if ! visited dfs(...)	boolean[] / boolean[][]	int / List / void	Outer loop to handle disconnected parts
Border DFS (Multi-Start)	Grid (heights/char) or Graph	Run DFS from borders	boolean[][] / boolean[]	void	Multiple starting points

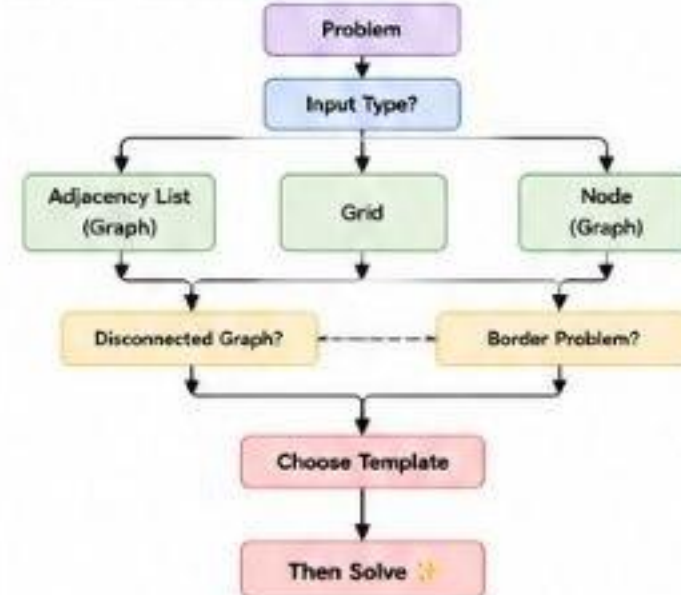
9 AHA MOMENTS ★★★★★

- Adjacency List changes only neighbor traversal.
- Grid DFS changes only movement using dirs[].
- Node Graph changes visited[] into HashMap.
- Outer Loop handles disconnected graphs.
- Border DFS changes only where DFS starts.
- Most interview problems modify only one or two lines.

10 COMMON INTERVIEW MISTAKES ★★★★★

- Forgot visited[].
- Marked visited too late (after recursion).
- Forgot boundary check (for grid).
- Forgot dirs[] for grid movement.
- Forgot outer loop for disconnected graphs.
- Forgot multiple starting points (border DFS).
- Forgot HashMap in Clone Graph.
- Infinite recursion (no base case).

11 INTERVIEW THINKING FLOW ★★★★★

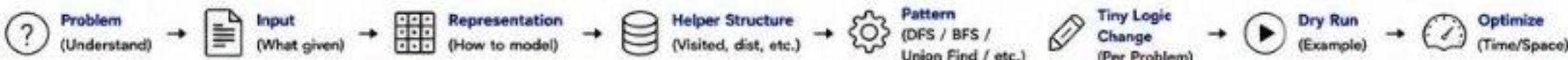


12 PREVIEW OF PAGE 5 ★★★★★

Next page contains DFS problem guide with real interview problems.

- Valid Path
- Connected Components
- Number of Islands
- Flood Fill
- Clone Graph
- Pacific Atlantic
- Surrounded Regions

Each problem highlights ONLY the changed lines from these templates.





1 MASTER IDEA



Almost every Graph DFS interview problem uses one of five reusable templates.

The DFS traversal almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH DFS TEMPLATE

★★★★★

Input	Adjacency List
Caller	dfs(start)
Visited	boolean[]
Return	void / boolean / int
Used In	- Valid Path - Connected Components - Graph Valid Tree

Java Template

```
private void dfs(int node, List<List<Integer>> graph, boolean[] visited) {
    visited[node] = true; // mark visited
    for (int neighbor : graph.get(node)) { // explore neighbors
        if (!visited[neighbor]) {
            dfs(neighbor, graph, visited);
        }
    }
}
```

AHA

Graph traversal only changes neighbor source: `graph.get(node)`

3 GRID DFS TEMPLATE

★★★★★

Java Template

```
private void dfs(int r, int c, char[][] grid, boolean[][] visited) {
    int m = grid.length;
    int n = grid[0].length;

    // 1. Boundary check
    if (r < 0 || r >= m || c < 0 || c >= n) return;

    // 2. Visited or blocked check
    if (visited[r][c] || grid[r][c] == '0' || grid[r][c] == 'X') return;

    // 3. Mark visited
    visited[r][c] = true;

    // 4. Explore 4 directions
    int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
    for (int[] d : dirs) {
        int nr = r + d[0];
        int nc = c + d[1];
        dfs(nr, nc, grid, visited);
    }
}
```

Input	Grid
Caller	dfs(r, c)
Visited	boolean[][]
Movement	dirs[]
Used In	- Number of Islands - Flood Fill - Max Area of Island - Surrounded Regions

AHA

Grid is an implicit graph.

4 NODE GRAPH TEMPLATE

★★★★★

Java Template

```
private Node dfs(Node node, HashMap<Node, Node> map) {
    if (node == null) return null;

    if (map.containsKey(node)) {
        return map.get(node); // already cloned
    }

    // 1. Create clone for current node
    Node clone = new Node(node.val);
    map.put(node, clone);

    // 2. Clone all neighbors
    for (Node nei : node.neighbors) {
        clone.neighbors.add(dfs(nei, map));
    }

    return clone;
}
```

Input	Node
Caller	dfs(node)
Visited	HashMap<Node, Node>
Return	Node (cloned node)
Used In	Clone Graph

AHA

HashMap replaces visited[].

5 MULTI-START DFS

★★★★★

A OUTER LOOP DFS

Pattern:	Used In
for (all nodes) if (!visited[i]) dfs(i)	- Connected Components - Number of Provinces - Number of Islands

Java Snippet

```
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i, graph, visited);
    }
}
```

B BORDER DFS

Pattern:	Used In
for (each border cell / node) if (!visited[b]) dfs(b)	- Pacific Atlantic - Surrounded Regions

Java Snippet

```
// Example for grid
for (int c = 0; c < n; c++) {
    dfs(0, c, grid, visited); // top row
    dfs(m - 1, c, grid, visited); // bottom row
}
for (int r = 0; r < m; r++) {
    dfs(r, 0, grid, visited); // left col
    dfs(r, n - 1, grid, visited); // right col
}
```



The DFS never changes. Only WHERE it starts changes.

6 WHAT ACTUALLY CHANGES?

★★★★★

Template	Input	Caller	Visited	Return	ONLY THING THAT CHANGES
Graph DFS (Adj List)	Adjacency List	dfs(start)	boolean[]	void / boolean / int	Neighbors come from graph.get(node)
Grid DFS	Grid	dfs(r, c)	boolean[][]	void / int or boolean	Movement using dirs[] + boundary checks
Node DFS	Node Graph	dfs(node)	HashMap<Node, Node>	Node	Visited is HashMap instead of boolean[]
Outer Loop DFS	Adj List / Grid	for all nodes if !visited dfs(i)	boolean[] / boolean[][]	int / List / void	Starts DFS from every unvisited node
Border DFS	Grid / Graph	for each border run dfs()	boolean[] / boolean[][]	void	Starts DFS from borders / multiple sources



This table is your 30-second interview revision sheet.

7 RETURN TYPE GUIDE

★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
void	Marking / Traversal (No direct output)	Flood Fill, Surrounded Regions, Pacific Atlantic	We only visit nodes / cells. No value returned.
boolean	Searching / Decision	Valid Path, Graph Valid Tree, Cycle Detection	Returns true as soon as condition is met.
int	Counting / Aggregation	Number of Islands, Max Area of Island, Count Components	Accumulate and return numeric result.
Node	Graph Construction	Clone Graph	Returns a new structure (node).
List	Collecting Results	All Paths, Topological Sort, Articulation Points	Collect multiple answers and return.

Changing the return type usually changes the solution.

8 AHA MOMENTS

★★★★★

- Adjacency List changes only neighbor traversal.
- Grid DFS changes only movement using dirs[].
- Node DFS changes visited[] into HashMap.
- Outer Loop handles disconnected graphs.
- Border DFS changes only starting nodes.
- 95% of interview problems modify only one or two lines from these templates.

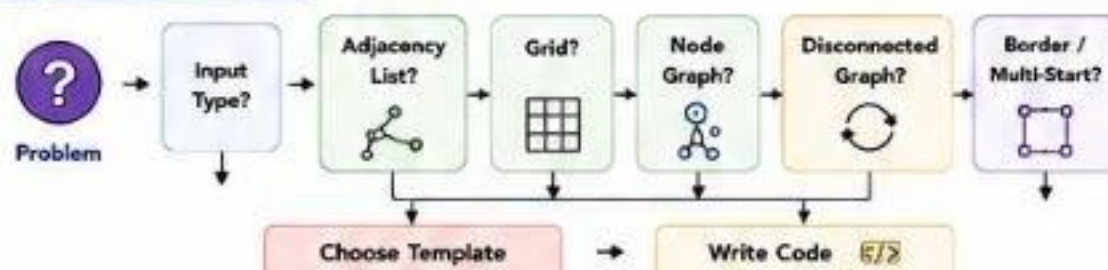
9 COMMON INTERVIEW MISTAKES

★★★★★

- Forgot visited[].
- Forgot boundary check (for grid).
- Forgot dirs[] for grid movement.
- Forgot outer loop for disconnected graphs.
- Forgot HashMap in Clone Graph.
- Infinite recursion.
- Marked visited after recursion.
- Forgot disconnected graph handling.

10 INTERVIEW THINKING

★★★★★

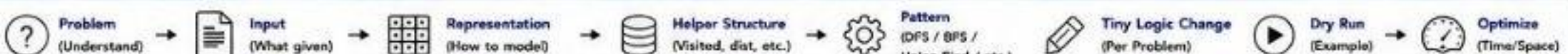


11 PAGE 5 PREVIEW

★★★★★



real interview problems.
For each problem, we show:





GRAPH PATTERN GUIDE

PAGE 6 — MASTER BFS TEMPLATE GUIDE

"One BFS algorithm. Five reusable templates."

1 MASTER IDEA ★★★★★



Almost every Graph BFS interview problem uses one of a few reusable templates.

The BFS traversal pattern almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH BFS TEMPLATE (ADJACENCY LIST) ★★★★★



Input	Adjacency List (List<List<Integer>>)
Caller	bfs(start)
Visited	boolean[]
Return	void / boolean / int
Used In	- Valid Path - Connected Components - Shortest Path (Unweighted) - Bipartite Check

Java Template

```
public void bfs(int start, List<List<Integer>> graph, boolean[] visited) {
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
        int node = q.poll();
        // process current node here (modify as per problem)
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.offer(neighbor);
            }
        }
    }
}
```

AHA

Graph neighbors come from graph.get(node). BFS explores level by level.

3 GRID BFS TEMPLATE ★★★★★

Java Template

```
public void bfs(int r, int c, int[][] grid, boolean[][] visited) {
    int m = grid.length, n = grid[0].length;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    Queue<int[]> q = new LinkedList<>();
    q.offer(new int[]{r, c});
    visited[r][c] = true;

    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int cr = cur[0], cc = cur[1];

        for (int[] d : dirs) {
            int nr = cr + d[0], nc = cc + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n
                && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

AHA

Grid is an implicit graph. We move using dirs[].

4 MULTI-SOURCE BFS TEMPLATE ★★★★★

Java Template

```
public void multiSourceBfs(List<int[]> sources, int m, int n,
    boolean[][] visited) {
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    Queue<int[]> q = new LinkedList<>();

    // 1. Add all sources to queue
    for (int[] src : sources) {
        q.offer(src);
        visited[src[0]][src[1]] = true;
    }

    // 2. Process together level by level
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int r = cur[0], c = cur[1];
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n
                && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

AHA

Multiple starting points become one BFS. Distances spread outwards.

5 LEVEL ORDER BFS TEMPLATE ★★★★★

Java Template

```
public List<List<Integer>> levelOrder(int start, List<List<Integer>> graph) {
    List<List<Integer>> ans = new ArrayList<>();
    boolean[] visited = new boolean[graph.size()];
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
        int size = q.size(); // level size
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            int node = q.poll();
            level.add(node);
            for (int nei : graph.get(node)) {
                if (!visited[nei]) {
                    visited[nei] = true;
                    q.offer(nei);
                }
            }
        }
        ans.add(level);
    }
    return ans;
}
```

AHA

Queue size gives the number of nodes at the current level.

6 DISTANCE BFS TEMPLATE ★★★★★

Java Template

```
public int bfsShortestPath(int start, int target, List<List<Integer>> graph) {
    int n = graph.size();
    boolean[] visited = new boolean[n];
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;
    int level = 0;

    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            int node = q.poll();
            if (node == target) return level;
            for (int nei : graph.get(node)) {
                if (!visited[nei]) {
                    visited[nei] = true;
                    q.offer(nei);
                }
            }
        }
        level++; // next distance
    }
    return -1; // not reachable
}
```

AHA

First time you reach a node is the shortest distance in unweighted graphs.

7 WHAT ACTUALLY CHANGES? ★★★★★

Template	Input	Caller	Visited	Return	ONLY THING THAT CHANGES
Graph BFS (Adj List)	Adjacency List (List<List<Integer>>)	bfs(start)	boolean[]	void / boolean / int	Neighbors come from graph.get(node)
Grid BFS	Grid (int[][])	bfs(r, c)	boolean[][]	void / int	Movement using dirs[] + boundary checks
Multi-Source BFS	Grid / Graph (Any)	multiSourceBfs(sources)	boolean[][] or boolean[]	void / int	Initial queue contains all sources
Level Order BFS	Graph / Tree (Any)	bfs(start)	boolean[]	List<List<Integer>> / void / int	Use queue size (level) to process by level
Distance BFS	Graph / Grid (Any)	bfs(start)	boolean[] (or distance[])	int	Track level or distance for each node

★ This table is your 30-second interview revision sheet. Memorize it.

8 RETURN TYPE GUIDE ★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
void	Traverse / Mark / Modify	Flood Fill, Surrounded Regions, Rotting Oranges	We only visit / update nodes.
boolean	Search / Existence Check	Valid Path, Word Ladder, Bipartite Check	Return true when condition is met.
int	Distance / Steps / Count	Shortest Path, Nearest Exit, 01 Matrix, Minimum Steps	Count level or distance.
int[] / int[][]	Store Distances	Walls and Gates, 01 Matrix, Distance to Nearest	Record best distance per node/cell.
List / List<List<>>	Collect Results	Level Order, All Shortest Paths, Top-K Level Problems	Collect multiple answers or levels.

★ Changing the return type usually changes the solution.

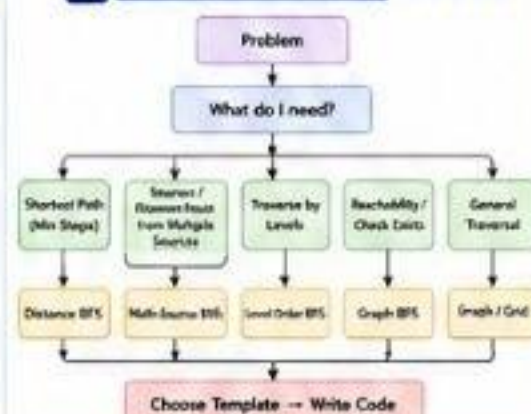
9 AHA MOMENTS ★★★★★

- BFS uses a Queue → explores level by level.
- First time you visit a node is always the shortest (when all edges have same weight).
- Multi-source BFS = start from everywhere at once.
- Queue size gives level size (very powerful).
- BFS is perfect for shortest path, minimum steps, nearest, distance problems.

10 COMMON INTERVIEW MISTAKES ★★★★★

- Forgot to mark visited.
- Marked visited too late (after adding to queue).
- Forgot queue size when doing level order.
- Added the same node multiple times.
- Didn't initialize all sources for multi-source BFS.
- Used DFS instead of BFS for shortest path.
- Infinite loop due to cycle (no visited).
- Forgot disconnected graph (use outer loop).

11 INTERVIEW THINKING FLOW ★★★★★



12 PAGE 7 PREVIEW ★★★★★

Next page contains GRAPH BFS PROBLEMS. For each problem, you will see:



Apply these templates with tiny changes and solve any BFS problem with confidence!



GRAPH PATTERN GUIDE

PAGE 6 — MASTER BFS TEMPLATE GUIDE

"One BFS algorithm. Five reusable templates."

1 MASTER IDEA



Almost every Graph BFS interview problem uses one of the five reusable templates.

The BFS traversal pattern almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH BFS TEMPLATE (ADJACENCY LIST)

★★★★★



Input Adjacency List (List<List<Integer>>)



Caller bfs(start)



Visited boolean[]



Return void / boolean / int



Used In

- Valid Path
- Connected Components
- Shortest Path (Unweighted)
- Bipartite Check

Java Template

```
public void bfs(int start, List<List<Integer>> graph, boolean[] visited) {
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
        int node = q.poll();
        // process current node here (modify as per problem)
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.offer(neighbor);
            }
        }
    }
}
```

AHA

Graph neighbors come from graph.get(node).
BFS explores level by level.

3 GRID BFS TEMPLATE

★★★★★

Java Template

```
public void bfs(int r, int c, int[][] grid, boolean[][] visited) {
    int m = grid.length, n = grid[0].length;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    Queue<int> q = new LinkedList<>();
    q.offer(new int[]{r, c});
    visited[r][c] = true;

    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int cr = cur[0], cc = cur[1];

        for (int[] d : dirs) {
            int nr = cr + d[0], nc = cc + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n
                && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

AHA

Grid is an implicit graph. We move using dirs[].

4 MULTI-SOURCE BFS TEMPLATE

★★★★★

Java Template

```
public void bfs(List<int> sources, int m, int n,
    int[][] dirs, boolean[][] visited) {
    Queue<int> q = new LinkedList<>();

    // 1. Add all sources first
    for (int src : sources) {
        q.offer(src);
        visited[src][0][src[1]] = true;
    }

    // 2. Process together level by level
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int r = cur[0], c = cur[1];
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n
                && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

AHA

Multiple starting points become one BFS. Distances spread outwards.

5 LEVEL ORDER BFS TEMPLATE

★★★★★

Java Template

```
public List<List<Integer>> levelOrder(List<List<Integer>> graph, int start) {
    List<List<Integer>> ans = new ArrayList<>();
    Queue<Integer> q = new LinkedList<>();
    boolean[] visited = new boolean[graph.size()];
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < q.size(); i++) {
            int node = q.poll();
            level.add(node);
            for (int nei : graph.get(node)) {
                if (!visited[nei]) {
                    visited[nei] = true;
                    q.offer(nei);
                }
            }
        }
        ans.add(level);
    }
    return ans;
}
```

AHA

Queue size gives the number of nodes at the current level.

6 DISTANCE BFS TEMPLATE

★★★★★

Java Template

```
public int bfsShortestPath(int start, int target,
    List<List<Integer>> graph) {
    Queue<Integer> q = new LinkedList<>();
    boolean[] visited = new boolean[graph.size()];
    q.offer(start);
    visited[start] = true;
    int level = 0;

    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            int node = q.poll();
            if (node == target) return level;
            for (int nei : graph.get(node)) {
                if (!visited[nei]) {
                    visited[nei] = true;
                    q.offer(nei);
                }
            }
        }
        level++; // next distance
    }
    return -1; // not reachable
}
```

AHA

First time you reach a node is the shortest distance in unweighted graphs.

7 WHAT ACTUALLY CHANGES?

★★★★★

Template	Input	Caller	Visited	Queue Init	Return	ONLY THING THAT CHANGES
Graph BFS (Adj List)	Adjacency List	bfs(start)	boolean[]	Add start node	void / boolean / int	Neighbors from graph.get(node)
Grid BFS	Grid	bfs(r, c)	boolean[][]	Add start cell	void / int	Movement using dirs[] + boundary checks
Multi-Source BFS	Grid / Graph (Any)	bfs(sources)	boolean[][] or boolean[]	Add all sources	void / int	Initial queue contains all sources
Level Order BFS	Graph / Tree (Any)	bfs(start)	boolean[]	Add start node	List<List<Integer>> / void / int	Use queue size (level) to process level
Distance BFS	Graph / Grid (Any)	bfs(start)	boolean[] or distance[]	Add start node	int	Track level or distance for each node

★ This table is your 30-second interview revision sheet. Memorize it.

8 RETURN TYPE GUIDE

★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
void	Traverse / Mark / Modify	Flood Fill, Rotting Oranges, Walls and Gates	We only visit / update nodes.
boolean	Search / Existence Check	Valid Path, Bipartite Check, Word Ladder	Return true when condition is met.
int	Distance / Steps / Count	Shortest Path, Nearest Exit, 01 Matrix, Minimum Steps	Count level or distance.
int[] / int[][]	Store Distances	Walls and Gates, 01 Matrix, Distance to Nearest	Record best distance per node/cell.
List / List<List<>>	Collect Results	Level Order, All Shortest Paths, Top-K Level Problems	Collect multiple answers or levels.

★ Changing the return type usually changes the solution.

9 AHA MOMENTS

★★★★★

- 1 BFS uses a Queue → explores level by level.
- 2 First time you visit a node is always the shortest (in unweighted graphs).
- 3 Multi-source BFS = start from everywhere at once.
- 4 Queue size gives level size (very powerful).
- 5 BFS is perfect for shortest path, minimum steps, nearest, distance problems.

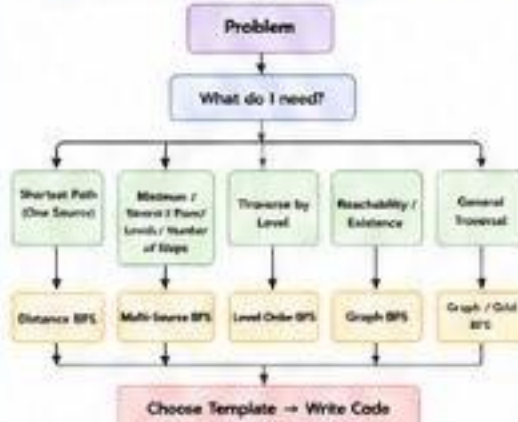
10 COMMON INTERVIEW MISTAKES

★★★★★

- 1 Forgot to mark visited.
- 2 Marked visited too late (after adding to queue).
- 3 Forgot queue size when doing level order.
- 4 Added the same node multiple times.
- 5 Didn't initialize all sources for multi-source BFS.
- 6 Used DFS instead of BFS for shortest path.
- 7 Infinite loop due to cycle (no visited).
- 8 Forgot disconnected graph (use outer loop).

11 INTERVIEW THINKING FLOW

★★★★★



12 PAGE 7 PREVIEW

★★★★★

Next page contains GRAPH BFS PROBLEMS.
For each problem, you will see:



Apply these templates with tiny changes and solve any BFS problem with confidence!



HOW TO USE THIS PAGE

- 1 Find the problem.
- 2 Identify the base template (from Page 8).
- 3 Read the CHANGED LINES.
- 4 Understand the AHA.
- 5 Review complexity & interview tip.
- 6 Practice writing from memory.



TEMPLATE LEGEND (From Page 8)

K

Kahn's (BFS)
Indegree + Queue

S

DFS + Stack
Postorder

H

Kahn's + Min Heap
Lexicographically Smallest

C

Cycle Check
(Any Method)

1 COURSE SCHEDULE

K

Base Template: Kahn's Algorithm (BFS)

Problem

Can we finish all courses? (No cycle)

Changed Lines (from Template)

```
// 1. Graph: for each [a, b] → b → a (take b then a)
for (int[] p : prerequisites) {
    int a = p[0], b = p[1];
    graph.get(b).add(a);
    indegree[a]++;
}

// 2. Return true if we process all courses
return count == numCourses;
```

Caller

canFinish(numCourses, prereq)

Return

boolean

Key Idea

All courses
processed?

AHA: If any course has remaining indegree → cycle.

Complexity

Time: $O(V + E)$ Space: $O(V + E)$

Interview Tip

Think in terms of ordering
dependencies.

2 COURSE SCHEDULE II

K

Base Template: Kahn's Algorithm (BFS)

Problem

Return any valid order to finish all courses.

Changed Lines (from Template)

```
// 1. Same graph as Course Schedule
for (int[] p : prerequisites) {
    int a = p[0], b = p[1];
    graph.get(b).add(a);
    indegree[a]++;
}

// 2. Store order while processing
List<Integer> order = new ArrayList<>();
...
order.add(node);
// 3. Final check
if (order.size() != numCourses) return new int[0];
// cycle
return order.stream().mapToInt(i -> i).toArray();
```

Caller

findOrder(numCourses, prereq)

Return

int[]

Key Idea

Build order while
processing

AHA: Kahn's gives order naturally.

Complexity

Time: $O(V + E)$ Space: $O(V + E)$

Interview Tip

If size < numCourses →
cycle, return empty.

3 ALIEN DICTIONARY

S

Base Template: DFS + Stack (Postorder)

Problem

Find order of characters from sorted dictionary.

Changed Lines (from Template)

```
// 1. Build graph from first diff char
// Word1 comes before Word2
// Edge: char1 → char2
for (int i = 0; i < words.length - 1; i++) {
    String w1 = words[i], w2 = words[i + 1];
    int len = Math.min(w1.length(), w2.length());
    for (int j = 0; j < len; j++) {
        if (w1.charAt(j) != w2.charAt(j)) {
            graph.get(w1.charAt(j)).add(w2.charAt(j));
            break;
        }
    }
}

// 2. DFS postorder gives character order
```

Caller

alienOrder(words)

Return

String

Key Idea

Edges from
first diff char

AHA: Postorder of DFS = valid character order.

Complexity

Time: $O(V + E)$ Space: $O(V + E)$

Interview Tip

Also handle invalid case:
"abc" before "ab".

4 BUILD ORDER

K

Base Template: Kahn's Algorithm (BFS)

Problem

Given projects and dependencies, return build order.

Changed Lines (from Template)

```
// 1. Graph: for each [a, b] → b → a
// means build b before a
for (int[] d : dependencies) {
    int a = d[0], b = d[1];
    graph.get(b).add(a);
    indegree[a]++;
}

// 2. Return order list
if (order.size() != numProjects) return new int[0];
```

Caller

findBuildOrder(numProjects, deps)

Return

int[]

Key Idea

Standard topo
order

AHA: Same as Course Schedule II.

Complexity

Time: $O(V + E)$ Space: $O(V + E)$

Interview Tip

Exactly same pattern,
different story.

5 TASK SCHEDULER (WITH ADVANCED CONSTRAINT)

K

Base Template: Kahn's Algorithm (BFS)

Problem

Return order if possible, else empty.

Changed Lines (from Template)

```
// 1. Build graph from conditions
for (int[] c : conditions) {
    int u = c[0], v = c[1]; // u before v
    graph.get(u).add(v);
    indegree[v]++;
}

// 2. Standard Kahn's
// Return order only if all tasks processed
```

Caller

taskOrder(tasks, conditions)

Return

int[]

Key Idea

Dependencies
between tasks

AHA: If some tasks remain → cycle.

Complexity

Time: $O(V + E)$ Space: $O(V + E)$

Interview Tip

Check indegree[] after
processing all nodes.

6 MINIMUM SEMESTERS

K

Base Template: Kahn's Algorithm (BFS)

Problem

Return minimum semesters to finish all courses.

Changed Lines (from Template)

```
// 1. Standard graph from prerequisites
// 2. Use level-based processing (BFS levels)
int semesters = 0;
while (!q.isEmpty()) {
    int size = q.size();
    semesters++;
    for (int i = 0; i < size; i++) {
        int node = q.poll();
        for (int nei : graph.get(node)) {
            indegree[nei]--;
            if (indegree[nei] == 0) q.offer(nei);
        }
    }
}

return (count == numCourses) ? semesters : -1;
```

Caller

minSemesters(numCourses, prereq)

Return

int

Key Idea

BFS levels =
semesters

AHA: Levels in BFS give minimum time.

Complexity

Time: $O(V + E)$ Space: $O(V + E)$

Interview Tip

Count levels, not nodes.
That's the answer.

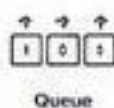
7 CHANGED LINES SUMMARY

#	Problem	Base Template	Main Change
1	Course Schedule	Kahn's (BFS)	Return boolean (processed all)
2	Course Schedule II	Kahn's (BFS)	Store order (list) & return array
3	Alien Dictionary	DFS + Stack	Build graph from words (first diff)
4	Build Order	Kahn's (BFS)	Same as Course Schedule II
5	Task Scheduler	Kahn's (BFS)	General dependencies
6	Minimum Semesters	Kahn's (BFS)	Level BFS → count semesters

8 TOPO SORT TEMPLATE RECAP

Kahn's (BFS)

- Indegree[]
- Queue of 0 indegree
- Process → reduce indegree
- Detect cycle if not all processed



Queue

DFS + Stack

- DFS all nodes
- Push after visiting neighbors
- Reverse stack = order
- Detect cycle with recursion stack



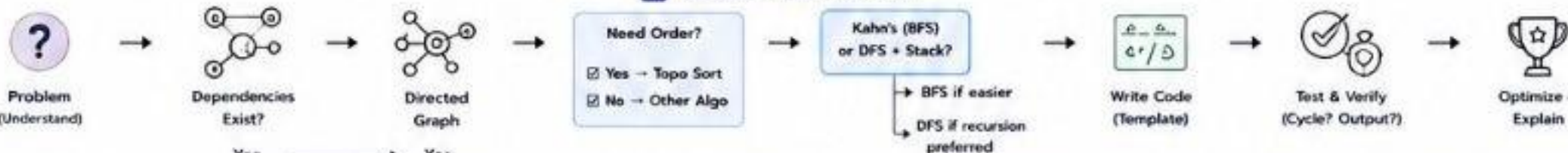
9 RETURN TYPE GUIDE

Return Type	Use When	Example Problems
boolean	Check if order is possible	Course Schedule
int[] / List<Integer>	Need the order	Course Schedule II, Build Order
String	Order of characters	Alien Dictionary
int	Minimum time / levels	Minimum Semesters
-1 / Empty	Cycle exists	All problems when impossible

10 INTERVIEW CHECKLIST

- ☒ Built correct graph direction?
- ☒ Calculated indegree[] correctly?
- ☒ Added all nodes (even with 0 indegree)?
- ☒ Used correct template (Kahn's or DFS)?
- ☒ Handled cycle case?
- ☒ Returned correct type?
- ☒ Time & space complexity?
- ☒ Dry run on small example?
- ☒ Can you explain your approach clearly?

11 INTERVIEW THINKING FLOW



CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)





“Order matters. Use Topological Sort.”

1 MASTER IDEA

Use Topological Sort when dependencies exist.

It gives a valid linear order of tasks.

If impossible → CYCLE exists.

TIP: If order.size() != numCourses → Cycle exists (impossible order).

2 KAHN'S ALGORITHM (BFS TOPOLOGICAL SORT) ★★★★★

Input	Directed Graph (Adj List)
Caller	topoSort(numCourses, edges)
Key Idea	Remove 0 indegree nodes repeatedly
Data Structure	Queue<Integer> + indegree[]
Return	List<Integer> order
Used In	• Course Schedule • Course Schedule II • Alien Dictionary • Build Order

```
Java Template
public List<Integer> topoSort(int numCourses, int[][] edges) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());

    int[] indegree = new int[numCourses];
    for (int[] e : edges) {
        int u = e[0], v = e[1]; // u -> v
        graph.get(u).add(v);
        indegree[v]++;
    }

    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) q.offer(i); // all starting points
    }

    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int node = q.poll();
        order.add(node);
        for (int nei : graph.get(node)) {
            indegree[nei]--;
            if (indegree[nei] == 0) q.offer(nei);
        }
    }

    return (order.size() == numCourses) ? order : new ArrayList<>(); // empty -> cycle
}
```

AHA MOMENTS

- Indegree 0 nodes are “ready” to take.
- Remove it, reduce indegree of neighbors.
- If all nodes are taken → DAG (no cycle).
- If not all nodes → cycle exists.

COMMON MISTAKES

- Forgot indegree[] initialization.
- Didn't add all 0 indegree nodes first.
- Used Stack instead of Queue (wrong order).
- Didn't check cycle (order.size() test).

3 TOPO SORT (DFS + STACK METHOD) ★★★★★

Input	Directed Graph (Adj List)
Caller	topoSortDFS(numCourses, edges)
Key Idea	Finish node → push to stack
Data Structure	Stack<Integer> + visited[] + recStack[]
Return	List<Integer> order
Used In	• Course Schedule II (Alt) • Alien Dictionary (Alt) • Build Order (Alt)

```
Java Template
public List<Integer> topoSortDFS(int numCourses, int[][] edges) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
    for (int[] e : edges) graph.get(e[0]).add(e[1]);

    boolean[] visited = new boolean[numCourses];
    boolean[] recStack = new boolean[numCourses];
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < numCourses; i++) {
        if (!visited[i]) {
            if (dfs(i, graph, visited, recStack, stack)) return new ArrayList<>(); // cycle
        }
    }

    List<Integer> order = new ArrayList<>();
    while (!stack.isEmpty()) order.add(stack.pop());
    return order;

    private boolean dfs(int node, List<List<Integer>> graph, boolean[] visited, boolean[] recStack, Stack<Integer> stack) {
        visited[node] = true;
        recStack[node] = true;
        for (int nei : graph.get(node)) {
            if (!visited[nei]) {
                if (dfs(nei, graph, visited, recStack, stack)) return true;
            } else if (recStack[nei]) {
                return true; // cycle
            }
        }
        recStack[node] = false;
        stack.push(node);
        return false;
    }
}
```

AHA

- Postorder of DFS gives a valid topological order.
- recStack detects back-edge → cycle.
- Stack reversed = correct order.

WHEN TO USE WHICH?

Approach	Use When
Kahn's (BFS)	Need lexicographically smallest order
Kahn's (BFS)	Graph is large / iterative preferred
DFS + Stack	Easy cycle detection in recursion
DFS + Stack	Natural for recursive thinking

4 LEXICOGRAPHICALLY SMALLEST ORDER (BFS + MIN HEAP) ★★★★★

Change From Kahn's Algorithm	Use PriorityQueue (Min Heap)
Data Structure	PriorityQueue<Integer> pq
Use In	• Course Schedule II (Lexico) • Smallest Order Problems

```
Java Template
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int i = 0; i < numCourses; i++) {
    if (indegree[i] == 0) pq.offer(i); // min heap instead of queue
}

List<Integer> order = new ArrayList<>();
while (!pq.isEmpty()) {
    int node = pq.poll(); // smallest node first
    order.add(node);
    for (int nei : graph.get(node)) {
        indegree[nei]--;
        if (indegree[nei] == 0) pq.offer(nei);
    }
}

return order;
```

AHA Min Heap ensures smallest available node is always picked.

5 CYCLE DETECTION (IN DAG CHECK) ★★★★★

Method	Kahn's (BFS)
Check	order.size() != V
Result	Cycle Exists
Why?	Some nodes always have indegree > 0 (cycle)

AHA MOMENT

- If we cannot remove all nodes → cycle exists.

COMMON PITFALLS

- Not returning empty list on cycle.
- Confusing 0 indegree concept.
- Not building graph properly.
- Forgetting disconnected components.

TIP: Always verify order.size() == numCourses (or V).

6 WHAT ACTUALLY CHANGES? ★★★★★

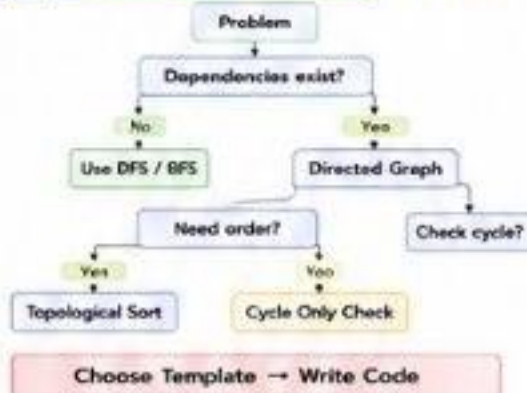
Template	Graph Type	Core Idea	Data Structure	Only Thing That Changes	Typical Problems
Kahn's (BFS)	Directed Acyclic Graph	Remove 0 indegree nodes	Queue + indegree[]	How we pick next node (Queue vs Min Heap)	Course Schedule, Alien Dictionary, Build Order
Kahn's (BFS + Min Heap)	DAG	Remove smallest 0 indegree	Min Heap + indegree[]	Use PriorityQueue instead of Queue	Course Schedule II (Lexico)
DFS + Stack	DAG	Finish node → push stack	Stack + visited[] + recStack[]	Order from stack (reverse)	Course Schedule II, Alien Dictionary
Cycle Detection	Directed Graph	If not all nodes visited → cycle	Same as above	Check order.size() != V	Course Schedule (cycle check)

7 RETURN TYPE GUIDE ★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
List<Integer>	Valid order	Course Schedule II, Alien Dictionary	Provide correct order
boolean	Cycle exists?	Course Schedule	Feasibility check
int[]	Order as array	Build Order	Return index-based order
String	Order as string	Alien Dictionary	Lexico order (characters)

Changing return type usually changes the solution slightly.

8 INTERVIEW THINKING FLOW ★★★★★



9 PAGE 9 PREVIEW ★★★★★

Next page contains TOPOLOGICAL SORT PROBLEM GUIDE. For each problem, you will see:



Apply these templates with tiny changes and solve any Topo Sort problem with confidence!



HOW TO USE THIS PAGE

- 1 Find the problem.
- 2 Identify the Base Template (from Page 4).
- 3 Read the CHANGED LINES.
- 4 Understand the AHA.
- 5 Write code in the interview.



TEMPLATE LEGEND (From Page 4)



Graph DFS (Adj List)



Grid DFS



Node DFS



Outer Loop DFS



Border DFS

1 VALID PATH

G

Base Template: Graph DFS (Adjacency List)

Problem

Determine if there is a path between source and destination.

Changed Lines (from Template)

```
// 1. Add destination and return boolean
if (node == destination) return true;

// 2. Change return type and propagate
for (int neighbor : graph.get(node)) {
    if (!visited[neighbor]) {
        if (dfs(neighbor, destination, graph, visited))
            return true;
    }
}
return false;
```

Caller	dfs(source, destination, graph, visited)	Return	boolean	Visited	boolean[]
--------	--	--------	---------	---------	-----------

Stop early when destination is found.

Complexity	Interview Tip
Time: $O(V + E)$	Mention early termination optimization.
Space: $O(V)$	

2 CONNECTED COMPONENTS

G

Base Template: Outer Loop DFS (Graph DFS + Outer Loop)

Problem

Count number of connected components in an undirected graph.

Changed Lines (from Template)

```
// 1. Outer loop to cover all nodes
int count = 0;
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        count++;
        dfs(i, graph, visited);
    }
}

// 2. DFS remains same (void)
```

Caller	for all nodes i	Return	int (count)	Visited	boolean[]
--------	-----------------	--------	-------------	---------	-----------

Each unvisited node starts a new component.

Complexity	Interview Tip
Time: $O(V + E)$	Explain why we need outer loop.
Space: $O(V)$	

3 NUMBER OF ISLANDS

Gd

Base Template: Outer Loop DFS (Grid DFS + Outer Loop)

Problem

Count number of islands (groups of 1s) in a 2D grid.

Changed Lines (from Template)

```
// 1. Outer loop over the entire grid
int count = 0;
for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        if (grid[r][c] == '1' && !visited[r][c]) {
            count++;
            dfs(r, c, grid, visited);
        }
    }
}

// 2. DFS remains same (mark all connected land)
```

Caller	for all cells (r, c)	Return	int (count)	Visited	boolean[][]
--------	----------------------	--------	-------------	---------	-------------

Every new unvisited land cell = new island.

Complexity	Interview Tip
Time: $O(m * n)$	Don't forget to mark visited.
Space: $O(m * n)$	

4 FLOOD FILL

Gd

Base Template: Grid DFS

Problem

Replace the starting pixel's color with a new color.

Changed Lines (from Template)

```
// 1. Store starting color
char originalColor = image[sr][sc];

// 2. Add color check
if (image[r][c] != originalColor) return;

// 3. Change the color
image[r][c] = newColor;

// 4. Rest same (visited + 4 directions)
```

Caller	dfs(sr, sc, image, visited, newColor, originalColor)	Return	void	Visited	boolean[][]
--------	--	--------	------	---------	-------------

Only change pixels with the original color.

Complexity	Interview Tip
Time: $O(m * n)$	Boundary + color check prevent extra work.
Space: $O(V * n)$	

5 PACIFIC ATLANTIC WATER FLOW

Bd

Base Template: Border DFS (Multi-Source DFS)

Problem

Find cells that can flow water to both Pacific and Atlantic.

Changed Lines (from Template)

```
// 1. Run DFS from all Pacific borders
for (int r = 0; r < m; r++) dfs(r, 0, heights, pacific);
for (int c = 0; c < n; c++) dfs(0, c, heights, pacific);

// 2. Run DFS from all Atlantic borders
for (int r = 0; r < m; r++) dfs(r, n-1, heights, atlantic);
for (int c = 0; c < n; c++) dfs(m-1, c, heights, atlantic);

// 3. Collect intersection
if (pacific[r][c] && atlantic[r][c]) add to answer;
```

Caller	Multiple border starts	Return	List<List<Integer>>	Visited	boolean[][] (two arrays)
--------	------------------------	--------	---------------------	---------	--------------------------

Flow is reverse. Start DFS from oceans.

Complexity	Interview Tip
Time: $O(m * n)$	Don't do DFS from every cell!
Space: $O(m * n)$	

6 SURROUNDED REGIONS

Bd

Base Template: Border DFS

Problem

Capture all regions not connected to boundary.

Changed Lines (from Template)

```
// 1. Mark boundary connected '0's
Run DFS from all borders and mark as 'safe'

// 2. Flip remaining '0's
for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        if (board[r][c] == '0' && !safe[r][c])
            board[r][c] = 'X';
    }
}
```

Caller	All border cells	Return	void	boolean[][] (safe)	
--------	------------------	--------	------	--------------------	--

Only border-connected regions are safe.

Interview Tip	Interview Tip
Time: $O(m * n)$	Two step thinking is the key.
Space: $O(m * n)$	

7 CLONE GRAPH

N

Base Template: Node Graph DFS

Problem

Clone an undirected graph.

Changed Lines (from Template)

```
// Already handled in template
// Key idea:
// If node not in map -> create clone
// For each neighbor -> recurse and add edge
```

Caller	dfs(node, map)	Return	Node (cloned)	Visited	HashMap<Node, Node>
--------	----------------	--------	---------------	---------	---------------------

HashMap avoids infinite loop in cycles.

Complexity	Interview Tip
Time: $O(V + E)$	Always check map before creating clone.
Space: $O(V)$	

QUICK REFERENCE (CHANGES OVERVIEW)

Problem	Base Template	Main Change	Return
1 Valid Path	G	Early stop + boolean	boolean
2 Connected Components	OL	Outer loop + count	int
3 Number of Islands	OL + Gd	Grid outer loop + count	int
4 Flood Fill	Gd	Color check + change	void
5 Pacific Atlantic	Bd	Multi-source + intersect	List<List<Integer>>
6 Surrounded Regions	Bd	Mark safe + flip rest	void
7 Clone Graph	N	HashMap for visited	Node

G Graph DFS Gd Grid DFS N Node DFS OL Outer Loop DFS Bd Border DFS

8 INTERVIEW CHECKLIST

- ☒ Identified the correct template?
- ☒ Built correct representation?
- ☒ Initialized visited structure correctly?
- ☒ Handled all starting points?
- ☒ Applied boundary / color / condition checks?
- ☒ Returned correct type?
- ☒ Considered disconnected components?
- ☒ Can you explain time/space complexity?
- ☒ Dry run on small example?
- ☒ Optimized when possible?



HOW TO USE THIS PAGE

- 1 Find the problem.
- 2 Identify the Base Template (from Page 6).
- 3 Read the CHANGED LINES.
- 4 Understand the AHA.
- 5 Review the complexity & interview tip.
- 6 Practice writing from memory.



TEMPLATE LEGEND (From Page 6)

Gb Graph BFS
(Adj List)**Gr** Grid BFS**Ms** Multi-Source
BFS**Lv** Level Order
BFS**Ds** Distance
BFS

1 ROTTING ORANGES

Ms

Base Template: Multi-Source BFS (Page 6)

Problem

Return minutes until all oranges rot. Return -1 if impossible.

Changed Lines (from Template)

```
// 1. Push all rotten oranges (sources)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (grid[i][j] == 2) {
            q.offer(new int[]{i, j});
        }
    }
}

// 2. Count fresh oranges
fresh = count all 1's in grid;

// 3. While loop level++ gives time (minutes)
int minutes = 0;
while (!q.isEmpty() && fresh > 0) {
    int size = q.size();
    minutes++;
    // process level
}

return fresh == 0 ? minutes : -1;
```

Caller	Return	Visited	Key Idea
multiSourceBfs()	int (minutes)	boolean[][]	Spread level by level

AHA: Multiple starting rotten oranges spread at the same time.

Complexity
Time: $O(n * m)$
Space: $O(n * m)$

Interview Tip
Don't forget to count fresh oranges before BFS.

2 WALLS AND GATES

Ms

Base Template: Multi-Source BFS

Problem

Fill each empty room with distance to nearest gate.

Changed Lines (from Template)

```
// 1. Push all gates (0) into queue
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (rooms[i][j] == 0) {
            q.offer(new int[]{i, j});
        }
    }
}

// 2. When visiting empty room, update distance
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : dirs) {
        int nr = cur[0] + d[0], nc = cur[1] + d[1];
        if (inBounds(nr, nc) && rooms[nr][nc] == INF) {
            rooms[nr][nc] = rooms[cur[0]][cur[1]] + 1;
            q.offer(new int[]{nr, nc});
        }
    }
}
```

Caller	Return	Visited	Key Idea
multiSourceBfs()	void (in-place)	rooms[][]	Distance from nearest gate

AHA: Gates are sources. Distances expand outward.

Complexity
Time: $O(n * m)$
Space: $O(n * m)$

Interview Tip
Use original rooms[][] as visited (INF = unvisited).

3 SHORTEST PATH IN BINARY MATRIX

Ds

Base Template: Grid BFS (Distance BFS)

Problem

Find shortest path from top-left to bottom-right in binary matrix.

Changed Lines (from Template)

```
// 1. If start or end is blocked
if (grid[0][0] == 1 || grid[n-1][m-1] == 1) return -1;

// 2. Start from (0,0)
q.offer(new int[]{0, 0});
visited[0][0] = true;
int steps = 1;

// 3. Level = path length
while (!q.isEmpty()) {
    int size = q.size();
    for (int k = 0; k < size; k++) {
        int[] cur = q.poll();
        if (cur[0] == n-1 && cur[1] == m-1) return steps;
        for (int[] d : dirs4) {
            // 8 directions
            int nr = cur[0] + d[0], nc = cur[1] + d[1];
            if (inBounds(nr, nc) && grid[nr][nc] == 0 && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
    steps++;
}

return -1;
```

Caller	Return	Visited	Key Idea
bfs(0,0)	int (steps)	boolean[][]	Level = path length

AHA: First time we reach destination = shortest path.

Complexity
Time: $O(n * m)$
Space: $O(n * m)$

Interview Tip
Don't forget 8 directions.

4 WORD LADDER (LENGTH)

Gb

Base Template: Graph BFS (Adj List) (Distance BFS)

Problem

Return length of shortest transformation sequence.

Changed Lines (from Template)

```
// 1. Build graph: word -> all one-letter transformations
Map<String, List<String>> graph = buildGraph(wordList);

// 2. BFS from beginWord
Queue<String> q = new LinkedList<>();
q.offer(beginWord);
visited.add(beginWord);
int steps = 1;
while (!q.isEmpty()) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
        String cur = q.poll();
        if (cur.equals(endWord)) return steps;
        for (String nei : graph.get(cur)) {
            if (!visited.contains(nei)) {
                visited.add(nei);
                q.offer(nei);
            }
        }
    }
    steps++;
}

return 0;
```

Caller	Return	Visited	Key Idea
bfs(beginWord)	int (length)	HashSet<String>	Level = sequence length

AHA: BFS guarantees first time we reach endWord is shortest.

Complexity
Time: $O(N * L^2)$
Space: $O(N * L^2)$

Interview Tip
Precompute neighbors to save time.

5 NEAREST EXIT FROM ENTRANCE IN MAZE

Gr

Base Template: Grid BFS (Distance BFS)

Problem

Find minimum steps to nearest exit from entrance.

Changed Lines (from Template)

```
// 1. Start from entrance only
q.offer(new int[]{entrance[0], entrance[1]});
visited[entrance[0]][entrance[1]] = true;
int steps = 0;

// 2. Exit = boundary cell (not entrance)
while (!q.isEmpty()) {
    int size = q.size();
    for (int k = 0; k < size; k++) {
        int[] cur = q.poll();
        if (isBoundary(r, c) && (r != entrance[0] || c != entrance[1])) {
            return steps;
        }
        for (int[] d : dirs4) {
            int nr = cur[0] + d[0], nc = cur[1] + d[1];
            if (inBounds(nr, nc) && maze[nr][nc] == '.' && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
    steps++;
}

return -1;
```

Caller	Return	Visited	Key Idea
bfs(entrance)	int (steps)	boolean[][]	First boundary cell found

AHA: BFS expands in layers. First exit found is minimum steps.

Complexity
Time: $O(n * m)$
Space: $O(n * m)$

Interview Tip
Don't treat entrance itself as exit.

6 01 MATRIX (UPDATE MATRIX)

Ms

Base Template: Multi-Source BFS

Problem

For each cell, distance to nearest 0.

Changed Lines (from Template)

```
// 1. Push all cells with 0
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (mat[i][j] == 0) q.offer(new int[]{i, j});
    }
}

// 2. Process and update 1's with distance
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : dirs4) {
        int nr = cur[0] + d[0], nc = cur[1] + d[1];
        if (inBounds(nr, nc) && mat[nr][nc] == 1) {
            mat[nr][nc] = mat[cur[0]][cur[1]] + 1;
            q.offer(new int[]{nr, nc});
        }
    }
}
```

Caller	Return	Visited	Key Idea
multiSourceBfs()	int[][] (in-place)	mat[][]	Distance from nearest 0

AHA: All zeros start together, distances fill the matrix.

Complexity
Time: $O(n * m)$
Space: $O(n * m)$

Interview Tip
Use matrix itself to store distances.

7 CHANGED LINES SUMMARY (QUICK REFERENCE)

#	Problem	Base Template	Only What Changes
1	Rotting Oranges	Multi-Source BFS	Push all rotten, count fresh, level = minutes
2	Walls and Gates	Multi-Source BFS	Push all gates, update room[nr][nc]
3	Shortest Path Binary Matrix	Grid BFS (Distance)	Start (0,0), level = steps, 8 directions
4	Word Ladder (Length)	Graph BFS (Distance)	Build graph, return level when end found
5	Nearest Exit in Maze	Grid BFS (Distance)	Exit = boundary cell, not entrance
6	01 Matrix	Multi-Source BFS	Push all 0's, update 1's with distance

8 BFS TEMPLATES USED (FROM PAGE 6)

Gb Graph BFS (Adj List)	Traverse graph level by level using queue.
Gr Grid BFS	Traverse grid using 4/8 directions.
Ms Multi-Source BFS	Start BFS from multiple sources at once.
Lv Level Order BFS	Process nodes level by level using queue size.
Ds Distance BFS	Level / steps represent shortest distance.

9 INTERVIEW CHECKLIST

- ☒ Did I choose the correct BFS template?
- ☒ Is my queue initialization correct?
- ☒ Am I marking visited at the right time?
- ☒ Is level / steps updated in the right place?
- ☒ Did I handle all boundary & edge cases?
- ☒ Is my return value correct?
- ☒ What is the time & space complexity?
- ☒ Can I optimize further?

BFS PROBLEM SOLVING FLOW (INTERVIEW APPROACH)





“Merge sets. Detect cycles. Count components. Super fast.”

1 MASTER IDEA

Union Find helps us manage disjoint sets.

It answers:

Are two nodes connected?

It helps in:

Cycle Detection

Components Count

Dynamic Connectivity

2 CORE OPERATIONS

Operation	Purpose
find(x)	Find root/parent of x (With Path Compression)
union(a, b)	Merge two sets (With Union by Rank/Size)
connected(a, b)	Check if a and b belong to same set
components	Number of disjoint sets

TIP: Path Compression + Union by Rank makes operations almost O(1) (α(N)).

3 UNION FIND TEMPLATE (OPTIMIZED)

Java Template

```
class UnionFind {
    private int[] parent;
    private int[] rank; // or use size[]
    private int count; // number of components

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        count = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    // Find with Path Compression
    public int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    // Union by Rank
    public boolean union(int a, int b) {
        int rootA = find(a);
        int rootB = find(b);
        if (rootA == rootB) return false; // already connected

        if (rank[rootA] < rank[rootB]) parent[rootA] = rootB;
        else if (rank[rootA] > rank[rootB]) parent[rootB] = rootA;
        else {
            parent[rootB] = rootA;
            rank[rootA]++;
        }
        count--;
        return true;
    }

    public boolean connected(int a, int b) {
        return find(a) == find(b);
    }

    public int components() {
        return count;
    }
}
```

4 VARIANTS

Union by Size (Alternative)

Use size[] instead of rank[]

- Attach smaller tree under bigger tree
- Update size of new root

Path Compression (Essential)

Always compress path in find(x) for almost O(1) performance.

5 MULTI-SET OPERATIONS

connected(a, b)	Same root?
union(a, b)	Merge sets
find(x)	Get root of x
components[]	Count disjoint sets

Example (N = 6)

Components = 2 Sets: {0,1,2,3}, {4,5}

6 WHAT ACTUALLY CHANGES?

Problem Type	What We Do	How We Use Union Find	Example Problems
Cycle Detection (Undirected)	For each edge (u,v)	If connected(u,v) → cycle Else union(u,v)	Redundant Connection Graph Valid Tree
Cycle Detection (Directed)	Topological sort recommended	Not ideal for directed cycles	Course Schedule (use Topo Sort)
Components Count	For each edge (u,v)	union(u,v) always Return uf.components()	Number of Provinces
Max Area / Island	Union adjacent cells	Map cell → id, union neighbors	Making A Large Island
Dynamic Connectivity	Online union queries	Process union & connected	Friend Circles (online)

TIP: Problem decision depends on whether graph is directed, weighted and what to return.

7 WHEN TO USE UNION FIND?

Use When...

- ✓ You need to check if two nodes are connected.
- ✓ You need to detect a cycle in an undirected graph.
- ✓ You need to count number of components.
- ✓ You need to connect cells dynamically.
- ✓ You have many union/find queries.

Do NOT Use When...

- ✗ Graph is directed and you need cycle detection.
- ✗ You need shortest path.
- ✗ You need ordering of nodes.
- ✗ You need traversal / component details.

Quick Decision

8 COMPLEXITY

find(x)	O(α(N))
union(a, b)	O(α(N))
connected(a, b)	O(α(N))
components()	O(1)

Where α(N) is Inverse Ackermann Function

- α(N) ≤ 4 for all practical N
- Nearly constant time!
- Best data structure for connectivity.

Space Complexity

O(N) for parent[] + rank[]

9 RETURN TYPE GUIDE

Return Type	Purpose	Typical Problems	Interview Meaning
boolean	Check if connected / cycle exists	Redundant Connection, Graph Valid Tree	Validity / Feasibility check
int	Count components	Number of Provinces	Count groups
void	Just perform unions	Build DSU then answer queries	Preprocessing / Setup
List / Set	List of components	Connected Components (Advanced)	Return actual groups

10 COMMON INTERVIEW MISTAKES

- ✗ Forgetting path compression.
- ✗ Union without checking roots.
- ✗ Wrong component count.
- ✗ Not handling 0-index / 1-index.
- ✗ Not initializing parent[i] = i.
- ✗ Using DSU for directed cycle.
- ✗ Using DSU for shortest path.
- ✗ Forgetting edge cases.

11 INTERVIEW THINKING FLOW

12 PAGE 11 PREVIEW

Next page contains UNION FIND PROBLEM GUIDE. For each problem, you will see:

Base Template

Changed Lines

AHA Moment

Complexity

Interview Tip

Apply these templates with tiny changes and solve any Union Find problem with confidence!

Problem (Understand)

Input (What given)

Representation (Graph + DSU)

Helper Structure (Parent[], Rank[])

Pattern (Union Find)

Tiny Logic Change (Apply Changes)

Dry Run (Example)

Optimize (Time/Space)



“Same idea. Different problems. Only the changing parts.”

HOW TO USE THIS PAGE

- 1 Find the problem.
- 2 Identify the Base Template (from Page 10).
- 3 Read the CHANGED LINES.
- 4 Understand the AHA.
- 5 Review complexity & interview tip.
- 6 Practice writing from memory.



TEMPLATE LEGEND (From Page 10)



Basic UF
Find + Union
by Rank



Cycle Detection
(Undirected)



Number of
Components



Max Area / Island
(Union Adjacent
Cells)



Dynamic
Connectivity
(Online Queries)

1 REDUNDANT CONNECTION



Base Template: Cycle Detection (Undirected)

Problem

Find the edge that creates a cycle (first redundant edge).

Changed Lines (from Template)

```
// For each edge
for (int[] e : edges) {
    int u = e[0], v = e[1];
    if (connected(u, v)) return e; // cycle found
    union(u, v);
}
return new int[0];
```

Caller

findRedundantConnection(edges)

Return

int[2]
(edge)

Key Idea

If already connected,
this edge is extra.

AHA: The first edge connecting two already connected nodes forms a cycle.

Complexity

Time: $O(N \alpha(N))$
Space: $O(N)$

Interview Tip

Stop and return immediately when cycle found.

2 GRAPH VALID TREE



Base Template: Cycle Detection + Components Count

Problem

Check if edges form a valid tree.

Changed Lines (from Template)

```
// n = number of nodes
if (edges.length != n - 1) return false; // must have n-1 edges

for (int[] e : edges) {
    int u = e[0], v = e[1];
    if (connected(u, v)) return false; // cycle
    union(u, v);
}
return count == 1; // single component
```

Caller

validTree(n, edges)

Return

boolean

Key Idea

No cycle + single
component

AHA: Tree = (n-1 edges) + (No cycle) + (Connected)

Complexity

Time: $O(N \alpha(N))$
Space: $O(N)$

Interview Tip

Always check edges.length == n - 1 first.

3 NUMBER OF PROVINCES



Base Template: Components Count

Problem

Count how many connected components (provinces) exist.

Changed Lines (from Template)

```
// isConnected[i][j] = 1 if i and j are connected
int n = isConnected.length;
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (isConnected[i][j] == 1) union(i, j);
    }
}
return count; // number of components
```

Caller

findCircleNum(isConnected)

Return

int

Key Idea

Union every connected
pair, return count

AHA: Each component represents one province.

Complexity

Time: $O(N^3 \alpha(N))$
Space: $O(N)$

Interview Tip

Matrix input → check only upper triangle.

4 SURROUNDED REGIONS



Base Template: Max Area / Island (Union Adjacent Cells)

Problem

Flip all 'O' that are surrounded by 'X' to 'X'.

'O' connected to boundary stays 'O'.

Changed Lines (from Template)

```
// 1. Add a dummy node: dummy = m*n
// 2. Union boundary 'O' with dummy
// 3. Union all 'O' with 4 neighbors
// 4. Final: if connected(cell, dummy) -> keep 'O' else -> 'X'
```

Caller

solve(board)

Return

void

Key Idea

Boundary safe 'O' connected
to dummy node

AHA: Dummy node represents all boundary 'O'.

Complexity

Time: $O(M*N \alpha(M*N))$
Space: $O(M*N)$

Interview Tip

Process boundary first, then inner cells.

5 MAKING A LARGE ISLAND



Base Template: Max Area / Island

Problem

Flip at most one 0 to 1 to get the largest island size.

Changed Lines (from Template)

```
// 1. Union all existing 1's and calc size of each root
// 2. For each 0 cell, try making it 1
// Take unique neighbor roots, sum their sizes + 1
```

Caller

largestIsland(grid)

Return

int

Key Idea

Merge unique neighboring
islands around 0

AHA: Use Set to avoid double counting same island.

Complexity

Time: $O(M*N \alpha(M*N))$
Space: $O(M*N)$

Interview Tip

Don't forget +1 for the flipped cell itself.

6 NUMBER OF OPERATIONS TO CONNECT NETWORK



Base Template: Components Count

Problem

Return minimum operations to connect all computers.

Changed Lines (from Template)

```
// After processing all connections
int components = count; // number of components
int extra = connections.length - (n - components);
// extra edges available
return (extra >= components - 1) ? components - 1 : -1;
```

Caller

makeConnected(n, connections)

Return

int

Key Idea

Need (components-1)
extra edges

AHA: Extra edges = totalEdges - (n - components)

Complexity

Time: $O(N \alpha(N))$
Space: $O(N)$

Interview Tip

Check if enough extra edges exist.

7 CHANGED LINES SUMMARY

#	Problem	Base Template	What Changes
1	Redundant Connection	Cycle Detection	Return edge when connected(u,v) is true
2	Graph Valid Tree	Cycle + Components	Edges must be n-1 & count must be 1
3	Number of Provinces	Components Count	Union all connected pairs in matrix
4	Surrounded Regions	Max Area / Island	Use dummy node for boundary 'O'
5	Making a Large Island	Max Area / Island	Try flipping 0, merge unique neighbor roots
6	Make Network Connected	Components Count	Use extra edges to connect components

8 UNION FIND TEMPLATE RECAP (From Page 10)

Find with Path Compression

```
if (parent[x] != x)
    parent[x] = find(parent[x]);
return parent[x];
```

Union by Rank

```
if (rank[a] < rank[b]) parent[a] = b;
else if (rank[a] > rank[b]) parent[b] = a;
else { parent[b] = a; rank[a]++; }
```

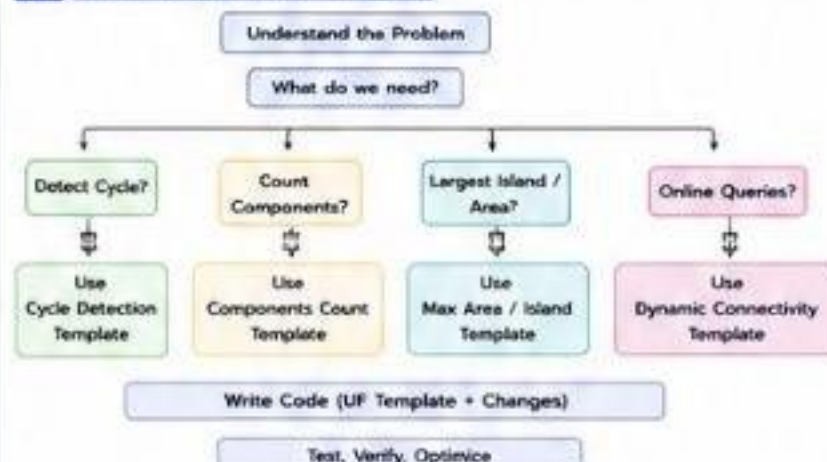
9 RETURN TYPE GUIDE

Return Type	Use When	Example Problems
boolean	Check cycle / Validity	Graph Valid Tree
int	Count / Min Ops / Size	Provinces, Operations to Connect, Island Size
int[]	Return edge / Pair	Redundant Connection
void	Modify structure (in-place)	Surrounded Regions

10 INTERVIEW CHECKLIST

- ✓ Did I initialize parent[i] = i and rank[i] = 1?
- ✓ Did I use path compression in find()?
- ✓ Did I union by rank/size?
- ✓ Am I checking connected(u, v) correctly?
- ✓ Am I handling 0-index / 1-index properly?
- ✓ Did I think about components count?
- ✓ Any edge cases (single node, no edges)?
- ✓ Is my time complexity $O(\alpha(N))$ or $O(N \alpha(N))$?

11 INTERVIEW THINKING FLOW



12 PAGE 12 PREVIEW

Next page contains DIJKSTRA (SHORTEST PATH) TEMPLATE GUIDE.

For each problem, you will see:



Apply these templates with tiny changes and solve any Dijkstra problem with confidence!



"Optimized for non-negative weights."

1 MASTER IDEA



Dijkstra finds the shortest path from a source to all other nodes in a weighted graph with non-negative weights.

Always pick the node with the smallest known distance.

Once picked, its distance is final.

2 CORE COMPONENTS

Component	Purpose
dist[]	Shortest known distance from source to each node
Priority Queue	Pick node with minimum distance (Min Heap)
visited[] / inPQ[]	Avoid processing a node multiple times
Relaxation	If better path found, update distance
Parent[] (opt.)	To reconstruct shortest path

TIP: Dijkstra does NOT work with negative edge weights.

3 DIJKSTRA TEMPLATE (ADJACENCY LIST) ★★★★★

Java Template

```
class Dijkstra {
    static class Pair {
        int node, dist;
        Pair(int n, int d) { node = n; dist = d; }
    }

    public static int[] dijkstra(List<List<Pair>> graph, int n, int src) {
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> a.dist - b.dist);
        pq.offer(new Pair(src, 0));

        boolean[] visited = new boolean[n];

        while (!pq.isEmpty()) {
            Pair cur = pq.poll();
            int u = cur.node, d = cur.dist;
            if (visited[u]) continue; // already finalized
            visited[u] = true;

            for (Pair nei : graph.get(u)) {
                int v = nei.node, w = nei.dist;
                if (d + w < dist[v]) { // relax
                    dist[v] = d + w;
                    pq.offer(new Pair(v, dist[v]));
                }
            }
        }

        return dist; // dist[i] = shortest distance from src to i
    }
}
```

AHA MOMENTS

- 🕒 We always expand the closest unprocessed node.
- 🕒 First time a node is popped from PQ, its distance is final.
- 🕒 Relaxation improves neighbors only.
- 🕒 Work only for non-negative weights.
- 🕒 Same idea used in GPS navigation!

COMMON MISTAKES

- ❌ Using Dijkstra with negative edges.
- ❌ Forgetting to check if node already visited.
- ❌ Not using long when dist can overflow (use long for large sums).
- ❌ Relaxing from outdated distance.
- ❌ Pushing same node many times without visited check.

4 DIJKSTRA TEMPLATE (ADJACENCY MATRIX) ★★★★★

Java Template

```
public static int[] dijkstraMatrix(int[][] mat, int n, int src) {
    int[] dist = new int[n];
    boolean[] visited = new boolean[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        // Pick min dist unvisited node
        int u = -1, min = Integer.MAX_VALUE;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && dist[i] < min) { min = dist[i]; u = i; }
        }
        visited[u] = true;

        // Relax all neighbors of u
        for (int v = 0; v < n; v++) {
            if (mat[u][v] > 0 && !visited[v] && dist[u] != Integer.MAX_VALUE && dist[u] + mat[u][v] < dist[v]) {
                dist[v] = dist[u] + mat[u][v];
            }
        }
    }

    return dist;
}
```

5 WHAT CHANGES? ★★★★★

Scenario	Change	Only This Part Changes
Different Source	Change src	Initialization dist[src] = 0
Get Path	Add parent[]	When relaxing, set parent[v] = u
Count Paths	Add count[]	If equal dist found, count[v] += count[u]
Min Cost with Steps Limit	Use (node, dist, steps) in PQ	Push with steps+1, check limit
Multiple Queries	Run Dijkstra multiple times	Reuse graph
Undirected Graph	Add both directions in adjacency	Graph build step

6 RETURN TYPE GUIDE ★★★★★

Return Type	Use When	Example Problems
int[] dist	Need shortest distance from src to all nodes	Network Delay Time, Dijkstra, Path With Minimum Effort (variant)
long[] dist	Distances can be large (sum > 2^31-1)	Large graphs with big weights
Pair (dist, path)	Need actual path reconstruction	Shortest Path in Weighted Graph
boolean (reachable)	Only check if target is reachable	Is Path Exists with Min Cost ≤ K
Custom Object	Need dist + other info (steps, ways, etc.)	Number of Ways to Arrive at Destination

TIP: For large values use long, not int.

7 WHEN TO USE DIJKSTRA?

Use When...

- ✓ All edge weights are non-negative.
- ✓ Need shortest path from one source to all nodes.
- ✓ Graph can be directed or undirected.
- ✓ Need fastest reliable algorithm.

Do NOT Use When...

- ❌ Graph has negative weight edges (Use Bellman-Ford).
- ❌ Need shortest path between all pairs (Use Floyd-Warshall).
- ❌ Graph is unweighted (Use BFS).

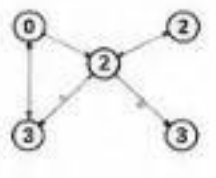
Decision Flow



8 COMPLEXITY ★★★★★

Adjacency List + PQ (Best Implementation)

Time: $O((V + E) \log V)$
Space: $O(V + E)$



Where:
V = number of vertices
E = number of edges

Adjacency Matrix (Simple Implementation)

Time: $O(V^2)$
Space: $O(V^2)$

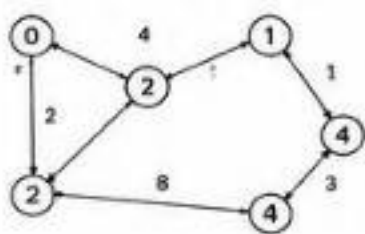
	0	1	2	3
0	0	4	0	1
1	4	0	2	0
2	0	2	0	3
3	1	0	3	0

9 DIJKSTRA SUMMARY

#	Step	What Happens
1	Initialize	dist[src] = 0, others = INF Push (src, 0) in PQ
2	Extract Min	Pop node u with smallest dist
3	Skip if Visited	If already visited, continue
4	Relax Neighbors	For each neighbor v of u, update dist[v] if better path
5	Repeat	Continue until PQ is empty
6	Done	dist[] has shortest paths

10 MINI EXAMPLE (ADJ LIST)

Graph (Undirected)



Final dist[] = [0, 3, 1, 4, 7]

Source = 0

Dijkstra Steps:

1. dist = [0, ∞, ∞, ∞, ∞]
PQ = [(0,0)]
2. Pop 0 → relax 1(4), 2(1)
dist = [0, 4, 1, ∞, ∞]
3. Pop 2 → relax 1(5), 3(6), 4(9)
dist = [0, 3, 1, 6, 9]
4. Pop 1 → relax 3(4) (better)
dist = [0, 3, 1, 4, 9]
5. Pop 3 → relax 4(7) (better)
dist = [0, 3, 1, 4, 7]
6. Pop 4 → done.

11 INTERVIEW CHECKLIST

- ✓ Did I build the graph correctly?
- ✓ Did I initialize dist[] correctly?
- ✓ Did I use Min Priority Queue?
- ✓ Did I check visited[] before relaxing?
- ✓ Did I relax all neighbors?
- ✓ Did I update and push to PQ only when better?
- ✓ Did I handle large values (use long if needed)?
- ✓ Did I think about path reconstruction if asked?
- ✓ Did I return correct type and values?
- ✓ Can I dry run on a small example?

Interview Tip

- Always communicate:
- What the algo does
 - Why it works
 - Complexity
 - How you handle edge cases

12 PAGE 13 PREVIEW

Next page contains DIJKSTRA PROBLEM GUIDE.
For each problem, you will see:

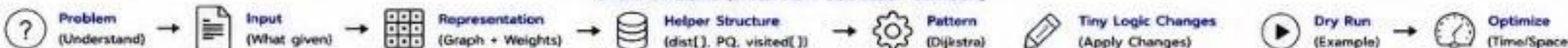


Apply these templates with tiny changes and solve any Dijkstra problem with confidence!

13 INTERVIEW THINKING FLOW



CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)



HOW TO USE THIS PAGE

1 Find the problem.

2 Identify the base template.

3 Read the CHANGED LINES.

4 Understand the AHA.

5 Review complexity & interview tip.

6 Practice writing from memory.

TEMPLATE LEGEND (From Page 14)

A Network Delay (Dijkstra)

B Dijkstra with Path

C Minimum Effort Path

D Cheapest Flights (K Stops)

E Path with Minimum Effort

F Shortest Path in Undirected Graph

1 CLONE GRAPH

S

Base Template: DFS (Graph Traversal)

Problem

Given a reference of a node in a connected undirected graph, return a deep copy (clone) of the graph.

Changed Lines (from Template)

```
Map<Node, Node> map = new HashMap<>();
private Node clone(Node node) {
    if (node == null) return null;
    if (map.containsKey(node)) return map.get(node); // already cloned
    Node copy = new Node(node.val); // clone current
    map.put(node, copy);
    for (Node nei : node.neighbors) {
        copy.neighbors.add(clone(nei));
    }
    return copy;
}
```

Caller

clone(node)

Return

Node

Key Idea

Use map to avoid infinite loop & duplicates

AHA: Store cloned node before traversing neighbors.

Complexity

Time: $O(V + E)$

Space: $O(V)$

Interview Tip

Essentially a graph + hash map problem.

2 SURROUNDED REGIONS

I

Base Template: DFS from Boundary

Problem

Capture all regions that are surrounded by 'X'. Regions connected to boundary stay 'O'.

Changed Lines (from Template)

```
// 1. DFS from all boundary 'O'
for (int i = 0; i < m; i++) {
    dfs(i, 0); dfs(i, n-1);
}
for (int j = 0; j < n; j++) {
    dfs(0, j); dfs(m-1, j);
}
// 2. Flip inside 'O' to 'X'
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (board[i][j] == 'O') board[i][j] = 'X';
        if (board[i][j] == 'E') board[i][j] = 'O';
    }
}
// dfs() marks reachable 'O' as 'E'
```

Caller

solve(board)

Return

void

Key Idea

Mark safe 'O's first, then flip the rest

AHA: Only boundary-connected 'O's are safe.

Complexity

Time: $O(m * n)$

Space: $O(m * n)$

Interview Tip

Classic boundary DFS pattern.

3 PACIFIC ATLANTIC WATER FLOW

I

Base Template: Multi-Source BFS / DFS

Problem

Find all cells where water can flow to both Pacific (top/left) and Atlantic (bottom/right) oceans.

Changed Lines (from Template)

```
// 1. BFS/DFS from all Pacific borders (mark visitP)
// 2. BFS/DFS from all Atlantic borders (mark visitA)
// 3. Cells reachable in both -> answer
// Condition: move to neighbor with >= height
public List<List<Integer>> pacificAtlantic(int[][] h) {
    int m = h.length, n = h[0].length;
    boolean[][] p = new boolean[m][n];
    boolean[][] a = new boolean[m][n];

    // bfs(pacific) & bfs(atlantic)
    // collect cells where p[i][j] && a[i][j]
}
```

Caller

pacificAtlantic(h)

Return

List<List<Integer>>

Key Idea

Reverse flow: ocean -> cells

AHA: Think reverse - water flows from ocean to land.

Complexity

Time: $O(m * n)$

Space: $O(m * n)$

Interview Tip

Two traversals + intersection.

4 ROTTING ORANGES

I

Base Template: Multi-Source BFS

Problem

Each minute, a rotten orange makes adjacent fresh oranges rotten. Return time to rot all or -1.

Changed Lines (from Template)

```
Queue<int[]> q = new LinkedList<>();
int fresh = 0, time = 0;
// 1. Add all rotten oranges to queue & count fresh
// 2. BFS level by level
while (!q.isEmpty() && fresh > 0) {
    int size = q.size();
    time++;
    while (size-- > 0) {
        int[] cur = q.poll();
        // infect fresh neighbors
    }
}
return fresh == 0 ? time : -1;
```

Caller

orangesRotting(grid)

Return

int

Key Idea

Level order = minutes

AHA: Processing one level = one minute.

Complexity

Time: $O(m * n)$

Space: $O(m * n)$

Interview Tip

Classic BFS with multi-source.

5 COURSE SCHEDULE II

K

Base Template: Topological Sort (BFS)

Problem

Return any valid order to finish all courses. If impossible, return [].

Changed Lines (from Template)

```
// Instead of just checking cycle, store order
List<Integer> order = new ArrayList<>();
Queue<Integer> q = new LinkedList<>();

// push all indegree 0
while (!q.isEmpty()) {
    int node = q.poll();
    order.add(node);
    for (int nei : graph.get(node)) {
        indegree[nei]--;
        if (indegree[nei] == 0) q.offer(nei);
    }
}
return order.size() == numCourses ? order : new ArrayList<>();
```

Caller

findOrder(n, prerequisites)

Return

int[]

Key Idea

If order size != n, cycle exists

AHA: Topo sort can produce order, not just detect cycle.

Complexity

Time: $O(V + E)$

Space: $O(V + E)$

Interview Tip

BFS topo sort is iterative & safe.

6 GRAPH VALID TREE

K

Base Template: Union Find / DFS

Problem

Determine if an undirected graph is a valid tree.

Changed Lines (from Template)

```
// For Union Find
if (edges.length != n - 1) return false;
for (int[] e : edges) {
    if (union(e[0], e[1])) return false; // cycle
}
return true;

// OR DFS: Check connected + no cycle
```

Caller

validTree(n, edges)

Return

boolean

Key Idea

Tree = (n-1 edges) + connected + no cycle

AHA: Quick checks save time: edges == n-1.

Complexity

Time: $O(V + E)$

Space: $O(V)$

Interview Tip

Union Find is fastest approach.

7 CHANGED LINES SUMMARY

8 COMPLEXITY QUICK GUIDE

9 RETURN TYPE GUIDE

10 INTERVIEW CHECKLIST

11 INTERVIEW THINKING FLOW

Understand Problem

Identify Graph Type

Pick Base Template

Apply Changes from Template

Test on Examples & Edge Cases

Optimize & Explain

TIP: Before coding - explain your plan. Communication matters!

12 PAGE 16 PREVIEW

Next page contains GRAPH PATTERN CHEAT SHEETS. For each pattern, you will see:

Pattern Summary

When to Use

Key Steps

Template Snapshot

Common Pitfalls

Complexity

Interview Tip

One-page cheatsheets. Quick recall before interviews!

CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)

Problem (Understand)

Input (What given)

Representation (Graph + Weights)

Choose Pattern (Best fit)

Helper Structure (PQ / Visited / DSU)

Logic Change (Apply Minimal)

Dry Run (Example)

Optimize (Time / Space)



HOW TO USE THIS PAGE

- 1 Find the problem.
- 2 Identify the base template (from Page 13).
- 3 Read the CHANGED LINES.
- 4 Understand the AHA.
- 5 Review complexity & interview tip.
- 6 Practice writing from memory.



TEMPLATE LEGEND (From Page 13)

A Network Delay Time**B** Dijkstra with Path**C** Minimum Effort Path**D** Cheapest Flights (K Stops)**E** Path with Minimum Effort**F** Shortest Path in Undirected Graph

1 SWIM IN RISING WATER

C

Base Template: Dijkstra (Minimize Max Edge)

Problem

Find minimum time to reach bottom-right cell.
Water level must be $\geq$ max height on the path.

Changed Lines (from Template)

```
int n = heights.length;
int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]); // effort, x, y
pq.offer(new int[]{heights[0][0], 0, 0});
int[][] effort = new int[n][n];
for (int[] row : effort) Arrays.fill(row, Integer.MAX_VALUE);
effort[0][0] = heights[0][0];

// When relaxing neighbor (nx, ny)
int newEff = Math.max(curEff, heights[nx][ny]);
if (newEff < effort[nx][ny]) {
    effort[nx][ny] = newEff;
    pq.offer(new int[]{newEff, nx, ny});
}

return effort[n-1][n-1];
```

Caller	Return	Key Idea
swimInWater(grid)	int	Minimize maximum edge on path

AHA: Path cost = max(height) on the path.

Complexity
Time: $O(n^2 \log n)$
Space: $O(n^2)$

Interview Tip
Use Dijkstra with max() relaxation.

2 PATH WITH MAX PROBABILITY

B

Base Template: Dijkstra + Parent[]

Problem

Given success probability on edges, find path from start to end with maximum probability.

Changed Lines (from Template)

```
double[] prob = new double[n];
Arrays.fill(prob, 0.0);
prob[start] = 1.0;
PriorityQueue<double[]> pq = new PriorityQueue<>((a,b) -> Double.compare(b[0], a[0])); // max-heap
pq.offer(new double[]{1.0, start});

// When relaxing
double newProb = prob[node] * edgeProb;
if (newProb > prob[next]) {
    prob[next] = newProb;
    parent[next] = node;
    pq.offer(new double[]{newProb, next});
}

return prob[end];
```

Caller	Return	Key Idea
maxProbability(n, edges, succProb, start, end)	double	Maximize product of probabilities

AHA: Use max-heap. Multiply, don't add.

Complexity
Time: $O(V + E) \log V$
Space: $O(V + E)$

Interview Tip
Initialize prob[start] = 1.0
All others = 0.0

3 MINIMUM MULTIPLICATION TO REACH END

A

Base Template: Dijkstra (Single Source)

Problem

Given multipliers and modulo, find min multiplications to reach end.

Changed Lines (from Template)

```
PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]);
int[] dist = new int[mod];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[start] = 0;
pq.offer(new int[]{0, start});
while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    int steps = cur[0], node = cur[1];
    if (node == end) return steps;
    for (int x : arr) {
        int next = (node * x) % mod;
        if (steps + 1 < dist[next]) {
            dist[next] = steps + 1;
            pq.offer(new int[]{steps+1, next});
        }
    }
}
return -1;
```

Caller	Return	Key Idea
minimumMultiplications(arr, start, end, mod)	int	Dijkstra on mod space graph

AHA: Nodes are numbers $\{0 \dots \text{mod}-1\}$.

Complexity
Time: $O(\text{mod} * k \log \text{mod})$
Space: $O(\text{mod})$

Interview Tip
Use mod graph. Avoid revisiting with larger steps.

4 MOST STONES REMOVED WITH SAME ROW/COL

F

Base Template: Connected Components (DSU)

Problem

Stones are connected if they share row or column.
Return max stones removable.

Changed Lines (from Template)

```
// Map rows and columns to DSU nodes
int base = 10001; // shift for columns
DSU dsu = new DSU(base * 2);
for (int[] s : stones) {
    int r = s[0];
    int c = s[1] + base;
    dsu.union(r, c);
    rowSet.add(r);
    colSet.add(c);
}

int components = 0;
for (int x : rowSet) if (dsu.find(x) == x) components++;
for (int x : colSet) if (dsu.find(x + base) == x + base) components++;
return stones.length - components;
```

Caller	Return	Key Idea
removeStones(stones)	int	Answer = stones - number of components

AHA: Make virtual nodes for rows and columns.

Complexity
Time: $O(N \alpha(N))$
Space: $O(N)$

Interview Tip
DSU beats DFS/BFS for large N.

5 COURSE SCHEDULE II (FIND ORDER)

F

Base Template: Topological Sort (Kahn's BFS)

Problem

Return any valid order to finish all courses. If impossible, return [].

Changed Lines (from Template)

```
Queue<Integer> q = new LinkedList<>();
for (int i = 0; i < n; i++)
    if (indegree[i] == 0) q.offer(i);

List<Integer> order = new ArrayList<>();
while (!q.isEmpty()) {
    int node = q.poll();
    order.add(node);
    for (int nei : graph.get(node)) {
        if (--indegree[nei] == 0) q.offer(nei);
    }
}

return order.size() == n ? order : new ArrayList<>();
```

Caller	Return	Key Idea
findOrder(numCourses, prerequisites)	int[]	Topological order using BFS

AHA: If order size < n, cycle exists.

Complexity
Time: $O(V + E)$
Space: $O(V + E)$

Interview Tip
Kahn's BFS is iterative and safe.

6 EVENTUAL SAFE STATES

Base Template: Topological Sort (Reverse Graph)

Problem

Return all nodes that are NOT part of any cycle.

Changed Lines (from Template)

```
// Build reverse graph
for (int u = 0; u < n; u++)
    for (int v : graph.get(u))
        revGraph.get(v).add(u);
// Start BFS with terminal nodes (outdegree 0)
for (int i = 0; i < n; i++)
    if (outdegree[i] == 0) q.offer(i);
List<Integer> safe = new ArrayList<>();
while (!q.isEmpty()) {
    int node = q.poll();
    safe.add(node);
    for (int pre : revGraph.get(node))
        if (--outdegree[pre] == 0) q.offer(pre);
}
Collections.sort(safe);
return safe;
```

Caller	Return	Key Idea
eventualSafeNodes(graph)	List<Integer>	Reverse BFS from terminal nodes

AHA: Nodes not in any cycle become safe.

Complexity
Time: $O(V + E)$
Space: $O(V + E)$

Interview Tip
Reverse graph trick is key.

7 CHANGED LINES SUMMARY

#	Problem	Base Template	Main Changes
1	Swim in Rising Water	C	Use max() in relax; grid as graph
2	Path with Max Probability	B	Max-heap; multiply probabilities
3	Min Multiplications	A	Nodes = $\{0, \text{mod}-1\}$; steps + 1
4	Most Stones Removed	F (DSU)	Map rows/cols; components count
5	Course Schedule II	F (Queue)	Store order while BFS
6	Eventual Safe States	F (Topo Rev)	Reverse graph; start from terminals

8 COMPLEXITY QUICK GUIDE

Pattern	Time Complexity	Space Complexity
Dijkstra (Min/Max)	$O((V+E) \log V)$	$O(V + E)$
Dijkstra (Adj List)	$O((V+E) \log V)$	$O(V + E)$
DSU (Union Find)	$O(N \alpha(N))$	$O(N)$
Topological Sort (BFS)	$O(V + E)$	$O(V + E)$
BFS / DFS	$O(V + E)$	$O(V)$

V = vertices, E = edges, $\alpha(N)$ is Inverse Ackermann ($\sim$ constant).

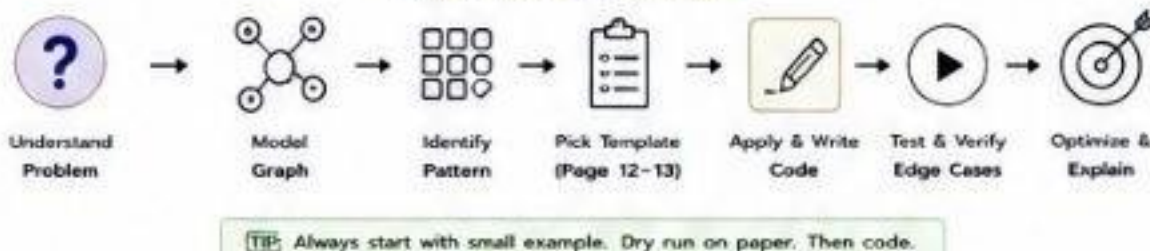
9 RETURN TYPE GUIDE

Pattern	Return Type	Use When
Shortest Distance	int / long	Min cost / steps
Path / Order	int[] / List<Integer>	Need sequence
Probability	double	Max probability path
Exist / Possible	boolean	Cycle / schedule
List of Nodes	List<Integer>	Safe nodes, result set

10 INTERVIEW CHECKLIST

- ✓ Did I model the graph correctly?
- ✓ Did I pick the right base template?
- ✓ Did I initialize data structures correctly?
- ✓ Did I handle edge cases (n=1, empty)?
- ✓ Did I avoid overflow (long / mod)?
- ✓ Is my complexity optimal?
- ✓ Can I explain my approach clearly?

11 INTERVIEW THINKING FLOW



12 PAGE 15 PREVIEW

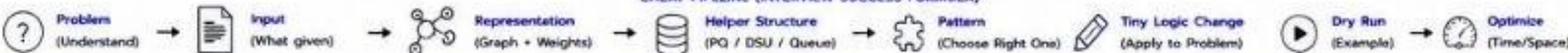
Next page contains SPECIAL GRAPH PROBLEMS GUIDE.

For each problem, you will see:



Master special problems. Solve any graph question in the interview!

CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)





HOW TO USE THIS PAGE

- Find the problem.
- Identify the base template (from Page 12).
- Read the CHANGED LINES.
- Understand the AHA.
- Review complexity & interview tip.
- Practice writing from memory.



TEMPLATE LEGEND (From Page 12)

A Network Delay Time (Single Source)**B** Dijkstra with Path (Parent[])**C** Minimum Effort Path (Minimize Max Edge)**D** Cheapest Flights (K Stops)**E** Path With Minimum Effort**F** Find Shortest Path (Undirected Weighted)

1 NETWORK DELAY TIME

A

Base Template: Dijkstra (Single Source Shortest Path)

Problem

Given a directed weighted graph, find time for all nodes to receive signal from node k. Return -1 if impossible.

Changed Lines (from Template)

```
// Build graph from times[u, v, w]
for (int[] t : times) {
    graph.get(t[0]).add(new int[]{t[1], t[2]});
}

// Answer = max distance among all nodes
int ans = 0;
for (int i = 1; i <= n; i++) {
    if (dist[i] == INF) return -1;
    ans = Math.max(ans, dist[i]);
}
return ans;
```

Caller

networkDelayTime(times, n, k)

Return

int (time or -1)

Key Idea

All nodes receive signal?

AHA: The last node to get the signal = max shortest distance.

Complexity

Time: $O((V + E) \log V)$ Space: $O(V + E)$

Interview Tip

Don't forget to check unreachable nodes.

4 CHEAPEST FLIGHTS WITHIN K STOPS

D

Base Template: Dijkstra with Stops Limit

Problem

Find cheapest price from src to dst with at most k stops.

Changed Lines (from Template)

```
PriorityQueue<int[]> pq = new PriorityQueue<
    (Comparator.comparingInt(a -> a[0]));
pq.offer(new int[]{0, src, 0}); // cost, node, stops
while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    int cost = cur[0], u = cur[1], stops = cur[2];
    if (u == dst) return cost;
    if (stops > k) continue;
    for (int[] nei : graph.get(u)) {
        int v = nei[0], w = nei[1];
        pq.offer(new int[]{cost + w, v, stops + 1});
    }
}
return -1;
```

Caller

findCheapestPrice(n, flights, src, dst, k)

Return

int (min cost or -1)

Key Idea

State includes stops

AHA: We need (cost, node, stops) state.

Complexity

Time: $O(K * E \log E)$ Space: $O(K * V)$

Interview Tip

Do NOT use plain dist[]: stops make it different.

7 CHANGED LINES SUMMARY

#	Problem	Template	Main Changes
1	Network Delay Time	A	Answer = max(dist[]), check unreachable
2	Dijkstra with Path	B	parent[] track + reconstruct path
3	Minimum Effort Path	C	Relax using max(edge, curEffort)
4	Cheapest Flights (K Stops)	D	State = (cost, node, stops)
5	Path with Minimum Effort	E	Minimize maximum edge on path
6	Undirected Weighted Graph	F	Add edges both directions

10 INTERVIEW CHECKLIST

- Did I initialize dist[src] = 0 correctly?
- Did I use PriorityQueue (min-heap) correctly?
- Did I skip if visited node is already processed?
- Did I relax correctly (dist[v] > dist[u] + w)?
- Did I handle unreachable nodes?
- Did I handle special constraint (stops / max edge)?
- Is my time complexity $O((V + E) \log V)$?

Interview Tip

Relaxation is the heart.

PQ always gives the next closest node.

Small changes solve different problems!

8 Dijkstra Template Recap (From Page 12)

Core Code (Adjacency List)

- Initialize dist[] = INF, src = 0
- PQ = (dist, node), push (0, src)
- While pq not empty:
 - Pop (d, u)
 - If visited[u] continue
 - For each neighbor (v, w):
 - If dist[v] > d + w → update & push
- Return dist[]

Works for all above problems with small changes.

11 INTERVIEW THINKING FLOW



9 RETURN TYPE GUIDE

Return Type	Use When	Example Problems
int[] dist	All pairs from single source	1, 6
int	Need final value (max / min effort)	1, 3, 5
List<Integer> path	Need actual shortest path	2
int (cost)	Cheapest with constraints (K stops)	4
Custom (dist + path)	Need both distance & path	2

12 PAGE 14 PREVIEW

Next page contains ADVANCED GRAPH PATTERNS GUIDE.

For each pattern, you will see:



Master patterns. Solve any graph problem in the interview!



HOW TO USE THIS PAGE

- 1 Use this page for quick recall.
- 2 Before coding, pick the right template.
- 3 Check complexity & edge cases.
- 4 Use interview tips to avoid mistakes.
- 5 Keep practicing. Patterns become reflex.
- 6 You've got the roadmap. Keep going!



TEMPLATE QUICK INDEX

- | | | |
|--|---|---|
| A Network Delay Time (Dijkstra) | E Path with Minimum Effort | I Multi-Source BFS (Grid / Matrix) |
| B Dijkstra with Path (Parent[]) | F Shortest Path in Undirected Graph | J Union Find (Disjoint Set Union) |
| C Minimum Effort Path (Minimize Max Edge) | G Clone Graph (DFS) | K Graph Valid Tree (Union Find / DFS) |
| D Cheapest Flights (K Stops) | H Topological Sort (Course Schedule) | L All Pairs Shortest Path (Floyd Warshall) |

1 ADJACENCY LIST (GRAPH BUILDING)

```
// n = number of nodes (0 to n-1)
List<List<Integer>> buildGraph(int n, int[][] edges, boolean
    undirected) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

    for (int[] e : edges) {
        int u = e[0], v = e[1];
        graph.get(u).add(v);
        if (undirected) graph.get(v).add(u);
    }
    return graph;
}
```

💡 Tip: Use List for neighbors. Avoid duplicates if needed.

2 BREADTH FIRST SEARCH (BFS)

```
List<Integer> bfs(int start, List<List<Integer>> g) {
    int n = g.size();
    boolean[] vis = new boolean[n];
    Queue<Integer> q = new LinkedList<>();
    List<Integer> order = new ArrayList<>();
    q.offer(start); vis[start] = true;
    while (!q.isEmpty()) {
        int node = q.poll();
        order.add(node);
        for (int nei : g.get(node)) {
            if (!vis[nei]) {
                vis[nei] = true;
                q.offer(nei);
            }
        }
    }
    return order;
}
```

💡 Tip: BFS = Level by level. Queue + Visited.

3 DEPTH FIRST SEARCH (DFS)

```
void dfs(int node, List<List<Integer>> g, boolean[] vis,
    List<Integer> order) {
    vis[node] = true;
    order.add(node);
    for (int nei : g.get(node)) {
        if (!vis[nei]) dfs(nei, g, vis, order);
    }
}

// Wrapper
List<Integer> dfsTraversal(int start, List<List<Integer>> g) {
    int n = g.size();
    boolean[] vis = new boolean[n];
    List<Integer> order = new ArrayList<>();
    dfs(start, g, vis, order);
    return order;
}
```

💡 Tip: DFS = Go deep, then backtrack. Recursion / Stack.

4 TIME COMPLEXITY GUIDE

Operation / Algorithm	Time Complexity
BFS / DFS (Adjacency List)	$O(V + E)$
BFS / DFS (Adjacency Matrix)	$O(V^2)$
Dijkstra (Binary Heap)	$O((V + E) \log V)$
Dijkstra (Min Heap)	$O((V + E) \log V)$
Topological Sort (Kahn's BFS)	$O(V + E)$
Topological Sort (DFS)	$O(V + E)$
Union Find (amortized)	$O(\alpha(V))$
Floyd Warshall (All Pairs)	$O(V^3)$

V = vertices, E = edges, α = inverse Ackermann (= constant)

5 SPACE COMPLEXITY GUIDE

Algorithm / Structure	Space Complexity
Adjacency List	$O(V + E)$
Adjacency Matrix	$O(V^2)$
BFS / DFS	$O(V)$ (visited + queue/stack)
Dijkstra	$O(V)$ (dist[]) + $O(E)$ (adj list)
Union Find	$O(V)$
Floyd Warshall	$O(V^2)$

Space depends on representation & data structures.

6 GRAPH REPRESENTATION COMPARISON

Type	Space	Add Edge	Check Edge (u,v)	Iterate Neighbors
Adjacency List	$O(V + E)$	$O(1)$	$O(\deg(u))$	$O(\deg(u))$
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(1)$	$O(V)$

💡 Use List for sparse graphs ($E \ll V^2$). Use Matrix for dense graphs.

7 WHEN TO USE WHAT?

Problem Type	Use
Shortest Path (Weighted)	Dijkstra (A, B, C, E)
Shortest Path (Unweighted)	BFS (F)
All Pairs Shortest Path	Floyd Warshall (L)
Cycle / Connected / Tree	Union Find / DFS (J, K)
Order / Scheduling	Topological Sort (H)
Grid / Multi-Source	BFS (I)

💡 Understand the goal → pick the right tool.

8 DIJKSTRA TEMPLATE (MIN HEAP)

```
int[] dijkstra(int n, List<List<Integer>> g, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int> pq = new PriorityQueue<>((a,b) -> a[0]-b[0]);
    pq.offer(new int[]{0, src}); // (dist, node)
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int d = cur[0], u = cur[1];
        if (d > dist[u]) continue;
        for (int[] e : g.get(u)) {
            int v = e[0], w = e[1];
            if (d + w < dist[v]) {
                dist[v] = d + w;
                pq.offer(new int[]{dist[v], v});
            }
        }
    }
    return dist;
}
```

9 TOPOSORT (KAHN'S BFS)

```
int[] topoSort(int n, List<List<Integer>> g) {
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : g.get(u)) indeg[v]++;
    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);
    int[] order = new int[n]; int idx = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); order[idx++] = u;
        for (int v : g.get(u))
            if (--indeg[v] == 0) q.offer(v);
    }
    if (idx != n) return new int[0]; // cycle
    return order;
}
```

10 UNION FIND (DSU)

```
class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[py] < rank[px]) parent[py] = px;
        else { parent[py] = px; rank[px]++; }
        return true;
    }
}
```

11 ALL PAIRS SHORTEST PATH

```
// Floyd Warshall
void floydWarshall(int[][] dist) {
    int n = dist.length;
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] != INF && dist[k][j] != INF
                    && dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}
```

💡 INF = a very large value (e.g. 1e9)

12 COMMON PITFALLS

#	Mistake	Why It Fails	Fix
1	Not marking visited	Infinite loop / TLE	Mark visited when pushing (BFS)
2	Wrong graph direction	Wrong answer	Check if graph is directed/undirected
3	Forgot edge weights	Wrong path	Use weighted template (Dijkstra)
4	PriorityQueue order wrong	TLE / Wrong path	Min-heap by distance
5	Integer overflow	Wrong result	Use long for distance
6	Cycle in topo sort	Invalid order	Check if result size < V

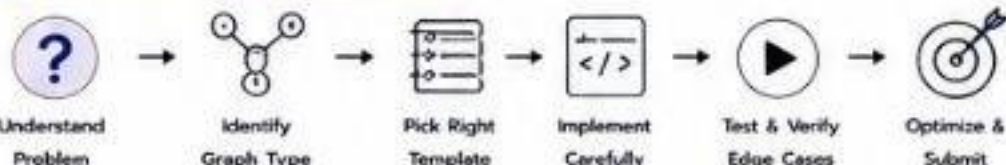
13 EDGE CASE CHECKLIST

☒ Empty graph / single node	☒ Large weights / zero weights
☒ Disconnected components	☒ Graph with no cycles
☒ Self loop	☒ Graph with cycles
☒ Multiple edges between same nodes	☒ Maximum constraints (V, E)

14 QUICK RECALL TABLE

Goal	Go-To Template
Shortest time / distance (positive weights)	Dijkstra (A, B, C, E)
Fewest steps (unweighted)	BFS (F)
Order of tasks / courses	Topological Sort (H)
Detect cycle / components	Union Find / DFS (J, K)
Grid / multi-source	BFS (I)
All pairs shortest path	Floyd Warshall (L)

15 GRAPH PATTERN FLOW (RECAP)



Tip: Think → Choose → Code → Test → Optimize → Succeed!

16 INTERVIEW TIP BOX

- ★ Talk aloud. Explain your approach.
- ★ State assumptions & edge cases.
- ★ Write clean, modular code.
- ★ Start with brute force if unsure.
- ★ Optimize step by step.

Confidence + Clarity = Crack FAANG!

17 NEXT PAGE PREVIEW

Next page (Page 17) contains
GRAPH PATTERN ROADMAP.
Follow the path.
Master the patterns.
Crack the interviews!



CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)



"Consistency builds pattern recognition. Pattern recognition wins interviews."

HOW TO USE THIS PAGE

1

Follow the roadmap in order.

2

Solve problems by pattern, not randomly.

3

Track your progress and weak areas.

4

Revise regularly using spaced repetition.

5

Revisit cheat sheets before interviews.

You've got this. Keep showing up!

GRAPH PATTERN ROADMAP (RECOMMENDED FLOW)

1

Foundations

- What is Graph?
- Representations
- Basic Traversals

2

Core Traversals

- DFS & BFS
- Connected
- Components

3

Grid / Matrix

- Islands, Area
- Multi-Source BFS
- Water Flow

4

Topological

- DAG, Order
- Course Schedule
- Cycle Detection

5

Advanced

- Shortest Path
- Multiple Sources
- Optimization

6

Mastery

- Hard Problems
- Simulation
- Edge Cases

"Tip: Master 1 step at a time. Revise previous steps while learning new ones."

GRAPH PATTERN SUMMARY (At a Glance)					
Pattern	What It Solves	Key Idea	Main Tools	Examples (Problems)	Time Complexity (Typical)
1 Network Delay Time	Time from source to all nodes	Shortest path from single source	Dijkstra (PQ)	Network Delay Time	$O((V + E) \log V)$
2 Dijkstra with Path	Shortest distance + actual path	Track parent while relaxing	Dijkstra + Parent[]	Path with Min Cost	$O((V + E) \log V)$
3 Minimum Effort Path	Minimize maximum edge	Minimize the max along path	Dijkstra (custom relax)	Path With Minimum Effort	$O((V + E) \log V)$
4 Cheapest Flights (K Stops)	Shortest path with stop limit	Use stops dimension	Dijkstra / BFS + stops	Cheapest Flights Within K Stops	$O(K \cdot E \log V)$
5 Topological Sort	Order tasks / DAG processing	Indegree 0 $\rightarrow$ queue	BFS (Kahn's)	Course Schedule II	$O(V + E)$
6 Cycle Detection	Detect cycle in directed graph	Back edge in DFS	DFS + Rec Stack	Course Schedule (I)	$O(V + E)$
7 Undirected Shortest Path	Shortest path unweighted	Level by level expansion	BFS	Shortest Path in UD Graph	$O(V + E)$
8 Multi-Source BFS	From many sources at once	Put all sources in queue	BFS (multi-source)	Rotting Oranges, 01 Matrix	$O(V + E)$
9 Disjoint Set (Union Find)	Check / Merge components	Union by rank + path compression	DSU	Valid Tree, Accounts Merge	$O(\alpha(V))$
10 Floyd Warshall	All pairs shortest path	DP on intermediate nodes	DP (3 loops)	All Pairs Shortest Path	$O(V^3)$
11 Strongly Connected Components	Group of nodes mutually reachable	Kosaraju / Tarjan	DFS	SCC Problems	$O(V + E)$
12 Special Patterns	Cloning, Regions, Water Flow	Use pattern + appropriate traversal	DFS / BFS	Clone Graph, Surrounded Regions, Pacific Atlantic	Varies

SPACED PRACTICE PLAN (4 WEEK PLAN)

Week	Focus	New Problems (2-3 / day)	Revision (Spaced)	Goal
Week 1	Foundations + Core Traversals	DFS, BFS, Connected Components, Islands, Clone Graph	Day 4: Review Day 1 Day 7: Review Day 1-3	Strong basics + confidence
Week 2	Topological + BFS Applications	Course Schedule, Topo Sort, Multipliers, Rotting Oranges	Day 11: Review Day 1-7 Day 14: Review Day 8-12	Solve medium comfortably
Week 3	Shortest Paths	Dijkstra variations, Min Effort, Cheapest Flights K Stops	Day 18: Review Day 1-14 Day 21: Review Day 15-18	Master shortest path patterns
Week 4	Advanced + Mixed	Floyd Warshall, Union Find, Hard graph problems	Day 25: Review Day 1-21 Day 28: Review Day 22-25	Tackle hard problems

💡

Tip: Active recall + spaced repetition = long term retention.

REVISION SCHEDULE (SPACED REPETITION)

●

Day 1-3Learn new concepts & solve problems

●

Day 4Review Day 1 (without looking)

●

Day 7Review Day 1-3

●

Day 11Review Day 1-7

●

Day 14Review Day 8-12

●

Day 18Review Day 1-14

●

Day 21Review Day 15-18

●

Day 25Review Day 1-21

●

Day 28Review Day 22-25

●

Day 30Full mock + weak area revision

ERROR LOG (Track & Improve)

Maintain an error log.

Write down:

•

Problem name

•

What went wrong?

•

Concept gap

•

Correct approach

•

Date of review

"The goal is not to never make mistakes. The goal is to never repeat the same mistake."

INTERVIEW STRATEGY ROADMAP

Understand the Problem

- Clarify
- Constraints
- Examples

Identify Graph Type

- Directed / Undirected
- Weighted / Unweighted
- Grid / Tree / DAG

Pick the Right Pattern

- Match pattern
- Choose algorithm
- Plan steps

Implement Cleanly

- Write modular code
- Handle edge cases
- Test small cases

Verify & Optimize

- Check results
- Optimize if needed
- Explain clearly

KEY MINDSET REMINDERS

✓

Always draw the graph.

✓

Think in patterns, not solutions.

✓

Start simple (brute force), then optimize.

✓

Dry run on small examples.

✓

Handle edge cases & constraints.

✓

Explain your thought process.

✓

Code clean, modular, readable.

Think $\rightarrow$ Plan $\rightarrow$ Code $\rightarrow$ Test $\rightarrow$ Optimize

COMPLEXITY REMINDERS

BFS / DFS: $O(V + E)$

Dijkstra (PQ): $O((V + E) \log V)$

Topo Sort: $O(V + E)$

Union Find: $O(\alpha(V))$

Floyd Warshall: $O(V^3)$

Space (Adj List): $O(V + E)$

💡

Tip: Use Adj List for sparse graphs.

COMMON PITFALLS

✗

Not marking visited nodes.

✗

Wrong data structure (Stack vs Queue vs PQ).

✗

Not handling disconnected components.

✗

Integer overflow in distances.

✗

Ignoring constraints (K stops, weights, etc.).

✗

Forgetting to return / edge cases.

⚠

Tip: Slow down, think & check!

QUICK CHECK BEFORE SUBMIT

✓

Handled all edge cases?

✓

Correct data structure used?

✓

Visited / parent tracked?

✓

Return value correct?

✓

Complexity acceptable?

✓

Dry run matches output?

⌚

Tip: 10 sec checklist saves 10 min regret!

INTERVIEW TIPS

★

Communicate throughout.

★

Break problem into small steps.

★

Use examples to explain.

★

If stuck, think out loud.

★

Optimize only after correct solution.

★

Stay calm, think patterns.

💬

Remember: Interview is a conversation!

NEXT PAGE PREVIEW

Next page (Page 18) contains

GRAPH PATTERN PRACTICE TRACKER & PROBLEM INDEX.

Track, practice, analyze and crack FAANG!

THE SUCCESS FORMULA

Problem (Understand)

Input (What given)

Representation (Graph Model)

Pattern (Identify)

Algorithm (Pick Right One)

Implementation (Code Clean)

Test (Dry Run)

Analyze (Complexity)

Optimize (If Needed)

Success (Interview)

Plan well. Practice smart. Stay consistent. You will get there! 🍀



HOW TO USE THIS PAGE

- 1 Identify the pattern.
- 2 Pick the right algorithm.
- 3 Use the template.
- 4 Handle edge cases.
- 5 Optimize & code clean.



ALGORITHM INDEX

- | | | |
|------------------------|-------------------------------|---------------------------|
| 1 DFS (Traversal) | 5 Clone Graph | 9 Rotting Oranges |
| 2 BFS (Traversal) | 6 Course Schedule (I) | 10 Topological Sort (BFS) |
| 3 Connected Components | 7 Course Schedule (II) | 11 Dijkstra (Min Heap) |
| 4 Number of Islands | 8 Pacific Atlantic Water Flow | 12 Union Find (DSU) |

1 DFS (Traversal)

Recursive Template

```
void dfs(int node, List<List<Integer>> g, boolean[] vis){
    vis[node] = true;
    // process node
    for(int nei : g.get(node)){
        if(!vis[nei]) dfs(nei, g, vis);
    }
}
```

Time: $O(V + E)$ Space: $O(V)$

Use for: component, path, cycle, topo (DFS)

2 BFS (Traversal)

Iterative Template

```
void bfs(int src, List<List<Integer>> g){
    boolean[] vis = new boolean[g.size()];
    Queue<Integer> q = new LinkedList<>();
    q.offer(src); vis[src] = true;
    while(!q.isEmpty()){
        int node = q.poll();
        // process node
        for(int nei : g.get(node)){
            if(!vis[nei]){
                vis[nei] = true;
                q.offer(nei);
            }
        }
    }
}
```

Time: $O(V + E)$ Space: $O(V)$

Use for: shortest path (unweighted), level order

3 Connected Components

DFS Template

```
int count = 0;
void dfs(int node, List<List<Integer>> g, boolean[] vis){
    vis[node] = true;
    for(int nei : g.get(node))
        if(!vis[nei]) dfs(nei, g, vis);
}

int countComponents(int n, int[][] edges){
    List<List<Integer>> g = buildGraph(n, edges);
    boolean[] vis = new boolean[n];
    for(int i=0; i<n; i++){
        if(!vis[i]) count++;
        dfs(i, g, vis);
    }
    return count;
}
```

Time: $O(V + E)$ Space: $O(V)$

Use for: number of connected components

4 Number of Islands

BFS/DFS on Grid

```
int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
int numIslands(char[][] grid){
    int m=grid.length, n=grid[0].length, cnt=0;
    boolean[][] vis = new boolean[m][n];
    for(int i=0; i<m; i++){
        for(int j=0; j<n; j++){
            if(grid[i][j]=='1' && !vis[i][j]){
                cnt++;
                bfs(i,j,grid,vis); // or dfs
            }
        }
    }
    return cnt;
}
```

Time: $O(M * N)$ Space: $O(M * N)$

Use for: island counting problems

5 Clone Graph

BFS Template

```
Node cloneGraph(Node node){
    if(node == null) return null;
    Map<Node,Node> map = new HashMap<>();
    Queue<Node> q = new LinkedList<>();
    q.offer(node);
    map.put(node, new Node(node.val));
    while(!q.isEmpty()){
        Node cur = q.poll();
        for(Node nei : cur.neighbors){
            if(!map.containsKey(nei)){
                map.put(nei, new Node(nei.val));
                q.offer(nei);
            }
            map.get(cur).neighbors.add(map.get(nei));
        }
    }
    return map.get(node);
}
```

Time: $O(V + E)$ Space: $O(V)$

Use for: deep copy of connected graph

6 Course Schedule (I)

Cycle Detection (BFS - Kahn's)

```
boolean canFinish(int n, int[][] pre){
    List<List<Integer>> g = buildGraph(n, pre);
    int[] indeg = new int[n];
    for(int i=0; i<n; i++){
        for(int nei : g.get(i)) indeg[nei]++;
    }
    Queue<Integer> q = new LinkedList<>();
    for(int i=0; i<n; i++) if(indeg[i]==0) q.offer(i);
    int cnt = 0;
    while(!q.isEmpty()){
        int node = q.poll(); cnt++;
        for(int nei : g.get(node))
            if(--indeg[nei]==0) q.offer(nei);
    }
    return cnt == n;
}
```

Time: $O(V + E)$ Space: $O(V)$

Use for: detect cycle in DAG (can finish?)

7 Course Schedule (II)

Topological Sort (BFS)

```
int[] findOrder(int n, int[][] pre){
    List<List<Integer>> g = buildGraph(n, pre);
    int[] indeg = new int[n];
    for(int i=0; i<n; i++){
        for(int nei : g.get(i)) indeg[nei]++;
    }
    Queue<Integer> q = new LinkedList<>();
    for(int i=0; i<n; i++) if(indeg[i]==0) q.offer(i);
    int[] order = new int[n]; int idx = 0;
    while(!q.isEmpty()){
        int node = q.poll(); order[idx++] = node;
        for(int nei : g.get(node))
            if(--indeg[nei]==0) q.offer(nei);
    }
    return idx==n ? order : new int[0];
}
```

Time: $O(V + E)$ Space: $O(V)$

Use for: return valid order / topological sort

8 Pacific Atlantic Water Flow

Multi-Source BFS (Reverse)

```
int pacificAtlantic(int[][] h){
    int m=h.length, n=h[0].length;
    boolean[][] pac = new boolean[m][n];
    boolean[][] atl = new boolean[m][n];
    Queue<int> qp = new LinkedList<>();
    Queue<int> qa = new LinkedList<>();
    // push borders to correspond queues
    // bfs to mark reachable cells
    // intersect both
    List<List<Integer>> res = new ArrayList<>();
    for(int i=0; i<m; i++) for(int j=0; j<n; j++){
        if(pac[i][j] && atl[i][j])
            res.add(Arrays.asList(i,j));
    }
    return res;
}
```

Time: $O(M * N)$ Space: $O(M * N)$

Use for: water flow / reachability from borders

9 Rotting Oranges

Multi-Source BFS

```
int orangesRotting(int[][] grid){
    Queue<int> q = new LinkedList<>();
    int fresh = 0, time = 0;
    // init queue with rotten oranges
    // count fresh
    while(!q.isEmpty() && fresh > 0){
        int size = q.size(); time++;
        while(size-- > 0){
            int[] cur = q.poll();
            for(int[] d : dLns){
                int x = cur[0]+d[0], y = cur[1]+d[1];
                if(valid(x,y) && grid[x][y]==1){
                    grid[x][y]=2; fresh--;
                    q.offer(new int[]{x,y});
                }
            }
        }
    }
    return fresh==0 ? time : -1;
}
```

Time: $O(M * N)$ Space: $O(M * N)$

Use for: multi-source BFS spread problems

10 Topological Sort (BFS)

Kahn's Algorithm

```
List<Integer> topoSort(int n, int[][] edges){
    List<List<Integer>> g = buildGraph(n, edges);
    int[] indeg = new int[n];
    for(int i=0; i<n; i++){
        for(int nei : g.get(i)) indeg[nei]++;
    }
    Queue<Integer> q = new LinkedList<>();
    for(int i=0; i<n; i++) if(indeg[i]==0) q.offer(i);
    List<Integer> order = new ArrayList<>();
    while(!q.isEmpty()){
        int node = q.poll(); order.add(node);
        for(int nei : g.get(node))
            if(--indeg[nei]==0) q.offer(nei);
    }
    return order.size()==n ? order : new ArrayList<>();
}
```

Time: $O(V + E)$ Space: $O(V)$

Use for: DAG ordering / dependencies

11 Dijkstra (Min Heap)

Shortest Path (Non-negative)

```
int[] dijkstra(int n, int[][] edges, int src){
    List<List<Integer>> g = new ArrayList<>();
    for(int i=0; i<n; i++) g.add(new ArrayList<>());
    for(int[] e : edges){
        g.get(e[0]).add(new int[]{e[1], e[2]});
        // If undirected: g.get(e[1]).add(new int[]{e[0], e[2]});
    }
    int[] dist = new int[n]; Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int> pq = new PriorityQueue<>((a,b)->a[0]-b[0]); // (dist, node)
    pq.offer(new int[]{0, src});
    while(!pq.isEmpty()){
        int[] cur = pq.poll(); int d = cur[0], u = cur[1];
        if(d != dist[u]) continue;
        for(int v : g.get(u)){
            int vDist = dist[v];
            if(d + v[2] < vDist){
                dist[v] = d + v[2];
                pq.offer(new int[]{dist[v], v[1]});
            }
        }
    }
    return dist;
}
```

Time: $O((V + E) \log V)$ Space: $O(V)$

Use for: single source shortest path

12 Union Find (DSU)

Disjoint Set Union

```
class DSU{
    int[] parent, rank;
    DSU(int n){
        parent = new int[n]; rank = new int[n];
        for(int i=0; i<n; i++) parent[i]=i; rank[i]=0;
    }
    int find(int x){
        if(parent[x]==x) return x;
        return parent[x] = find(parent[x]);
    }
    boolean union(int x, int y){
        int px = find(x), py = find(y);
        if(px==py) return false;
        if(rank[px] < rank[py]) parent[px] = py;
        else if(rank[py] < rank[px]) parent[py] = px;
        else parent[py] = px; rank[px]++;
        return true;
    }
}
```

Time: $O(\alpha(V))$ per op Space: $O(V)$

Use for: components, cycle in undirected

GRAPH REPRESENTATIONS

Adjacency List

- Best for sparse graphs
- Space: $O(V + E)$
- Iterate neighbors fast



Adjacency Matrix

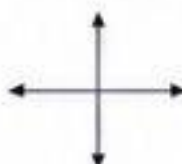
- Best for dense graphs
- Space: $O(V^2)$
- Check edge exists fast

	0	1	2	3
0	0	1	1	0
1	1	0	0	1
2	1	0	0	1
3	0	1	1	0

COMMON DIRECTIONS (Grid Problems)

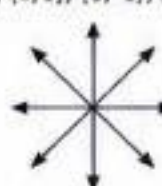
4 Directions

```
int[][] dirs4 = {{1,0},{-1,0},{0,1},{0,-1}};
```



8 Directions

```
int[][] dirs8 = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
```



EDGE TYPES

- Directed / Undirected
- Weighted / Unweighted
- Cyclic / Acyclic (DAG)
- Connected / Disconnected

COMPLEXITY QUICK REFERENCE

| V = Vertices | E = Edges | M = Rows | N = Cols |

- DFS / BFS : $O(V + E)$
- Topological Sort : $O(V + E)$
- Dijkstra (Min Heap) : $O((V + E) \log V)$

- Number of Islands : $O(M * N)$
- Rotting Oranges : $O(M * N)$
- Pacific Atlantic : $O(M * N)$

- Connected Components : $O(V + E)$
- Clone Graph : $O(V + E)$
- Union Find (per op) : $O(\alpha(V))$



$\alpha(V)$ = Inverse Ackermann ($\approx$ constant)
For all practical input sizes, treat as $O(1)$.



PATTERN



RECOGNITION



APPLY TEMPLATE



TEST BY TEMPLATE



HANDLE EDGE CASES



CODE CLEAN



CRACK FAANG



Core Patterns. Process + Code + Use + Pitfalls. Revise before your interview!

HOW TO USE THIS PAGE

- 1 Identify the pattern from the problem.
- 2 Pick the right algorithm.
- 3 Follow the process (steps).
- 4 Use the template code.
- 5 Watch out for common mistakes.
- 6 Optimize & handle edge cases.


 Think → Choose → Code →
Test → Optimize

GRAPH PROBLEM SOLVING ROADMAP (HOW TO THINK)



★ Tip: Master these patterns and 90% of graph problems are solved!

1 DFS (Depth First Search)

Explore as deep as possible, then backtrack.

PROCESS (STEPS)

- 1 Mark current node as visited.
- 2 Process the node (add to result etc.).
- 3 Recur for all unvisited neighbors.
- 4 Backtrack when no unvisited neighbors.

CODE TEMPLATE (Java)

```

void dfs(int node, List<List<Integer>> g,
boolean[] vis) {
    vis[node] = true;
    // process node
    for (int nei : g.get(node)) {
        if (!vis[nei]) dfs(nei, g, vis);
    }
}

```



AHA

 Go deep first,
then backtrack.


Use When

 Explore all nodes,
components, cycle
detection, topo sort,
backtracking.


Watch Out

 Mark visited
before recursion
to avoid cycles.
Time: $O(V + E)$ Space: $O(V)$ (Recursion Stack)

2 BFS (Breadth First Search)

Visit level by level using a queue.

PROCESS (STEPS)

- 1 Push start node in queue.
- 2 Mark visited.
- 3 While queue not empty:
 - Pop front
 - Process
 - Push unvisited neighbors

CODE TEMPLATE (Java)

```

void bfs(int src, List<List<Integer>> g) {
    boolean[] vis = new boolean[g.size()];
    Queue<Integer> q = new LinkedList<>();
    q.offer(src); vis[src] = true;
    while (!q.isEmpty()) {
        int node = q.poll();
        // process node
        for (int nei : g.get(node)) {
            if (!vis[nei]) {
                vis[nei] = true;
                q.offer(nei);
            }
        }
    }
}

```



AHA

 First visit =
shortest path
(unweighted).


Use When

 Minimum steps,
nearest node,
level order.


Watch Out

 Mark visited
when enqueueing,
not dequeuing.
Time: $O(V + E)$ Space: $O(V)$ (Queue)

3 Topological Sort (BFS - Kahn's)

Linear ordering of DAG based on dependencies.

PROCESS (STEPS)

- 1 Compute indegree of all nodes.
- 2 Push all nodes with indegree 0.
- 3 Pop node, add to result.
- 4 Reduce indegree of neighbors.
- 5 If indegree becomes 0, push to queue.
- 6 If result size < V → Cycle exists.

CODE TEMPLATE (Java)

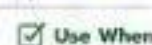
```

List<Integer> topoSort(int n,
List<List<Integer>> g) {
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : g.get(u)) indeg[v]++;
    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < n; i++)
        if (indeg[i] == 0) q.offer(i);
    List<Integer> res = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll(); res.add(u);
        for (int v : g.get(u)) {
            if (--indeg[v] == 0) q.offer(v);
        }
    }
    if (res.size() != n) return new ArrayList<>();
    return res;
}

```



AHA

 Process node only
after all prerequisites
are satisfied.


Use When

 DAG, dependencies,
course schedule,
build order.


Watch Out

 If processed nodes
< n → cycle
exists.
Time: $O(V + E)$ Space: $O(V)$

4 Dijkstra (Min Heap)

Shortest path from single source (positive weights).

PROCESS (STEPS)

- 1 Init dist[] = INF, dist[src] = 0.
- 2 Push (0, src) to min-heap.
- 3 Pop min (d, node). If d > dist[node], skip.
- 4 Relax all edges of node.
- 5 Update dist & push to heap.

CODE TEMPLATE (Java)

```

int[] dijkstra(int n, List<List<int>> g, int src) {
    int INF = (int)1e9;
    int[] dist = new int[n];
    Arrays.fill(dist, INF); dist[src] = 0;
    PriorityQueue<int> pq = new PriorityQueue<
    (Comparator.comparingInt(a -> a[0]));
    pq.offer(new int[] {0, src}); // {dist, node}
    while (!pq.isEmpty()) {
        int d = cur[0], u = cur[1];
        if (d > dist[u]) continue;
        for (int[] e : g.get(u)) {
            int v = e[0], w = e[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[] {dist[v], v});
            }
        }
    }
    return dist;
}

```



AHA

 Relax edges. First
time you pop a node,
its shortest distance
is finalized.


Use When

 Positive weighted
shortest path,
network delay,
min cost.


Watch Out

 Ignore outdated
entries: if d >
dist[node] skip.
Time: $O(V + E) \log V$ Space: $O(V)$

5 Union Find (Disjoint Set Union)

Maintain groups and merge components.

PROCESS (STEPS)

- 1 Make each node its own parent.
- 2 Find: return root with path compression.
- 3 Union: merge by rank/size.

CODE TEMPLATE (Java)

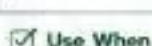
```

class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i; rank[i] = 0;
        }
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[py] < rank[px]) parent[py] = px;
        else { parent[py] = px; rank[px]++; }
        return true;
    }
}

```



AHA

 Build groups instead
of traversing them.


Use When

 Connectivity, merge
components, cycle
in undirected graph.


Watch Out

 Always use path
compression +
union by rank.
Time: $O(\alpha(V))$ per opSpace: $O(V)$

6 Grid DFS / BFS

For grid based problems (islands, regions, flow).

PROCESS (STEPS)

- 1 Check bounds & valid cell.
- 2 Mark visited.
- 3 Traverse 4 or 8 directions.
- 4 Count / collect / compare.

CODE TEMPLATE (Java)

```

// 4 Directions
int[][] dir4 = {{1,0},{-1,0},{0,1},{0,-1}};

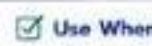
// 8 Directions
int[][] dir8 = {{1,0},{-1,0},{0,1},{0,-1},
               {1,1},{-1,-1},{-1,1},{1,-1}};

boolean isValid(int r, int c, int n, int m,
boolean[][] vis) {
    return r >= 0 && r < n && c >= 0 && c < m && !vis[r][c];
}

```



AHA

 Grid problems =
graph problems
with constraints.


Use When

 Islands, surrounded
regions, flood fill,
matrix distance.


Watch Out

 Boundary check
& mark visited
immediately.
Time: $O(M * N)$ Space: $O(M * N)$

GRAPH REPRESENTATIONS

Adjacency List (Preferred)

- Best for sparse graphs
- Space: $O(V + E)$
- Iterate neighbors fast



Adjacency Matrix

- Best for dense graphs
- Space: $O(V^2)$
- Check edge exists fast

	0	1	2	3
0	0	1	1	1
1	1	0	0	1
2	1	0	0	1
3	0	1	1	0

DIRECTIONS (Grid Problems)

4 Directions
(Up, Down, Left, Right)

```

int[][] dir4 = {{1,0},{-1,0},{0,1},{0,-1}};

```

8 Directions
(All)

```

int[][] dir8 = {{1,0},{-1,0},{0,1},{0,-1},
               {1,1},{-1,-1},{-1,1},{1,-1}};

```

EDGE TYPES

- Directed / Undirected
- Weighted / Unweighted
- Cyclic / Acyclic (DAG)
- Connected / Disconnected
- Sparse / Dense

COMPLEXITY QUICK REFERENCE

DFS / BFS

 Time: $O(V + E)$
Space: $O(V)$

Connected Components

 Time: $O(V + E)$
Space: $O(V)$

Topological Sort (BFS)

 Time: $O(V + E)$
Space: $O(V)$

Dijkstra (Min Heap)

 Time: $O((V + E) \log V)$
Space: $O(V)$

Union Find (DSU)

 Time: $O(\alpha(V))$ per op
Space: $O(V)$

Grid DFS / BFS

 Time: $O(M * N)$
Space: $O(M * N)$

 WHERE $\alpha(V)$ =
Inverse Ackermann
(almost constant)

FINAL INTERVIEW CHECKLIST

- ☒ Understand the problem clearly.
- ☒ Pick the correct graph pattern.
- ☒ Choose representation (List / Matrix).
- ☒ Dry run on small example.
- ☒ Handle edge cases.
- ☒ Write clean code using template.
- ☒ Check complexity.
- ☒ Optimize if needed.
- ☒ Test & confirm output.



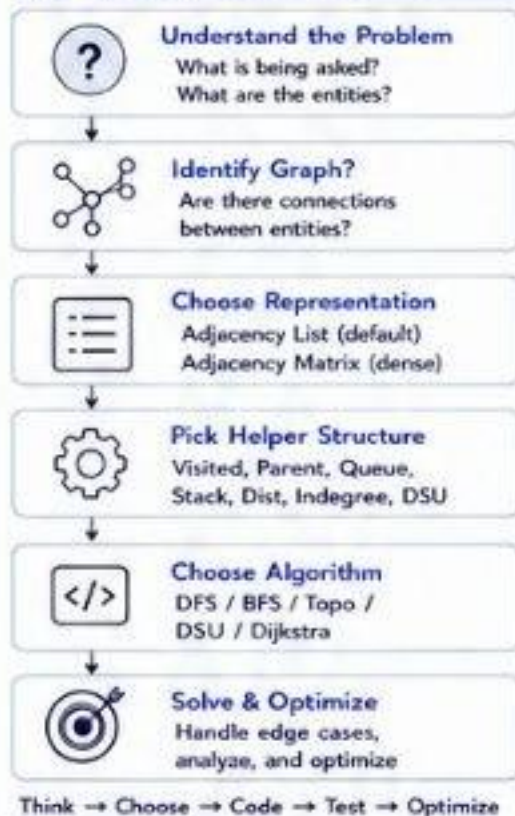
GOLDEN RULE

 Master patterns,
write clean code,
crack FAANG!

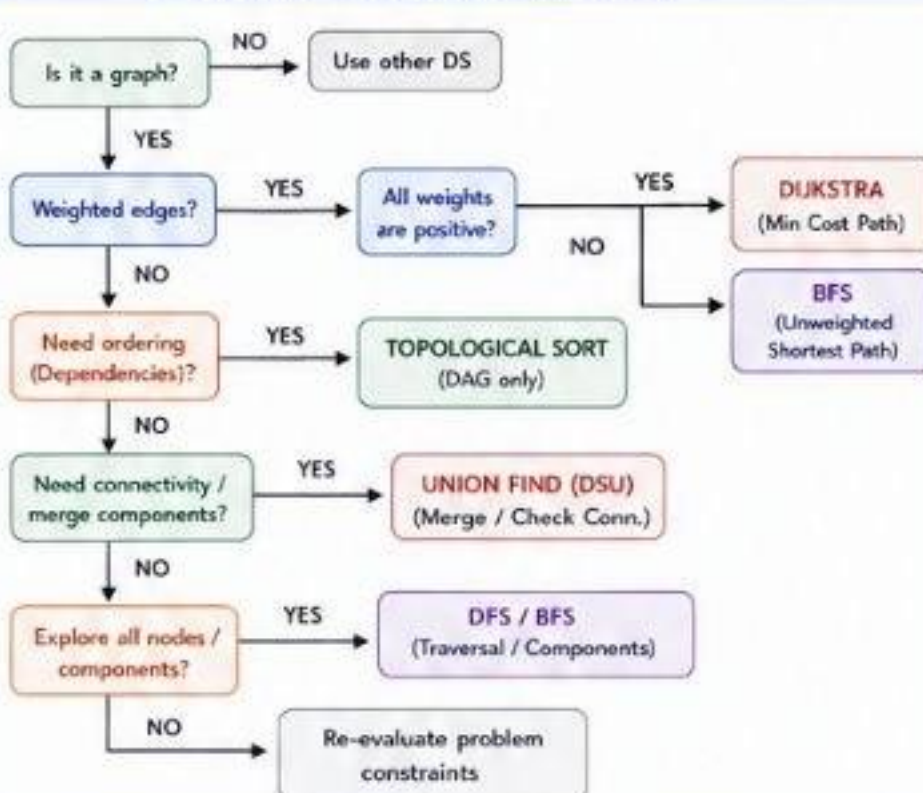


Recognize the pattern. Pick the right tool. Solve with confidence. Crack FAANG!

1 PATTERN RECOGNITION FLOW



2 PROBLEM → SOLUTION DECISION TREE



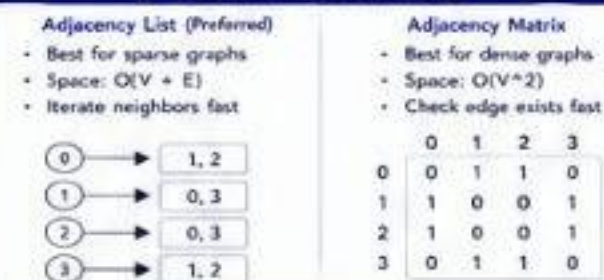
3 HIDDEN GRAPH RECOGNITION (THINK LIKE THIS!)



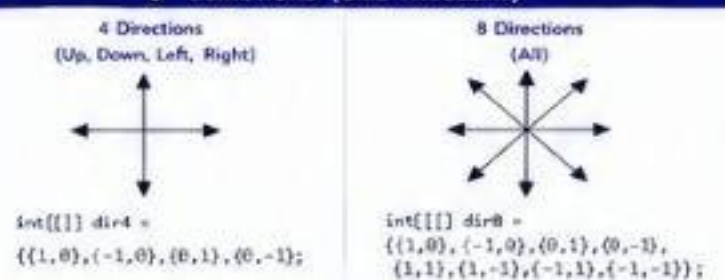
4 CORE GRAPH ALGORITHMS - WHEN, WHY & HOW

Algorithm	When to Use	Key Idea (AHA)	Steps (Quick Process)	Core Template (Java)	Time	Space
DFS (Depth First Search)	Explore all nodes, components, cycles, topo sort, paths	Go deep first, then backtrack.	- Mark node visited - Recur for all unvisited neighbors - Backtrack	<pre>void dfs(int node, List<Integer> g, boolean[] vis) { vis[node] = true; for (int nei : g.get(node)) if (!vis[nei]) dfs(nei, g, vis); }</pre>	$O(V + E)$	$O(V)$ (Recursion Stack)
BFS (Breadth First Search)	Shortest path (unweighted), level order, nearest	First visit = shortest path.	- Push start in queue, mark visited - While queue not empty: - Pop front - Push unvisited neighbors	<pre>Queue<Integer> q = new LinkedList<>(); q.offer(start); vis[start] = true; while (!q.isEmpty()) { int node = q.poll(); for (int nei : g.get(node)) if (!vis[nei]) { vis[nei] = true; q.offer(nei); } }</pre>	$O(V + E)$	$O(V)$ (Queue)
Topological Sort (Kahn's BFS)	DAG ordering, course schedule, dependencies	Process node after all prerequisites are satisfied.	- Compute indegree of all nodes - Push all nodes with indegree 0 - Pop, add to order, reduce indegree - If processed $< V$ → cycle exists	<pre>Queue<Integer> q = new LinkedList<>(); int[] indeg = new int[n]; // compute indegree for (int i=0; i<n; i++) if (indeg[i]==0) q.offer(i); List<Integer> order = new ArrayList<>(); while (!q.isEmpty()) { int node=q.poll(); order.add(node); for (int nei : g.get(node)) if (--indeg[nei]==0) q.offer(nei); }</pre>	$O(V + E)$	$O(V)$
Dijkstra (Min Heap)	Shortest path in weighted graph (positive weights)	Relax edges. First time pop from heap = shortest distance.	- Init dist[] = INF, dist[src]=0 - Push (0, src) in min-heap - Pop min, skip if outdated - Relax neighbors - Repeat	<pre>PriorityQueue<int[]> pq = new ... (a,b) -> a[0]-b[0]; int[] dist = new int[n]; Arrays.fill(dist, INF); dist[src]=0; pq.offer(new int[]{0, src}); while (!pq.isEmpty()) { int[] cur=pq.poll(); int d=cur[0], node=cur[1]; if (d>dist[node]) continue; for (int nei : g.get(node)) { int w=edgeW(node,nei); if (dist[nei]>d+w) { dist[nei]=d+w; pq.offer(new int[]{dist[nei],nei}); } }</pre>	$O((V+E) \log V)$	$O(V+E)$
Union Find (DSU)	Connectivity, merge components, dynamic graph	Build groups instead of traversing.	- Make each node its own parent - Find with path compression - Union by rank/size - Find to check connectivity	<pre>int find(int x) { if (parent[x]==x) return x; return find(parent[x]); } void union(int x,int y) { x=find(x); y=find(y); if (x==y) return; if (rank[x]<rank[y]) parent[x]=y; else if (rank[x]>rank[y]) parent[y]=x; else { parent[y]=x; rank[x]++; } }</pre>	$O(\alpha(V))$ per op	$O(V)$
Grid DFS / BFS	Islands, regions, surrounded, basin, matrix path, flood	Traverse 4 or 8 directions with bounds check.	- Check bounds & valid cell - Mark visited - Traverse directions - Count / collect / compare	<pre>int[][] dir4 = {{1,0},{-1,0},{0,1},{0,-1}}; int[][] dir8 = {{1,0},{-1,0},{0,1},{0,-1},{1,1},{1,-1},{-1,1},{-1,-1}}; boolean isValid(int r,int c) { return r>=0&&r<n&&c>=0&&c<m && !vis[r][c]; }</pre>	$O(V)$	$O(V)$

5 GRAPH REPRESENTATIONS



6 DIRECTIONS (GRID PROBLEMS)



7 EDGE TYPES

- Directed / Undirected → One way / Two way
- Weighted / Unweighted → Has cost / No cost
- Cyclic / Acyclic (DAG) → Has cycle / No cycle
- Connected / Disconnected → Single / Multiple comps
- Sparse / Dense → $E \ll V^2$ / $E \approx V^2$

8 COMPLEXITY SUMMARY

Algorithm	Time Complexity	Space Complexity
DFS	$O(V + E)$	$O(V)$
BFS	$O(V + E)$	$O(V)$
Topological Sort	$O(V + E)$	$O(V)$
Dijkstra (Min Heap)	$O((V + E) \log V)$	$O(V + E)$
Union Find (DSU)	$O(\alpha(V))$ per op	$O(V)$
Grid DFS / BFS	$O(V)$	$O(V)$

9 COMMON PITFALLS (DON'T LOSE MARKS!)

- ✗ Not marking visited → infinite loops
- ✗ Wrong data structure (Stack vs Queue vs PQ)
- ✗ Index out of bounds in grid problems
- ✗ Forgetting to handle disconnected components
- ✗ Ignoring edge direction
- ✗ Topological sort on cyclic graph
- ✗ Not checking if all nodes processed (cycle)
- ✗ Dijkstra with negative weights
- ✗ Union Find without path compression / rank
- ✗ Modifying list while iterating

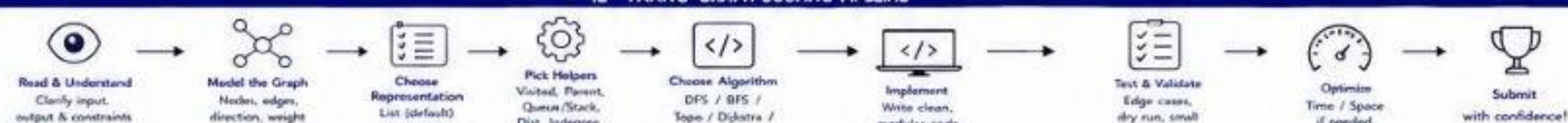
10 AHA MOMENTS (MEMORIZE THESE!)

- 💡 DFS → Go deep, then backtrack.
- 💡 BFS → First visit = shortest path (unweighted).
- 💡 Topo Sort → Process only after all prerequisites done.
- 💡 Dijkstra → Relax edges, pick next closest.
- 💡 DSU → Build groups, don't traverse.
- 💡 Grid → Direction + Bounds + Visit = Safe traversal.
- 💡 Disconnected → Run algorithm from every unvisited node.
- 💡 Cycle in Topo → If result size $< V$.
- 💡 Why PQ in Dijkstra? → Always take min distance.

11 INTERVIEW CHECKLIST

- ☑ Restate the problem
- ☑ Identify graph & entities
- ☑ Choose representation
- ☑ Pick helper structure
- ☑ Choose algorithm
- ☑ Handle edge cases
- ☑ Write clean code
- ☑ Analyze complexity
- ☑ Dry run / Test on cases
- ☑ Confirm output

12 FAANG GRAPH SOLVING PIPELINE



GOLDEN RULE
Master patterns,
write clean code,
crack FAANG!

REMEMBER

- ★ Patterns > Solutions
- ★ Think before you code
- ★ Handle edge cases
- ★ Simple code wins
- ★ Practice daily
- ★ Become pattern expert

YOU'VE GOT THIS!
Stay consistent.
Keep grinding. Win big!





1 DFS (Depth First Search)

Go deep first, then backtrack.

PROCESS (STEPS)

- 1 Mark current node visited.
- 2 Process the node (add to result, etc.).
- 3 Recur for all unvisited neighbors.
- 4 Backtrack when no unvisited neighbors remain.

JAVA TEMPLATE (Adjacency List)

```
void dfs(int node, List<List<Integer>> g, boolean[] vis) {  
    vis[node] = true; // mark  
    // process node here  
    for (int nei : g.get(node)) {  
        if (!vis[nei]) {  
            dfs(nei, g, vis);  
        }  
    }  
}
```

AHA

Go deep first, explore until no option left, then backtrack.

USE WHEN

- Explore all nodes
- Connected comps
- Cycle detection
- Topo Sort (DFS)
- Path existence

WATCH OUT

- Mark visited before recursion
- Stack overflow on deep graph
- Forgetting backtrack

COMPLEXITY

Time: $O(V + E)$
Space: $O(V)$
(Recursion Stack)

2 BFS (Breadth First Search)

Visit level by level using a queue.

PROCESS (STEPS)

- 1 Push start node in queue.
- 2 Mark visited.
- 3 While queue not empty:
 - Pop front
 - Process
 - Push all unvisited neighbors

JAVA TEMPLATE (Adjacency List)

```
void bfs(int src, List<List<Integer>> g) {  
    boolean[] vis = new boolean[g.size()];  
    Queue<Integer> q = new LinkedList<>();  
    q.offer(src); vis[src] = true;  
  
    while (!q.isEmpty()) {  
        int node = q.poll();  
        // process node here  
        for (int nei : g.get(node)) {  
            if (!vis[nei]) {  
                vis[nei] = true;  
                q.offer(nei);  
            }  
        }  
    }  
}
```

AHA

First visit to a node gives the shortest path (unweighted).

USE WHEN

- Shortest path (unweighted)
- Level order traversal
- Nearest node/level
- Minimum steps

WATCH OUT

- Mark visited when enqueueing (not when dequeuing)
- Forgetting to push all neighbors

COMPLEXITY

Time: $O(V + E)$
Space: $O(V)$
(Queue)

3 TOPOLOGICAL SORT (Kahn's BFS)

Process nodes after all prerequisites are satisfied.

PROCESS (STEPS)

- 1 Compute indegree of all nodes.
- 2 Push all nodes with indegree = 0.
- 3 While queue not empty:
 - Pop node
 - Add to result
 - Reduce indegree of neighbors
 - If indegree = 0, push to queue
- 4 If result size < V → cycle exists.

JAVA TEMPLATE (Kahn's Algorithm)

```
List<Integer> topoSort(int n, List<List<Integer>> g) {  
    int[] indeg = new int[n];  
    for (List<Integer> nbrs : g)  
        for (int v : nbrs) indeg[v]++;  
  
    Queue<Integer> q = new LinkedList<>();  
    for (int i = 0; i < n; i++)  
        if (indeg[i] == 0) q.offer(i);  
  
    List<Integer> order = new ArrayList<>();  
    while (!q.isEmpty()) {  
        int node = q.poll();  
        order.add(node);  
        for (int nei : g.get(node)) {  
            if (--indeg[nei] == 0) q.offer(nei);  
        }  
    }  
  
    return (order.size() == n) ? order : new ArrayList<>();  
}
```

AHA

Node enters result only after all its prerequisites are done.

USE WHEN

- DAG ordering
- Course schedule
- Build order
- Task dependencies

WATCH OUT

- If result size < V, there is a cycle
- Don't forget to reduce indegree

COMPLEXITY

Time: $O(V + E)$
Space: $O(V)$
(Queue + Array)

4 DIJKSTRA (Min Heap)

Relax edges. First time you pop a node, you have its shortest distance.

PROCESS (STEPS)

- 1 Init dist[] = INF, dist[src] = 0.
- 2 Push (0, src) in min-heap.
- 3 Pop min (d, node). If d > dist[node] → skip.
- 4 For each neighbor, relax edge: if newDist < dist[v] update & push.
- 5 Repeat until heap is empty.

JAVA TEMPLATE (PriorityQueue)

```
int[] dijkstra(int n, List<List<int[]>> g, int src) {  
    int[] dist = new int[n];  
    Arrays.fill(dist, Integer.MAX_VALUE);  
    dist[src] = 0;  
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));  
    pq.offer(new int[]{0, src}); // (dist, node)  
  
    while (!pq.isEmpty()) {  
        int[] cur = pq.poll();  
        int d = cur[0], node = cur[1];  
        if (d > dist[node]) continue; // outdated  
        for (int[] edge : g.get(node)) {  
            int nei = edge[0], wt = edge[1];  
            int nd = d + wt;  
            if (nd < dist[nei]) {  
                dist[nei] = nd;  
                pq.offer(new int[]{nd, nei});  
            }  
        }  
    }  
    return dist;  
}
```

AHA

Greedily pick the closest unprocessed node using min-heap.

USE WHEN

- Shortest path in weighted graph (positive weights)
- Network routing

WATCH OUT

- Works only for non-negative edges
- Skip outdated entry (d > dist[node])

COMPLEXITY

Time: $O((V + E) \log V)$
Space: $O(V)$
(Heap + Dist Array)

5 UNION FIND (DSU)

Build groups instead of traversing them.

PROCESS (STEPS)

- 1 Make each node its own parent.
- 2 Find: return root with path compression.
- 3 Union: merge by rank/size.

JAVA TEMPLATE (Path Compression + Union by Rank)

```
class DSU {  
    int[] parent, rank;  
    DSU(int n) {  
        parent = new int[n];  
        rank = new int[n];  
        for (int i = 0; i < n; i++) parent[i] = i;  
    }  
  
    int find(int x) {  
        if (parent[x] != x) parent[x] = find(parent[x]);  
        return parent[x];  
    }  
  
    void union(int x, int y) {  
        int px = find(x), py = find(y);  
        if (px == py) return;  
        if (rank[px] < rank[py]) parent[px] = py;  
        else if (rank[py] < rank[px]) parent[py] = px;  
        else { parent[py] = px; rank[px]++; }  
    }  
  
    boolean connected(int x, int y) { return find(x) == find(y); }  
}
```

AHA

Merge components without traversing them.

USE WHEN

- Connectivity check
- Merge components
- Dynamic graph
- Kruskal's MST

WATCH OUT

- Always use path compression
- Always union by rank/size

COMPLEXITY

Time: $O(\alpha(V))$ per op
Space: $O(V)$
(α = inverse Ackermann)

V = Vertices (Nodes) E = Edges $\alpha(V)$ = Inverse Ackermann ($\approx$ constant)

6 GRID DFS / BFS

For grid based problems (islands, regions, path, etc.)

PROCESS (STEPS)

- 1 Check bounds & valid cell.
- 2 Mark visited.
- 3 Traverse 4 or 8 directions.
- 4 Count / collect / compare.

JAVA TEMPLATE

```
// 4 Directions  
int[][] dir4 = {{1,0},{-1,0},{0,1},{0,-1}};  
// 8 Directions  
int[][] dir8 = {{1,0},{-1,0},{0,1},{0,-1},{1,1},{1,-1},{-1,1},{-1,-1}};  
  
boolean isValid(int r, int c, int n, int m) {  
    return r >= 0 && r < n && c >= 0 && c < m;  
}  
  
// BFS on grid example (4 directions)  
void bfsGrid(int sr, int sc, char[][] grid, boolean[][] vis) {  
    int n = grid.length, m = grid[0].length;  
    Queue<int[]> q = new LinkedList<>();  
    q.offer(new int[]{sr, sc}); vis[sr][sc] = true;  
    int[][] dir = {{1,0},{-1,0},{0,1},{0,-1}};  
    while (!q.isEmpty()) {  
        int[] cur = q.poll();  
        for (int[] d : dir) {  
            int nr = cur[0] + d[0], nc = cur[1] + d[1];  
            if (isValid(nr, nc, n, m) && !vis[nr][nc] && grid[nr][nc] == '1') {  
                vis[nr][nc] = true;  
                q.offer(new int[]{nr, nc});  
            }  
        }  
    }  
}
```

AHA

Grid problems = graph problems with constraints.

USE WHEN

- Islands / Regions
- Surrounded area
- Shortest path in grid
- Flood fill

WATCH OUT

- Bounds check always
- Mark visited immediately

COMPLEXITY

Time: $O(V + E)$
Space: $O(V)$
(V = n*m)

QUICK RECALL – ALGORITHM SELECTION GUIDE

Explore Everything
→ DFS

Shortest Path
(Unweighted)
→ BFS

Shortest Path
(Weighted +)
→ Dijkstra

Ordering / Dependencies
→ Topological Sort

Connectivity / Merge
→ Union Find

Grid Problems
→ Grid DFS / BFS



GOLDEN RULE
Know the pattern.
Pick the right tool.
Write clean code.
Crack FAANG!

1 GRAPH REPRESENTATIONS		
	ADJACENCY LIST (Preferred)	ADJACENCY MATRIX
Best for	Sparse graphs	Dense graphs
Space	$O(V + E)$	$O(V^2)$
Add Edge	$O(1)$	$O(1)$
Check Edge (u,v)	$O(\deg(u))$	$O(1)$
Iterate Neighbors	$O(\deg(u))$	$O(V)$
Memory Usage	Low	High
Example (Undirected)	<pre>Adj List 0: [1,2] 1: [0,3] 2: [0,3] 3: [1,2]</pre>	
Use When	Almost all interview problems	When graph is dense or need $O(1)$ edge check

2 DIRECTIONS (GRID MOVEMENTS)

4 DIRECTIONS
(Up, Down, Left, Right)

8 DIRECTIONS
(All Directions)

```
int[][] dir4 = {{-1,0},{1,0},{0,-1},{0,1}};
```

```
int[][] dir8 = {{-1,0},{1,0},{0,-1},{0,1},{-1,-1},{-1,1},{1,-1},{1,1}};
```

Use In: Grid problems, Islands, Flood Fill, Pacific Atlantic, Rotting Oranges, etc.

3 EDGE TYPES	
	Directed / Undirected One way / Two way
	Weighted / Unweighted Has cost / No cost
	Cyclic / Acyclic (DAG) Has cycle / No cycle
	Connected / Disconnected One comp / Multiple comps
	Sparse / Dense $E \ll V^2$ / $E = V^2$

4 COMPLEXITY SUMMARY		
Algorithm	Time Complexity	Space Complexity
DFS	$O(V + E)$	$O(V)$
BFS	$O(V + E)$	$O(V)$
Topological Sort (BFS)	$O(V + E)$	$O(V)$
Topological Sort (DFS)	$O(V + E)$	$O(V)$
Dijkstra (Min Heap)	$O((V + E) \log V)$	$O(V)$
Union Find (DSU)	$O(\alpha(V))$ per op	$O(V)$
Grid DFS / BFS	$O(M * N)$	$O(M * N)$
V = Vertices, E = Edges, $\alpha(V)$ = Inverse Ackermann ($\approx$ constant)		

5 HELPER DATA STRUCTURES	
Structure	Use For
Visited Array / Set	Avoid revisiting nodes
Queue (FIFO)	BFS, Level order, Topo Sort (Kahn's)
Stack / Recursion	DFS, Topo Sort (DFS)
Priority Queue (Min Heap)	Dijkstra, Prim's MST
Indegree Array	Topological Sort (Kahn's BFS)
Parent Array	DSU, Path reconstruction
Distance Array	Shortest path, Dijkstra
DSU (Parent, Rank/Size)	Connectivity & Components
Grid / Directions Array	Grid based problems

6 COMMON RETURN TYPES	
Type	Used In
boolean	Cycle exists, can finish, connected?
int	Count (components, islands, paths)
int[]	Distances, parents, order
List<Integer>	Topological order, path
List<List<Integer>>	Graph, adjacency list, result paths
Node	Clone graph, any node return
void	Modify in-place
Tip: Always read the return type first. It reveals the intention of the problem.	

7 COMMON BUGS (DON'T LOSE MARKS!)

Not marking visited at the right time.

Wrong data structure (Stack vs Queue vs PQ).

Index out of bounds in grid problems.

Forgetting to handle disconnected components.

Ignoring edge direction in directed graph.

Topological sort: not checking if all nodes processed.

Dijkstra: not skipping outdated entries from PQ.

Union Find: not using path compression / rank.

Off by one errors in loops and indices.

Modifying list while iterating.

Think before you code. Avoid careless mistakes!

8 CHOOSE REPRESENTATION

How to decide?

1 Is graph dense? YES → Use Adjacency Matrix

2 Need fast edge existence check? YES → Use Adjacency Matrix

3 Iterate neighbors frequently? YES → Use Adjacency List

Use Adjacency List (Default Choice)

In interviews, Adjacency List is correct 90% of the time.

9 GRID PROBLEM CHECKLIST

Check bounds at every move.

Mark visited before exploring neighbors.

Decide: 4 or 8 directions?

Use BFS when minimum steps/levels needed.

Use DFS when exploring all paths/regions.

Watch for obstacles / blocked cells.

Start from all sources if needed (multi-source BFS).

Grid problems are graphs in disguise!

10 PATTERN → ALGORITHM MAP				
Problem Pattern	Key Clue	Algorithm	Data Structure	Example Problems
Explore everything	Visit all nodes / components	DFS	Stack / Recursion	Number of Islands, Clone Graph, Graph Valid Tree
Minimum steps / level	Shortest path (unweighted)	BFS	Queue	Shortest Path, Rotting Oranges, Word Ladder
Ordering / Dependencies	Prerequisites / DAG	Topological Sort	Queue / Stack	Course Schedule, Alien Dictionary
Shortest path (weighted)	Has weights (positive)	Dijkstra	Min Heap (PQ)	Network Delay Time, Path With Minimum Effort
Merge / Connectivity	Need components / connections	Union Find (DSU)	Parent + Rank	Number of Provinces, Redundant Connection

11 INTERVIEW CHECKLIST	
Understand the problem & constraints. Identify it as graph & its pattern. Choose representation (List / Matrix). Choose helper data structure. Pick the right algorithm. Handle all edge cases. Mark visited correctly.	Check disconnected components. Validate output format. Optimize time & space. Dry run on small example. Test all cases. Write clean & modular code. Double check everything before submit.

12 GOLDEN RULES	
Solve the pattern, not the problem. Pick the right tool for the job. Think → Choose → Code → Test → Optimize. Simplicity beats complexity. Read the problem twice. Dry run before you code. Clean code = Easy to debug.	

13 KEY TAKEAWAY

Master the patterns.
Everything else is logic.
Think smart.
Code clean.
Crack FAANG!

1ALGORITHM DECISION TREE (How to choose the right algorithm)

```
graph TD
    Start([Start With Problem]) --> Q1{Is it a Graph Problem?}
    Q1 -- NO --> DS[Use other DS  
(Array, Tree, String...)]
    Q1 -- YES --> Q2{Do edges have weights?}
    Q2 -- NO --> Q3{Need shortest path?}
    Q3 -- YES --> BFS[BFS  
(Unweighted)]
    Q3 -- NO --> Q4{Need ordering / sequence?}
    Q4 -- YES --> TS1[Topological Sort  
(DAG only)]
    Q4 -- NO --> Q5{Need connectivity / components?}
    Q5 -- YES --> UF[Union Find (DSU)  
(Connectivity / Merge)]
    Q5 -- NO --> Q6{Explore all nodes / components?}
    Q6 -- YES --> DFS[DFS / BFS  
(Traversal / Components)]
    Q6 -- NO --> ReEval[Re-evaluate Problem Constraints]
```

Golden Rule: Understand the problem 80%, thinking 15%, coding only 5%.

2HIDDEN GRAPH PATTERN RECOGNITION

PATTERN TYPE	THINK OF	EXAMPLES
Grid / Matrix	2D cells connected 4 or 8 directions	Number of Islands Pacific Atlantic, Surrounded Regions Rotting Oranges, Word Search
Courses / Prerequisites	Dependencies / order	Course Schedule I / II Alien Dictionary
Friends / Social Network	People are nodes	Number of Provinces Accounts Merge, Clone Graph
Flights / Routes / Maps	Cities are nodes	Network Delay Time Cheapest Flights Within K Stops Path With Minimum Effort
Groups / Components	Find connected parts	Connected Components Redundant Connection
Flow / Water / Spread	Spread in steps	Pacific Atlantic Rotting Oranges
Dependencies / Order	Tasks / jobs depend	Course Schedule Build Order Problems
Shortest Path (Weighted)	Min cost / distance	Dijkstra Problems Minimum Cost Path
Dynamic Connectivity	Add / remove edges	Number of Islands II Dynamic Graph Problems

★ Tip: If you can draw nodes & edges, it's probably a graph!

3REPRESENTATION SELECTION GUIDE

```
graph TD
    Q1{Is graph dense?  
(E ≈ V^2)} -- YES --> AM1[Use Adjacency Matrix  
Better for dense graphs]
    Q1 -- NO --> Q2{Need fast edge existence  
check (u,x) frequently?}
    Q2 -- YES --> AM2[Use Adjacency Matrix  
O(1) check]
    Q2 -- NO --> Q3{Iterate neighbors  
of a node often?}
    Q3 -- YES --> AL[Use Adjacency List  
Better iteration]
    Q3 -- NO --> Default[Default Choice -> Adjacency List  
(Space efficient & flexible)]
```

4HELPER STRUCTURE SELECTION

IF YOU NEED TO...	USE	WHY?
Traverse all nodes deeply	Stack / Recursion	DFS, Backtracking
Traverse level by level	Queue (FIFO)	BFS, Level order
Get minimum quickly	Priority Queue (Min Heap)	Dijkstra, Prim's MST
Group / Merge components	Union Find (DSU)	Efficient connectivity
Store visited nodes	Visited Array / Set	Avoid cycles / repeats
Store parent / path	Parent Array / Map	Reconstruction
Store distances	Distance Array / Map	Shortest path
Store indegree	Array / Map	Topological Sort

5ALGORITHM CHOOSER AT A GLANCE

DFS (Depth First Search)	BFS (Breadth First Search)	TOPOLOGICAL SORT (Kahn's BFS / DFS)	DIJKSTRA (Min Heap)	UNION FIND (DSU) (Disjoint Set Union)	GRID DFS / BFS (Matrix Problems)
USE WHEN - Explore all nodes - Find components - Backtracking - Cycle detection (directed) - Topological Sort (DFS)	USE WHEN - Shortest path (unweighted) - Level order traversal - Minimum steps / levels - Find components	USE WHEN - DAG ordering - Course schedule - Build order - Task dependencies	USE WHEN - Shortest path - Weighted graph - Positive weights - Single source	USE WHEN - Connectivity check - Components count - Merge groups - Dynamic connectivity	USE WHEN - Islands / regions - Flood fill - Surrounded regions - Shortest path in grid
TIME O(V + E)	TIME O(V + E)	TIME O(V + E)	TIME O((V + E) log V)	TIME O(α(V)) per op	TIME O(M * N)
SPACE O(V) Recursion	SPACE O(V) Queue	SPACE O(V)	SPACE O(V)	SPACE O(V)	SPACE O(M * N)

6FAANG GRAPH SOLVING PIPELINE (Follow this every time)

- 1. READ & UNDERSTAND**
 - Read carefully
 - Identify what is asked
 - List entities & constraints
- 2. MODEL AS GRAPH**
 - Define Nodes
 - Define Edges
 - Decide direction & weight
- 3. CHOOSE REPRESENTATION**
 - Adjacency List / Matrix
 - List preferred (default)
 - Consider constraints
- 4. PICK HELPER STRUCTURES**
 - Visited, Queue, Stack
 - Dist, Parent, Indegree
 - DSU, PQ (if needed)
- 5. CHOOSE ALGORITHM**
 - DFS / BFS / Topo / Dijkstra / DSU
 - Match with pattern
- 6. CODE & TEST**
 - Implement clean code
 - Dry run small cases
 - Handle edge cases
- 7. OPTIMIZE & FINALIZE**
 - Check complexity
 - Optimize if needed
 - Submit with confidence

7KEY NOTATIONS

V = Number of Vertices (Nodes)
E = Number of Edges
O(V) = Linear in vertices
O(E) = Linear in edges
α(V) = Inverse Ackermann (= constant)

8COMPLEXITY SNAPSHOT

Algorithm	Time Complexity	Space Complexity
DFS	O(V + E)	O(V)
BFS	O(V + E)	O(V)
Topo Sort (Kahn's BFS)	O(V + E)	O(V)
Topo Sort (DFS)	O(V + E)	O(V)
Dijkstra (Min Heap)	O((V + E) log V)	O(V)
Union Find (DSU)	O(α(V)) per op	O(V)
Grid DFS / BFS	O(M * N)	O(M * N)

9BIGGEST INTERVIEW PITFALLS

- ✗ Not marking visited → infinite loop
- ✗ Wrong data structure (Stack vs Queue vs PQ)
- ✗ Index out of bounds in grid
- ✗ Forgetting disconnected components
- ✗ Ignoring edge direction in directed graph
- ✗ Topo sort: not all nodes processed (cycle)
- ✗ Dijkstra with negative weights
- ✗ Union Find without path compression / rank
- ✗ Modifying list while iterating

10REMEMBER THIS!

- 💡 Pattern recognition is king.
- 💡 Draw the graph (mentally or on paper).
- 💡 Choose the right tool.
- 💡 Simplicity beats cleverness.
- 💡 Write clean & modular code.
- 💡 Test all cases.
- 💡 Stay calm. Think. Solve. Conquer.

YOU GOT THIS! CRACK FAANG! 🚀



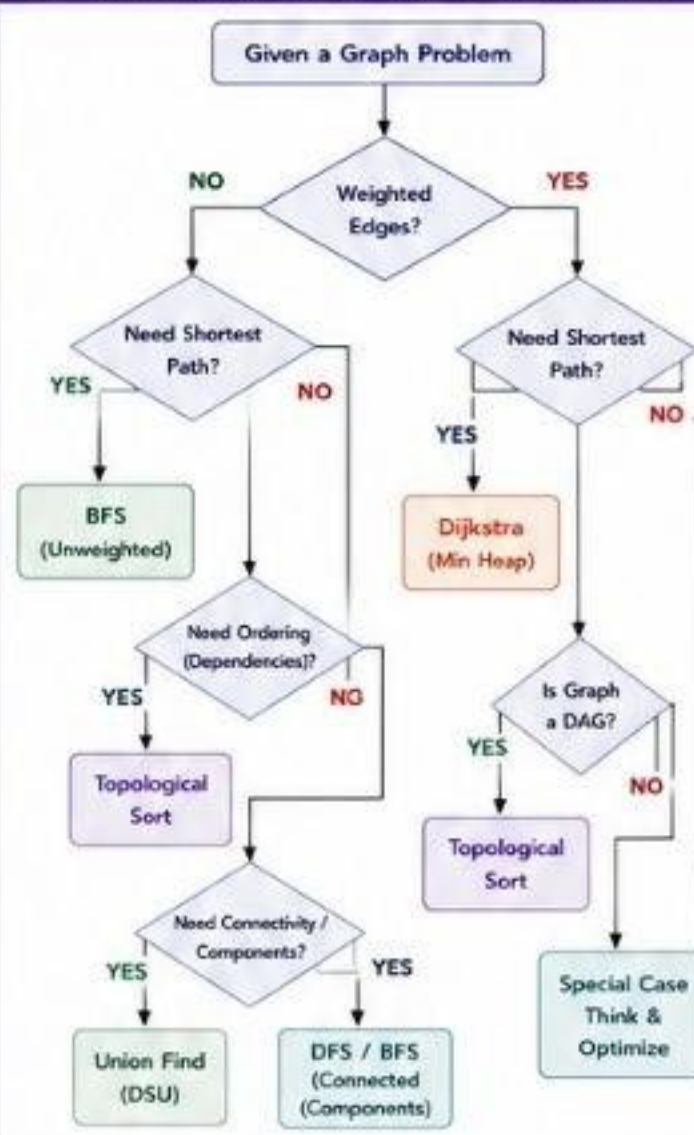
1 AHA MOMENTS – REMEMBER THESE!

- Graph = Connections between entities.
Node = Entity, Edge = Relationship.
- Most problems are pattern based.
Recognize pattern → Select algorithm.
- Representation matters!
Adj List for sparse, Matrix for dense / fast edge check.
- Think in terms of traversal.
Explore, Order, Shortest Path, Connectivity.
- Weight decides your path.
Unweighted → BFS, Weighted → Dijkstra.
- Dependencies need order.
Use Topological Sort for DAG.
- Groups & Merges?
Use Union Find (DSU).
- In grid problems → it is still a graph!
Cells are nodes, moves are edges.

2 PATTERN RECOGNITION MAP

PROBLEM CLUE	THINK	USE
2D cells, directions, islands, regions, flood	Grid Graph	BFS / DFS (Grid)
Courses, tasks, prerequisites	Dependencies / Order	Topological Sort
Friends, accounts, network, social	Connectivity / Components	DFS / BFS / Union Find
Flights, routes, maps, cities	Shortest Path	BFS (unweighted) Dijkstra (weighted)
Water flow, spread, rotting oranges	Multi-source BFS	BFS (Multi-source)
Provinces, redundant connection	Connected Components	Union Find (DSU)
Find order, sequence, valid ordering	Topological Ordering	Topological Sort
Cheapest path, min cost	Minimum Cost Path	Dijkstra (Min Heap)

3 ALGORITHM SELECTION CHEAT SHEET



4 TEMPLATE RECALL (WHAT TO WRITE)

Algorithm	What to Write First
DFS	Visited array + dfs(node)
BFS	Queue + visited + bfs(start)
Topological Sort	Indegree (or DFS stack)
Dijkstra	MinHeap + dist[]
Union Find (DSU)	parent[] + rank/size[]
Grid DFS/BFS	Directions + bounds check

Tip: Don't memorize code.
Memorize the skeleton & modify.

5 COMPLEXITY QUICK RECALL

Algorithm	Time Complexity	Space Complexity
DFS	$O(V + E)$	$O(V)$
BFS	$O(V + E)$	$O(V)$
Topological Sort (BFS)	$O(V + E)$	$O(V)$
Topological Sort (DFS)	$O(V + E)$	$O(V)$
Dijkstra (Min Heap)	$O((V + E) \log V)$	$O(V)$
Union Find (DSU)	$O(\alpha(V))$ per op	$O(V)$
Grid DFS / BFS	$O(M * N)$	$O(M * N)$

$\alpha(V)$ is inverse Ackermann function ($\approx$ constant)

6 COMMON BUGS (DON'T LOSE MARKS!)

- ✗ Not marking visited at the right time.
- ✗ Forgetting to handle disconnected components.
- ✗ Wrong data structure (Stack vs Queue vs PQ).
- ✗ Index out of bounds in grid problems.
- ✗ Ignoring edge direction in directed graph.
- ✗ Topo sort: not checking all nodes processed.
- ✗ Dijkstra: not skipping outdated entries from PQ.
- ✗ Union Find: not using path compression / rank.
- ✗ Off by one errors in loops and indices.
- ✗ Modifying list while iterating.

Avoid these = Easy AC + Happy Interviewer!

7 DATA STRUCTURES QUICK RECALL

Data Structure	Use For
Array / List	Adjacency List, Visited, Dist
Queue (FIFO)	BFS, Level Order, Multi-source BFS
Stack / Recursion	DFS, Topo Sort (DFS)
Priority Queue (Min Heap)	Dijkstra, Prim's MST
Map / HashMap	Graph with labels, Count
Set	Visited, Unique nodes
Union Find (DSU)	Connectivity, Components

8 RETURN TYPES – THINK FIRST

Return Type	Use When
boolean	Existence, Valid / Invalid, Possible / Not Possible
int	Count, Number of components, Steps, Levels
int[]	Distances, Parents, Order
List<Integer>	Order, Path, Component
List/List<Integer>>	All paths, Adjacency list, Components
Node	Clone graph, Any node return
void	Modify in-place, Just traversal

Read return type first. It reveals the problem intent.

9 MINI CHECKLIST BEFORE SUBMIT

- ✓ Have I understood the problem & constraints?
- ✓ Have I identified the graph pattern?
- ✓ Have I chosen correct representation?
- ✓ Have I selected the right algorithm?
- ✓ Have I handled all edge cases?
- ✓ Have I marked visited properly?
- ✓ Have I checked time & space complexity?
- ✓ Have I tested with small examples?
- ✓ Have I validated the output format?
- ✓ Clean code + Good variable names?

10 FAANG GRAPH SOLVING PIPELINE (REPEAT EVERY TIME)



GOLDEN RULE: PATTERN RECOGNITION + RIGHT TOOL + CLEAN CODE = FAANG SUCCESS!



THINK LIKE AN ENGINEER. SOLVE LIKE A MASTER. CRACK FAANG!

